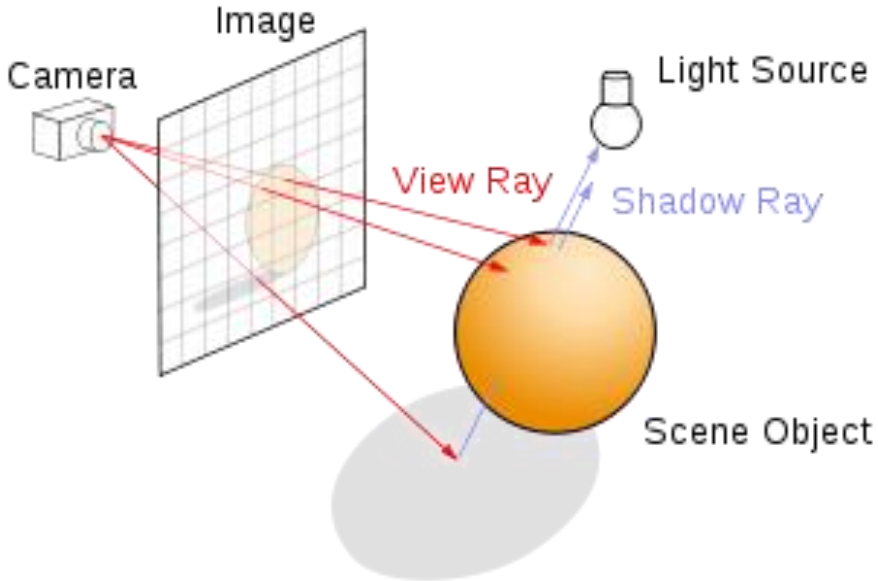
COMP 4300
Intro to Game Programming

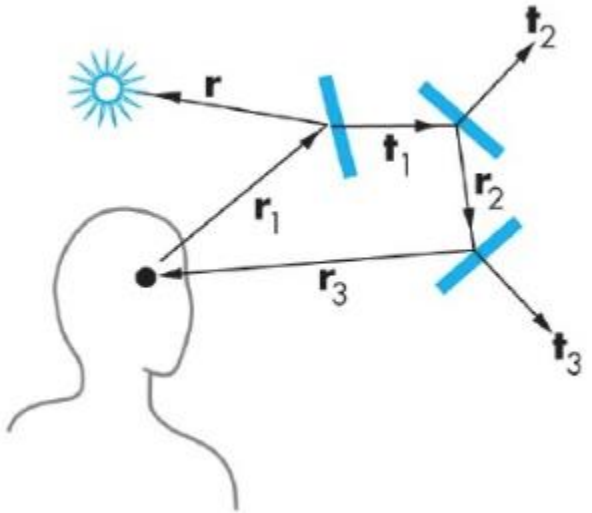
Lecture 12
2D Ray Casting
Line Segment Intersection

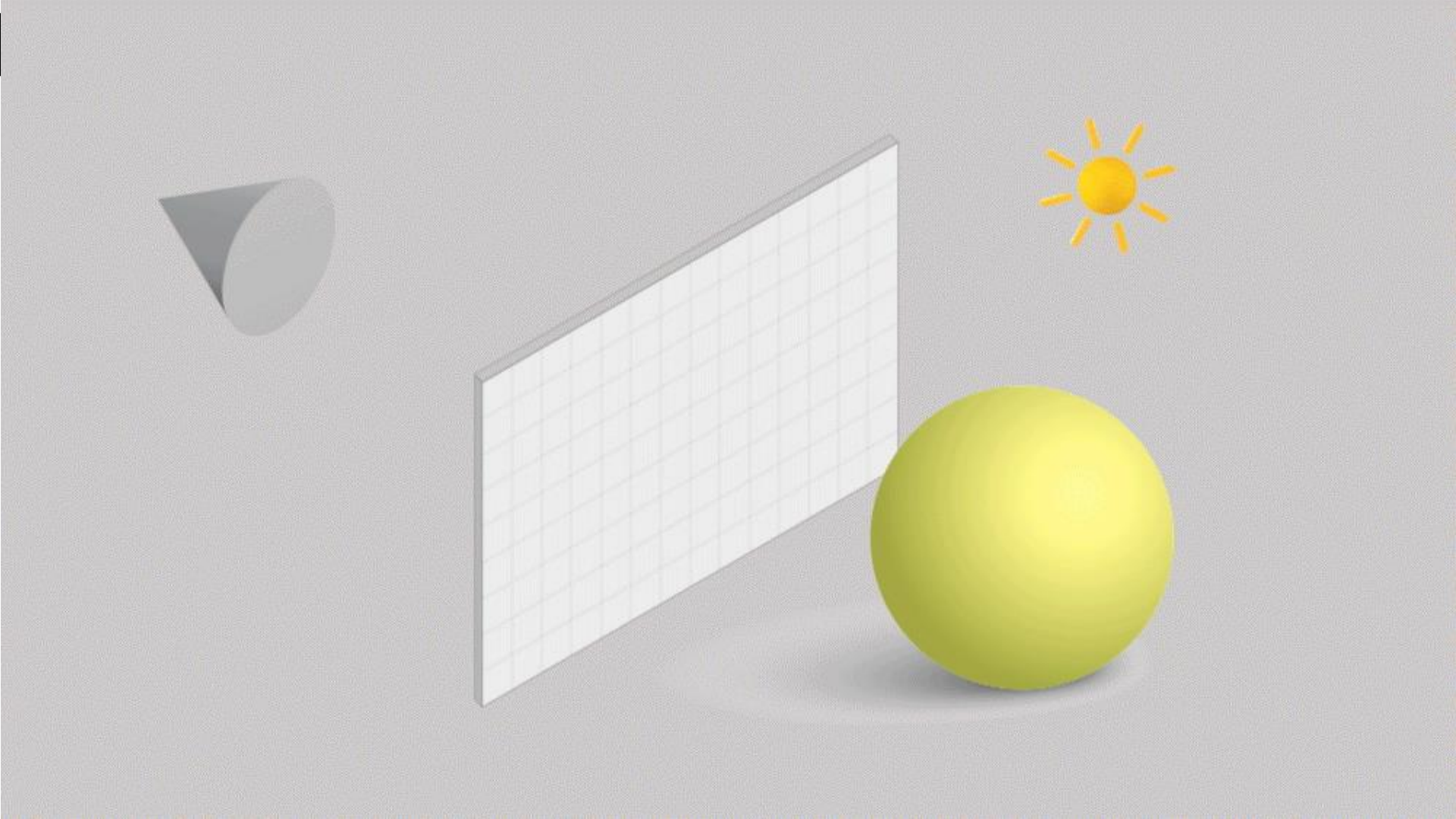
What is Ray Casting

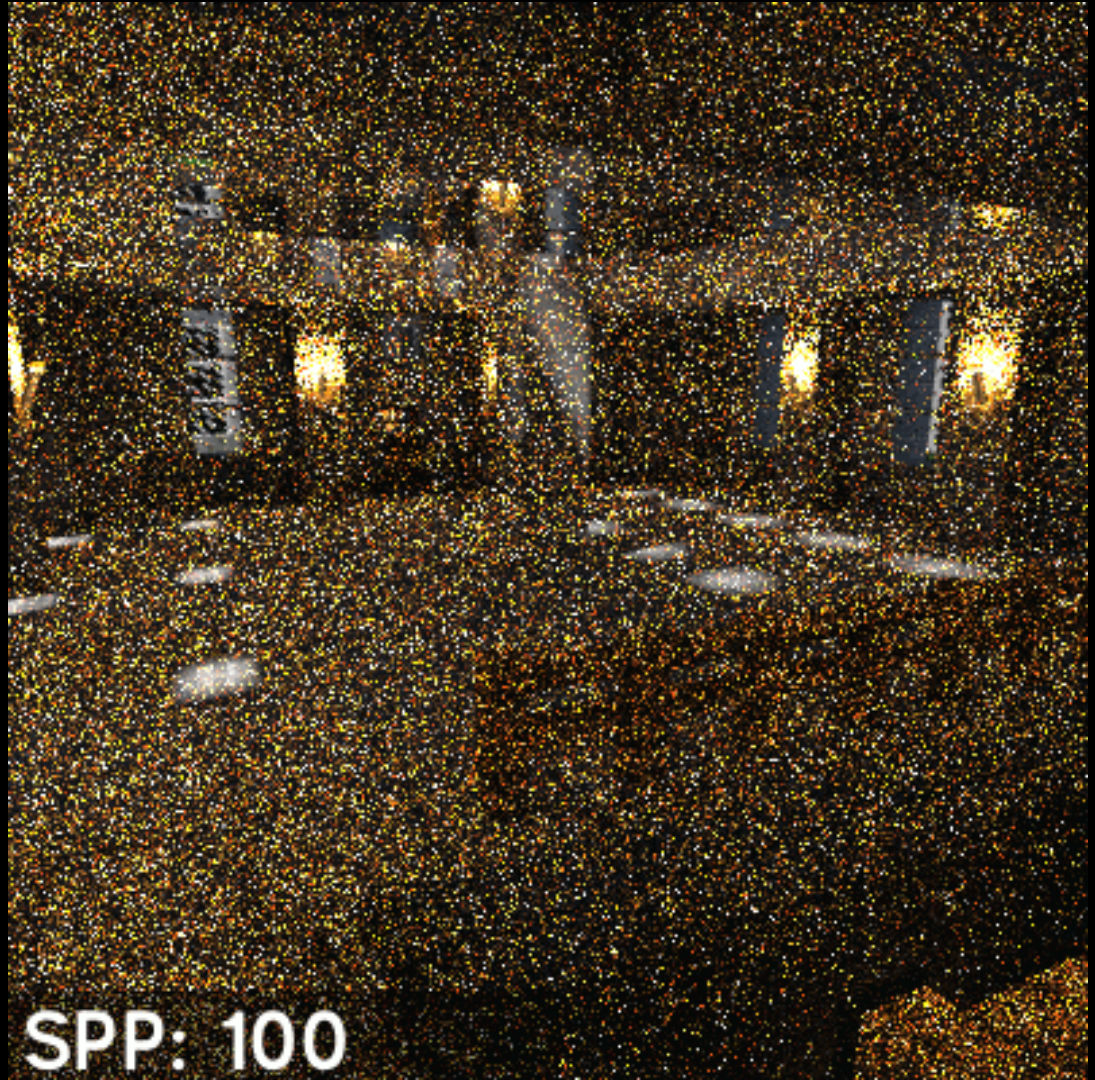
- Ray Casting is the name for a technique which has many applications in games
- Ray Tracing = Iterative Ray Casting
- Intuitively: “Cast rays (lines) in a given direction, where do they intersect/reflect”
- Very heavy on vector math, but has many useful applications

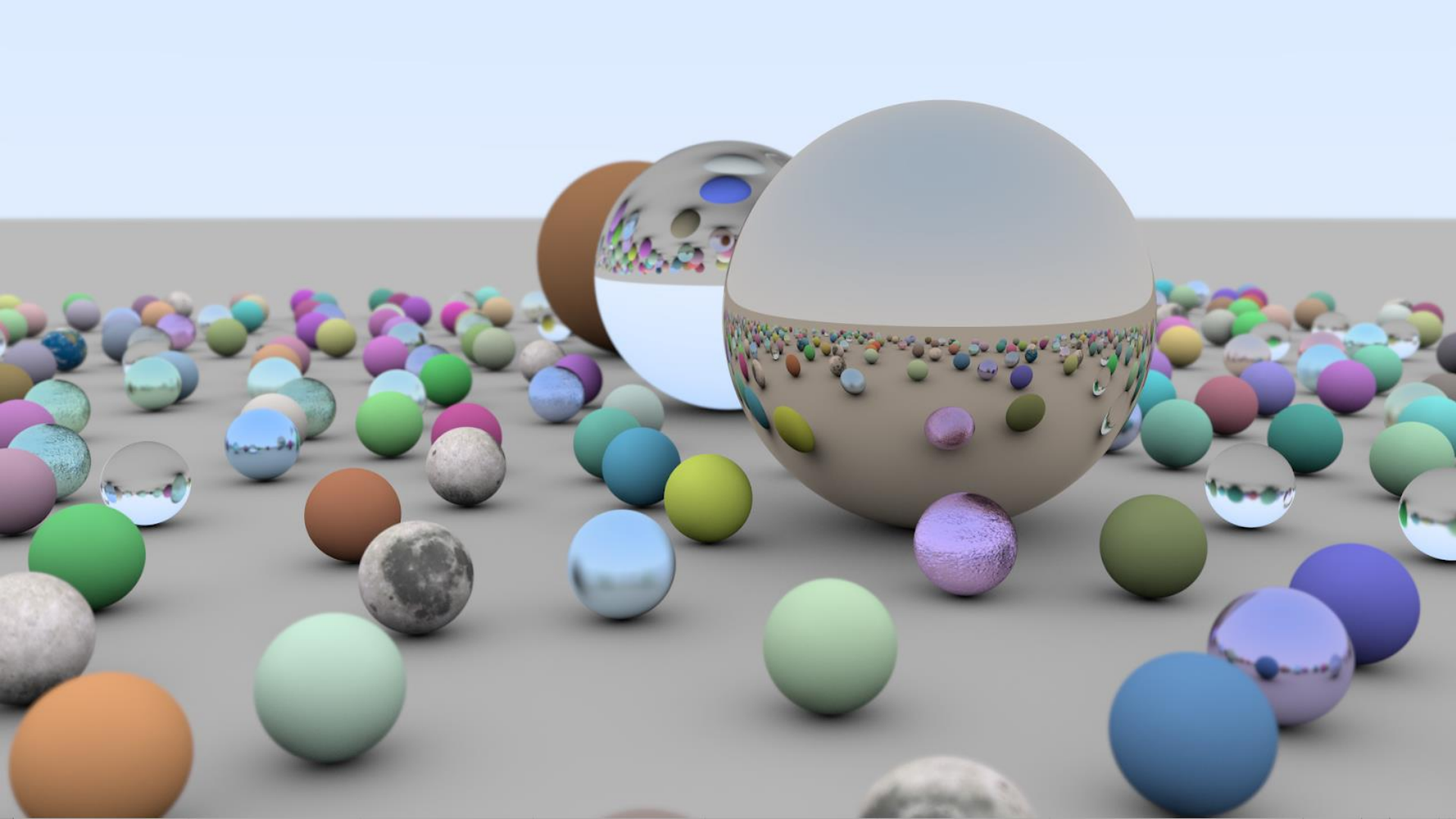
Ray Casting / Tracing







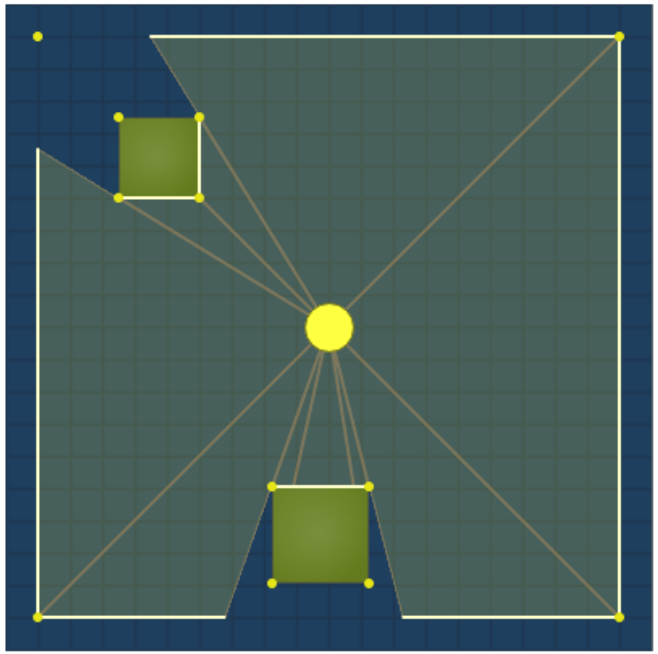

SPP: 100

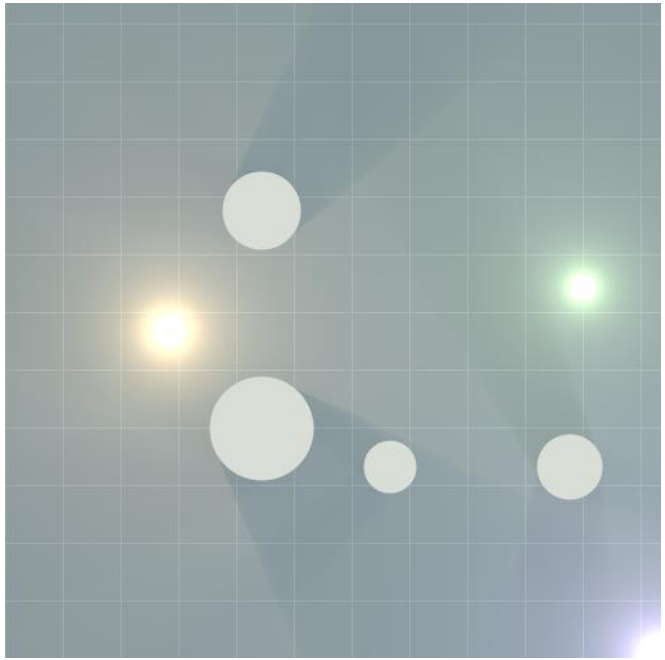


Ray Casting

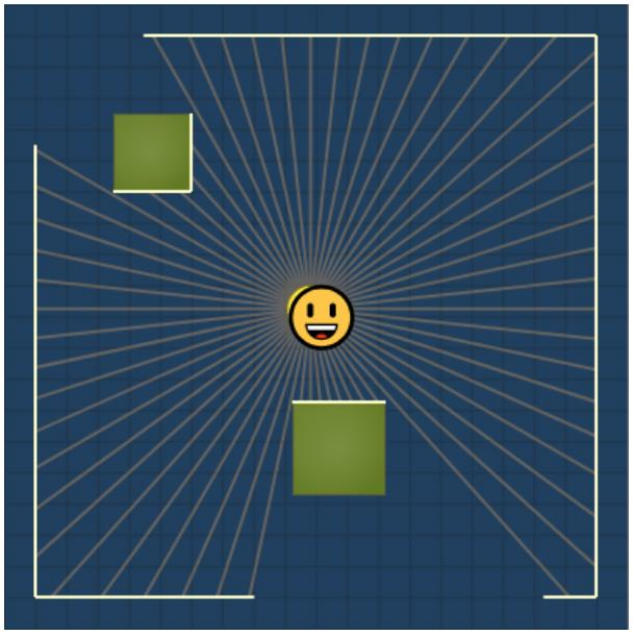
- Draw a line between two points
- Detect if and where a collision occurs
- Ray Tracing = Where did it ‘bounce’ to?
- If no collision occurs on a line between two points, then they can ‘see’ each other
- 2D Ray Casting primarily solves problems that deal with light / visibility

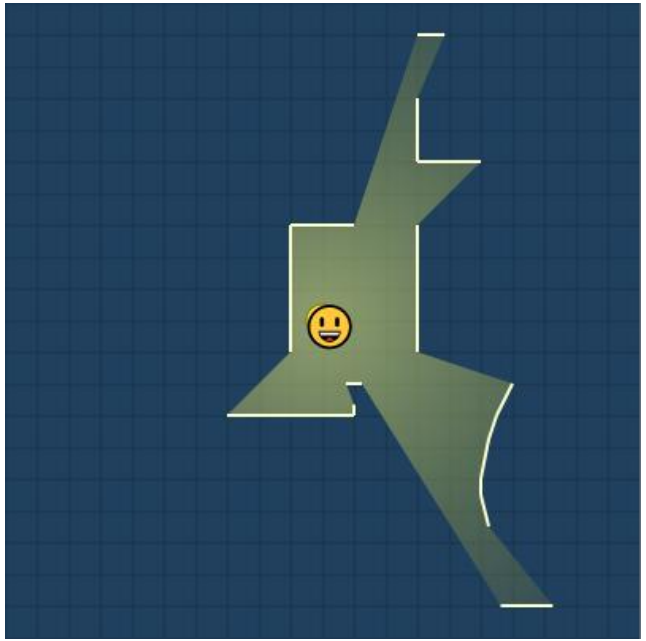
2D Ray Casting - Lighting





2D Ray Casting - Visibility



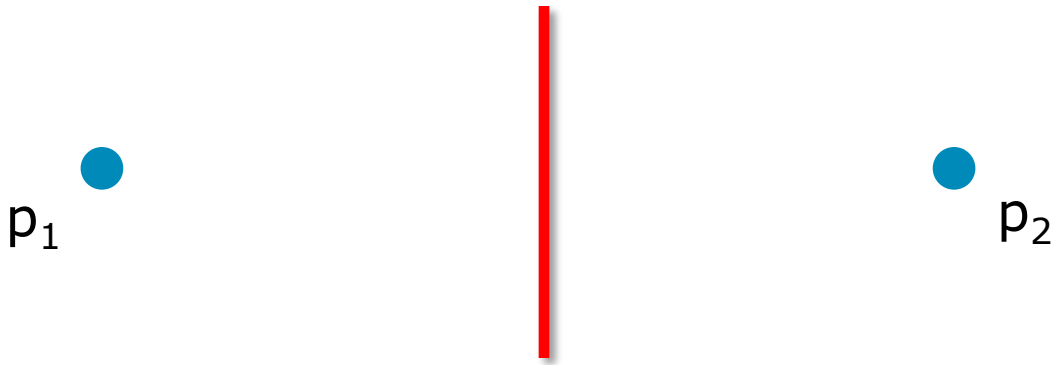


https://www.redblobgames.com/articles/visibility/

Third Eye Crime – Moonshot Games

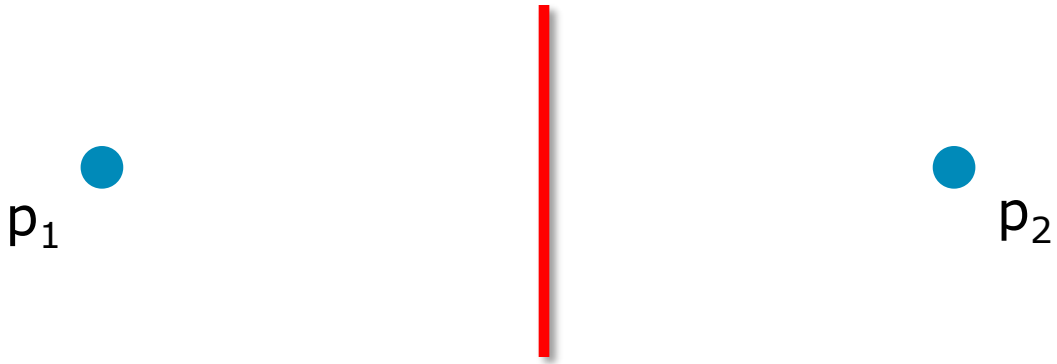
Ray Casting Basics

- Ray Casting asks ‘is anything blocking the path between these two 2D points’



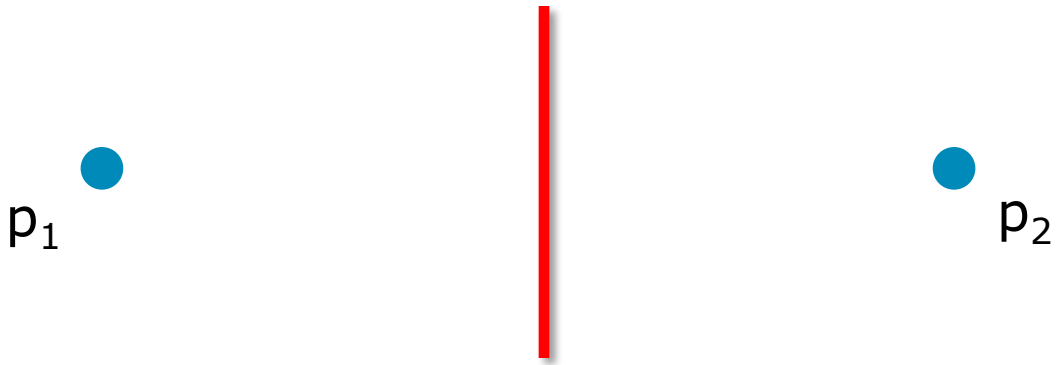
Ray Casting Basics

- Ray Casting asks ‘is anything blocking the path between these two 2D points’
- How do we detect this mathematically?



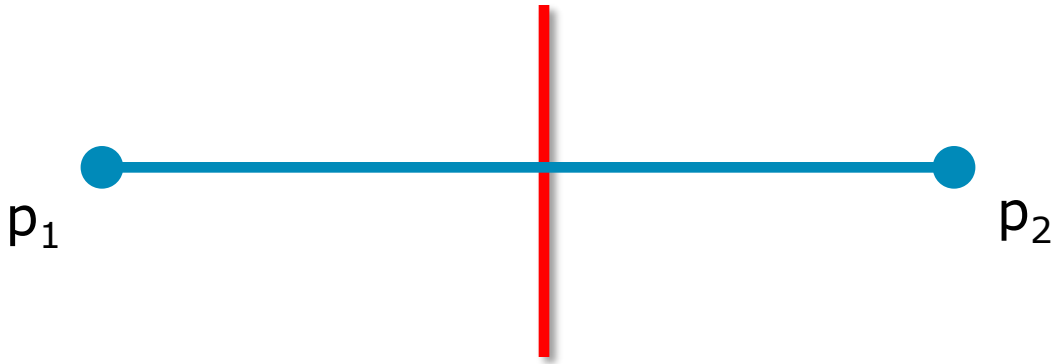
Line Segment Intersection

- The ‘path’ between two points is ‘blocked’ if the line segment between them intersects with another existing line segment



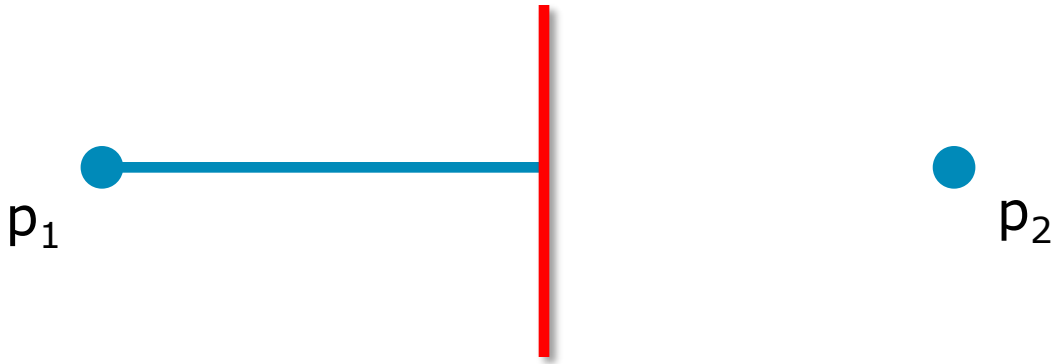
Line Segment Intersection

- The ‘path’ between two points is ‘blocked’ if the line segment between them intersects with another existing line segment



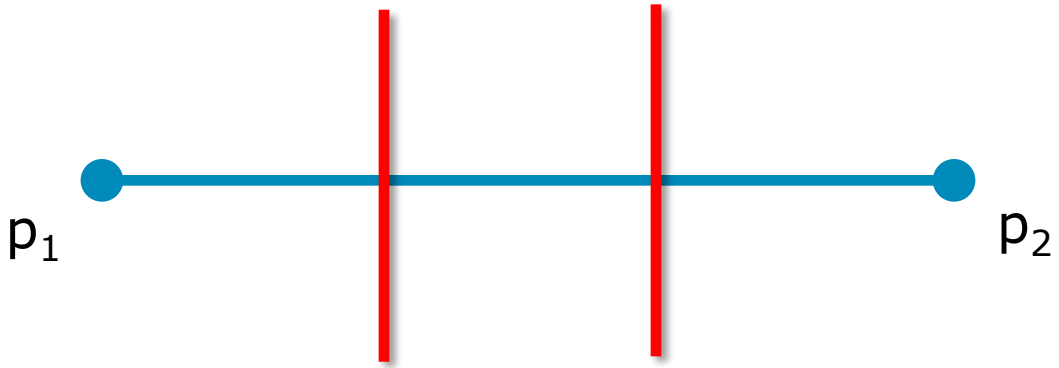
Line Segment Intersection

- The ‘path’ between two points is ‘blocked’ if the line segment between them intersects with another existing line segment



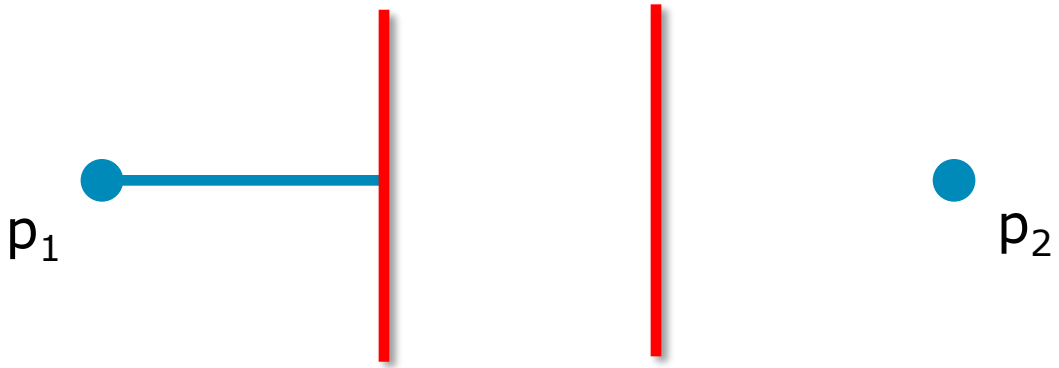
Line Segment Intersection

- Depending on the application, we may also want to know the ‘first’ such intersection, and exactly where it occurred



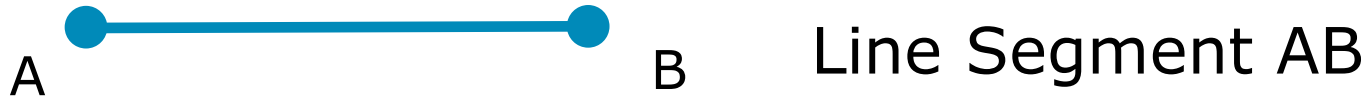
Line Segment Intersection

- Depending on the application, we may also want to know the ‘first’ such intersection, and exactly where it occurred



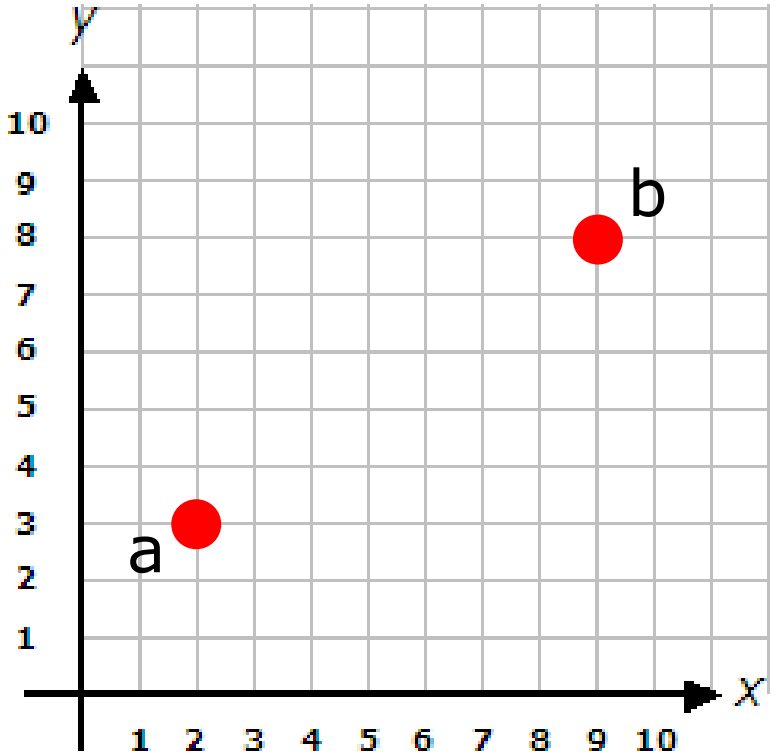
Line Segment Intersection

- So far in the course, we have been using the Vec2 class to represent points in 2D space as a 2D Vector with (x, y)
- A line segment can be defined by the start and end point of a line using Vec2



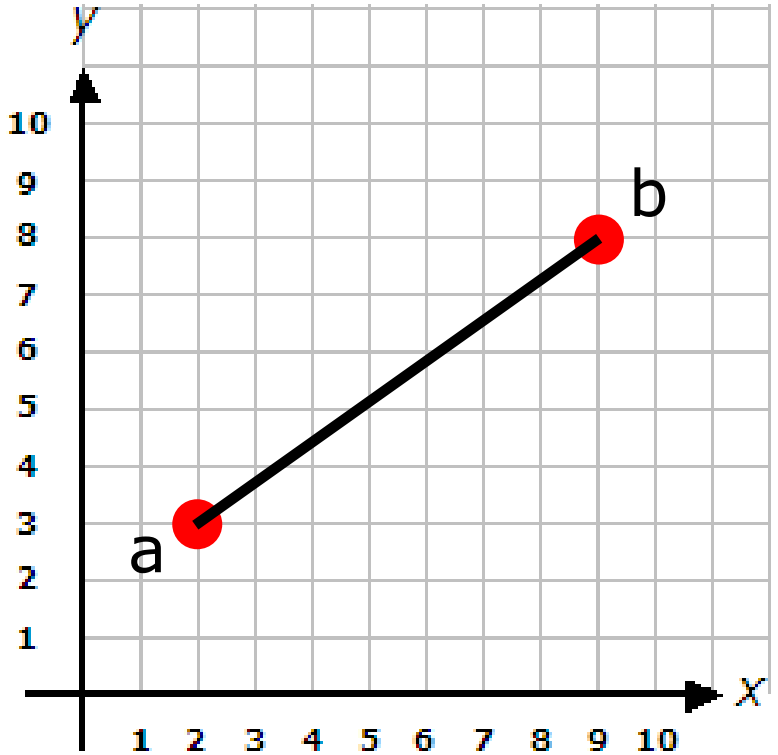
Line Segments

- Points a, b
  - Stored as Vec2



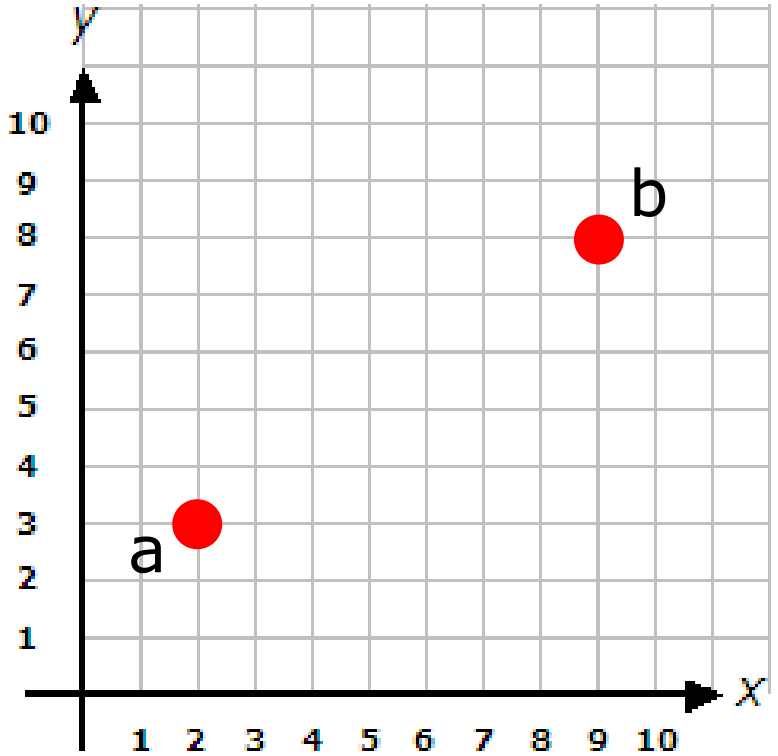
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab



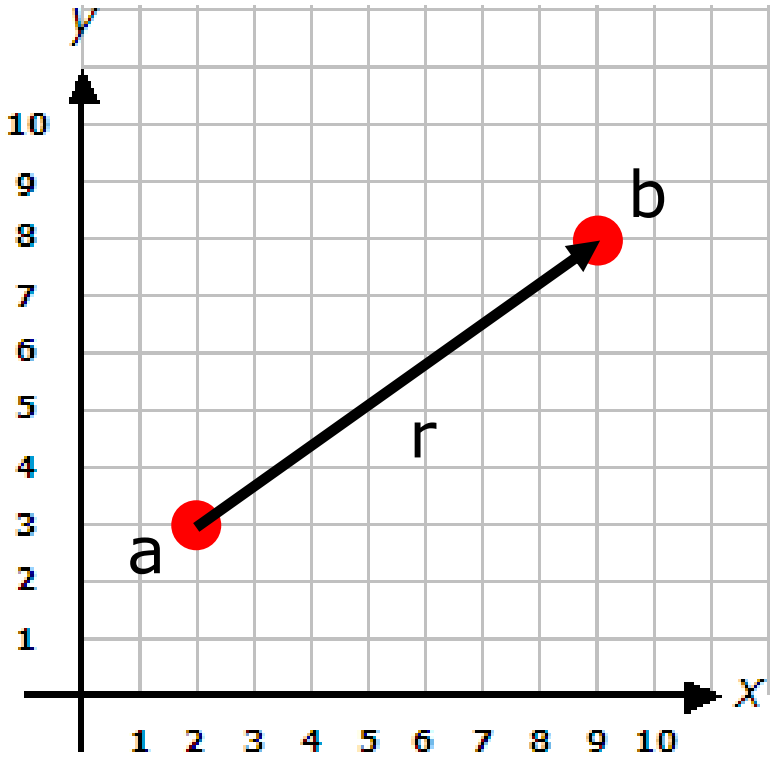
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract



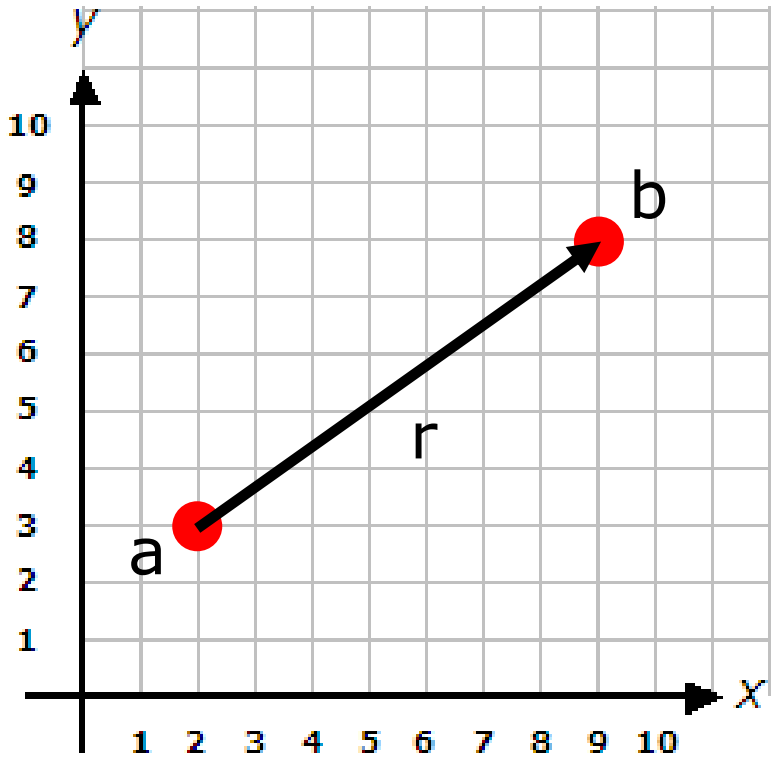
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract
- r = b – a
  - So: b = a + r



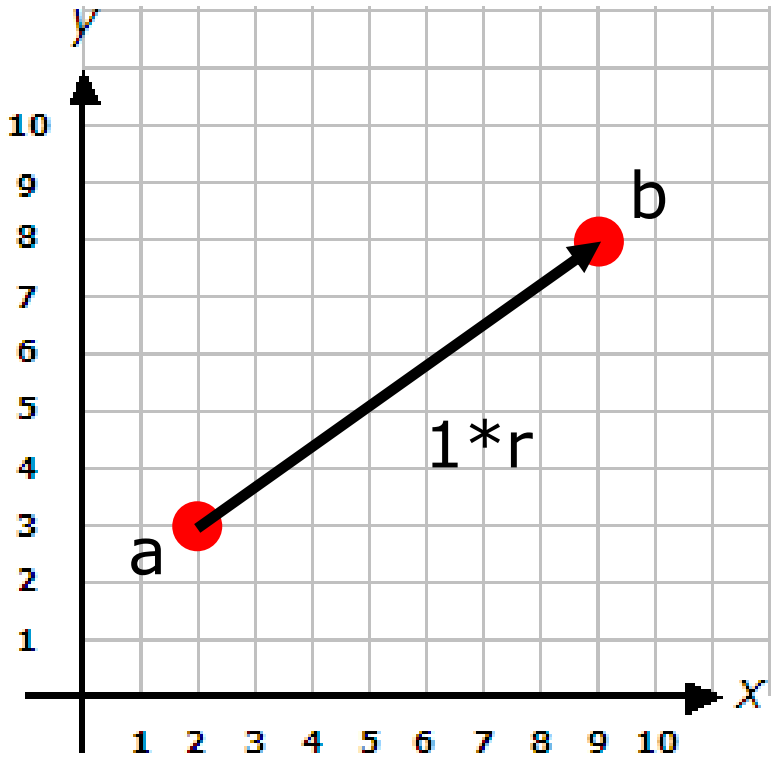
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract
- r = b – a
  - So: b = a + r
- Let's scale r by a scalar



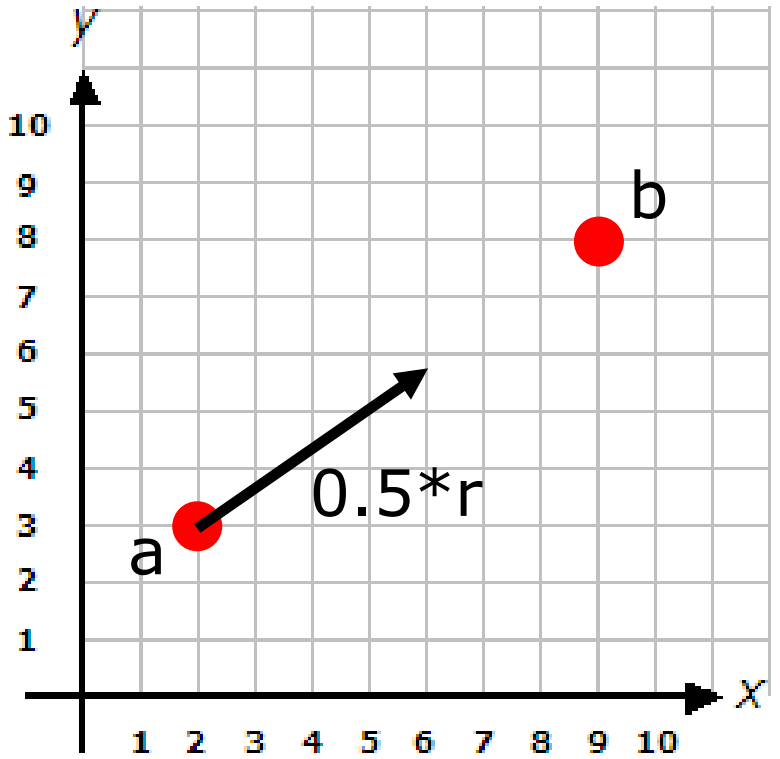
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract
- r = b – a
  - So: b = a + r
- Let's scale r by a scalar



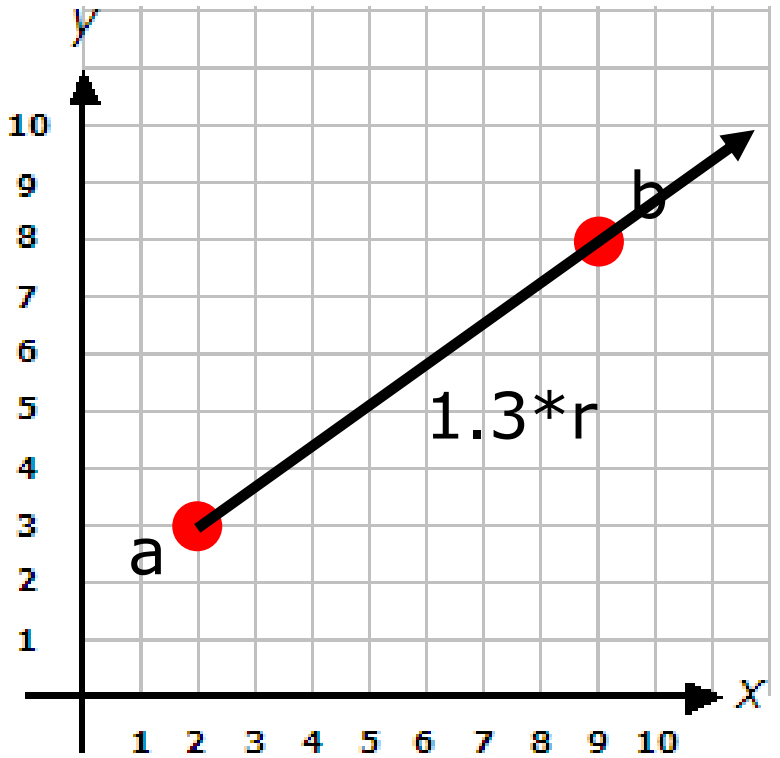
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract
- r = b – a
  - So: b = a + r
- Let's scale r by a scalar



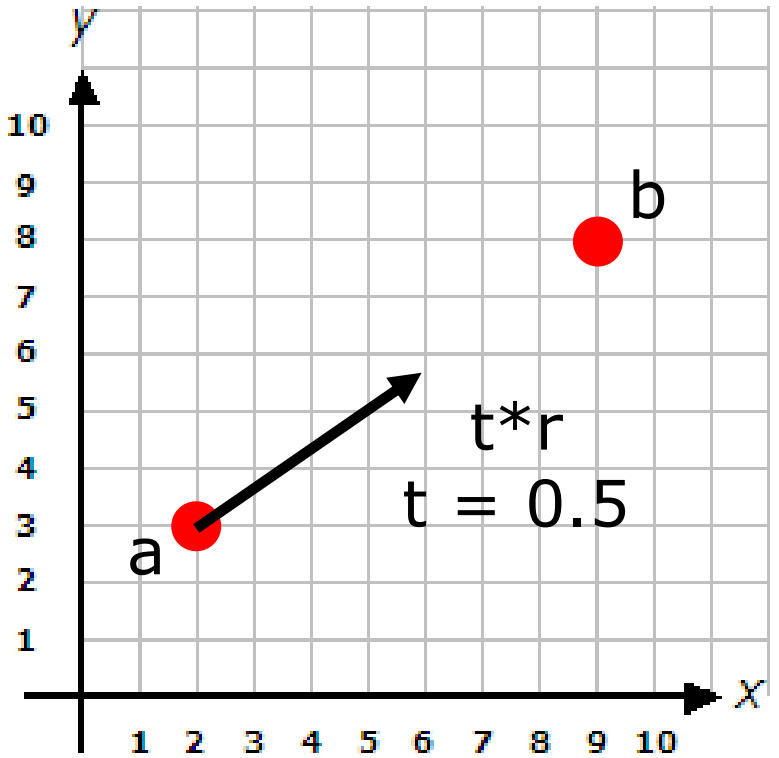
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract
- r = b – a
  - So: b = a + r
- Let's scale r by a scalar



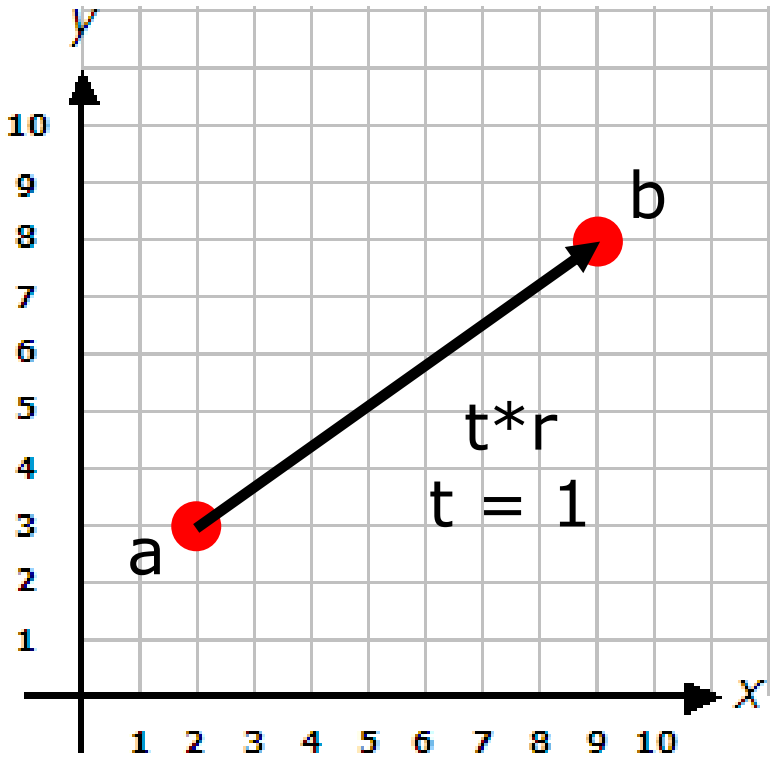
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract
- r = b – a
  - So: b = a + r
- Let's scale r by scalar t



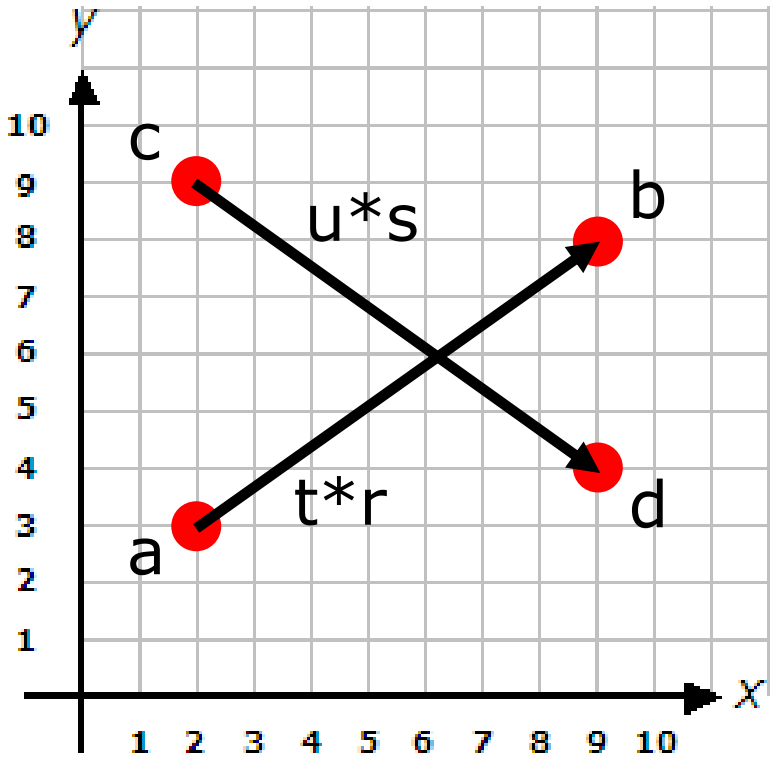
Line Segments

- Points a, b
  - Stored as Vec2
- Line segment ab
- Let's vector subtract
- r = b – a
  - So: b = a + r
- b = a + t*r (t=1)



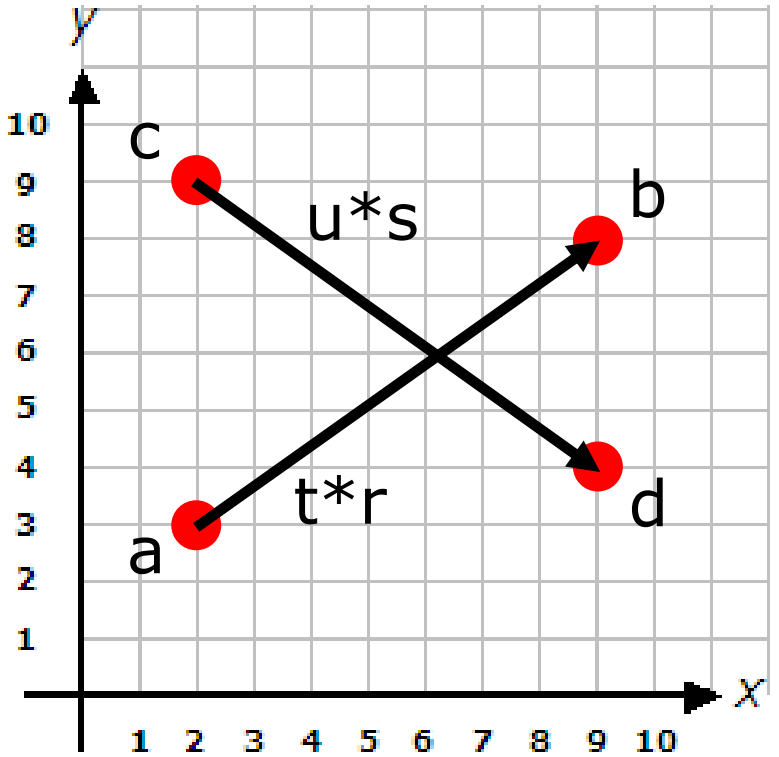
Line Segments

- Add another line
- Points c, d
- s = d – c
- Scale s by scalar u
- When r and s have correct lengths, the segments intersect



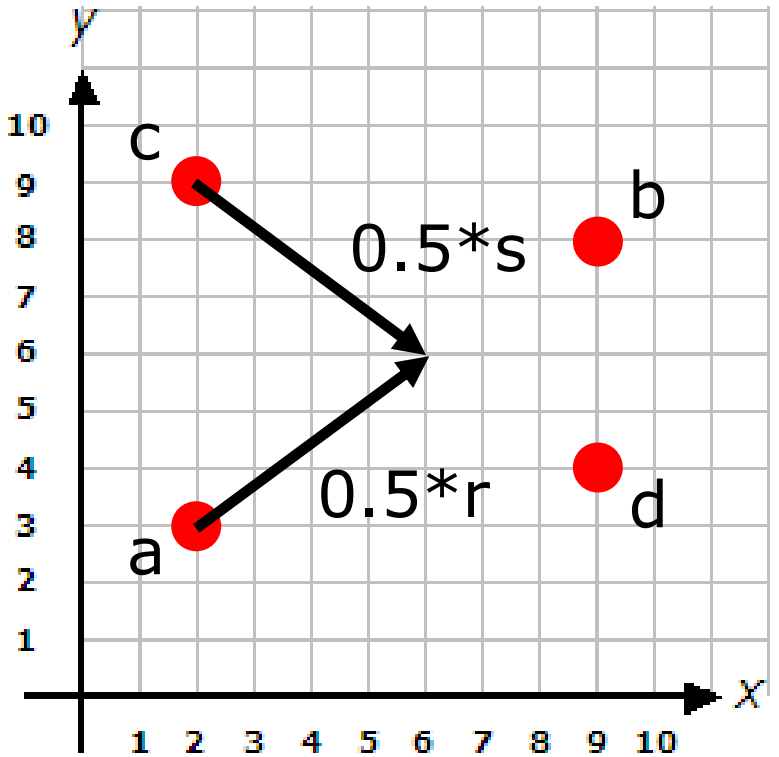
Line Seg Intersect

- Find the correct scale values for r and s that make them meet



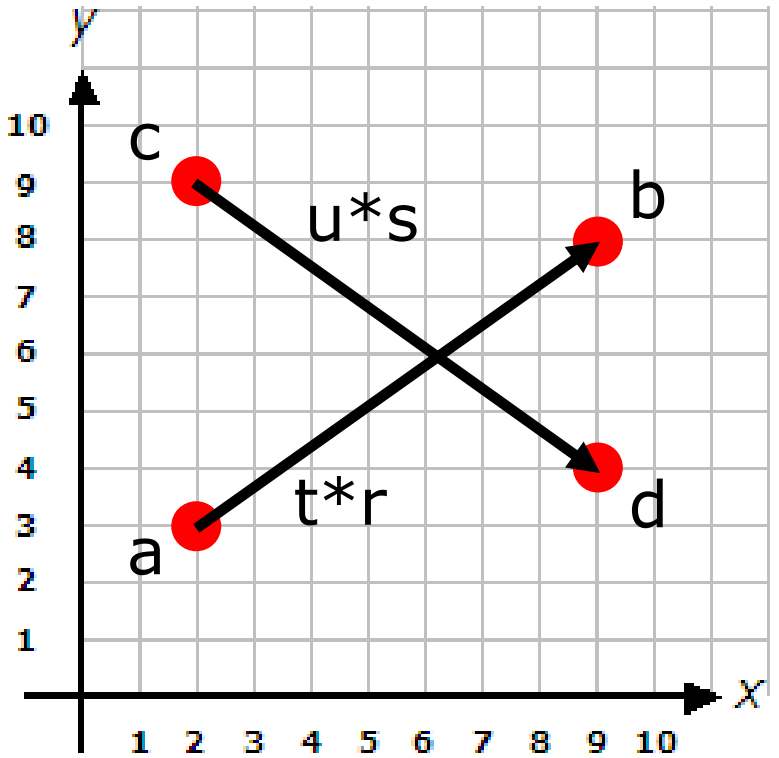
Line Seg Intersect

- Find the correct scale values for u and r that make them meet
- Example:
  - t = 0.5, u = 0.5



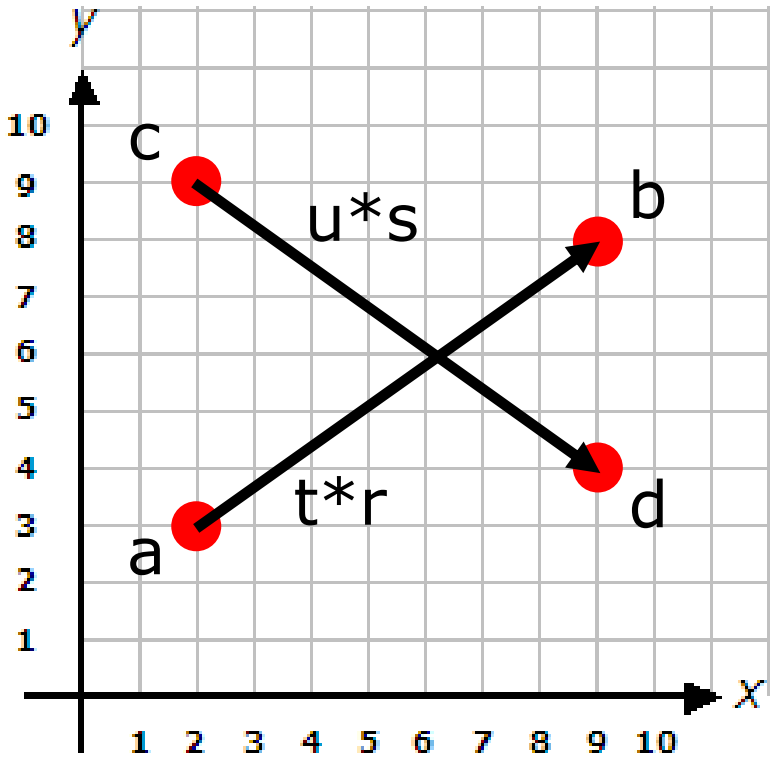
Line Seg Intersect

- b = a + t*r
- d = c + u*s
- Lines intersect when b and d are equal
- a + t*r = c + u*s
- Lines intersect if both t and u between [0,1]



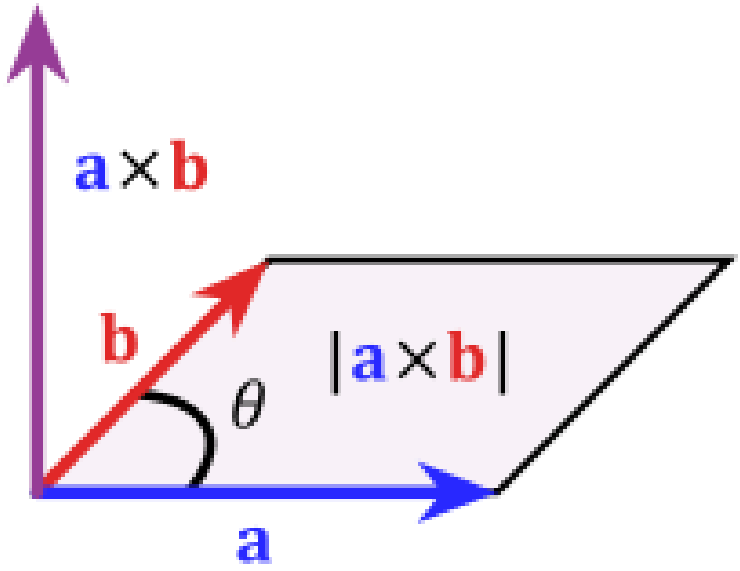
Line Seg Intersect

- a + t*r = c + u*s
- How to solve for two unknowns t and u?
- '2D Cross Product' to the rescue!



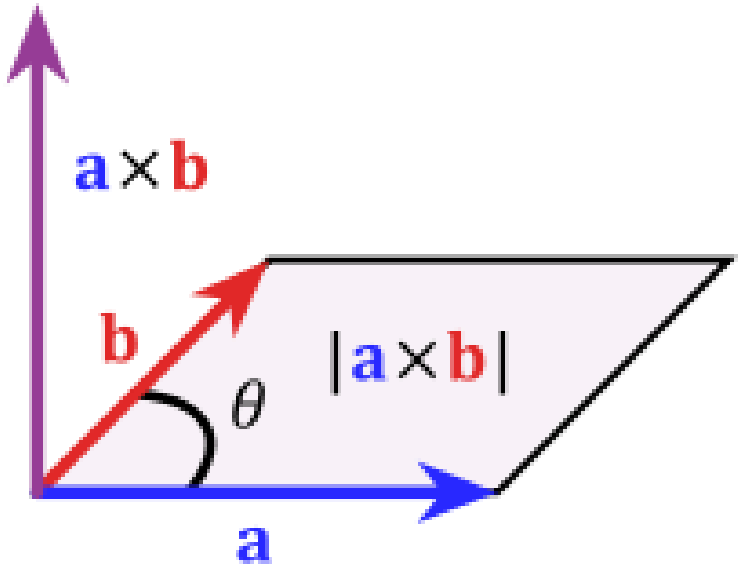
3D Cross Product

- Cross Product a × b
- Produces a 3D vector which is orthogonal to both vec a and b
- 'Normal Vector' to a,b
- Doesn't make intuitive sense when applied to 2D vectors



2D Cross Product

- 2D cross product
- a × b =
  - a.x*b.y - a.y*b.x
- Produces a scalar
- Important properties
  - a × a = 0
  - (a+r) × s = a × s + r × s (distributive)



Intersection: Solve for t

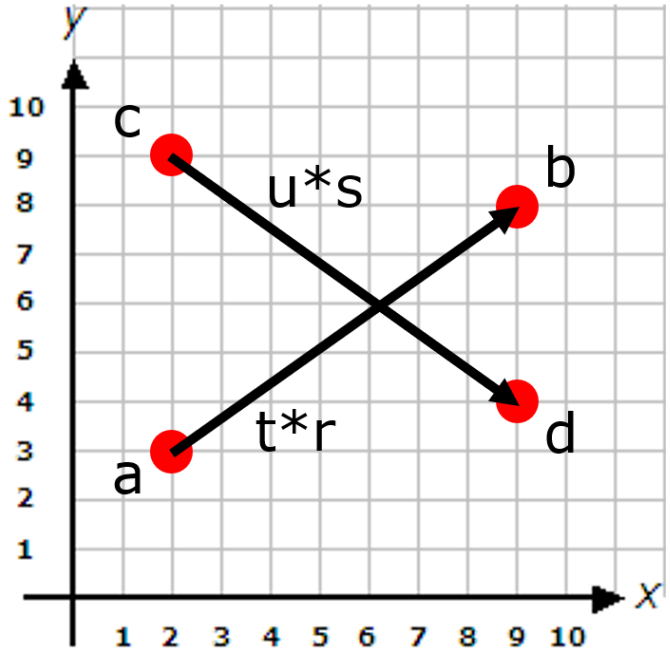
- $a + t*r = c + u*s$
- Apply cross product s both sides
- $(a + t*r) \times s = (c + u*s) \times s$
- $a \times s + t(r \times s) = c \times s + u(s \times s)$
- $a \times s + t(r \times s) = c \times s$
- $t\,(r \times s) = c \times s - a \times s = (c-a) \times s$
- $t = ((c-a) \times s) \,/\, (r \times s)$

Intersection: Solve for u

- $a + t*r = c + u*s$
- Apply cross product r both sides
- $(a + t*r) \times r = (c + u*s) \times r$
- $a \times r + t(r \times r) = c \times r + u(s \times r)$
- $a \times r = c \times r + u(s \times r)$
- $a \times r - c \times r = u(s \times r)$
- $u = ((a-c) \times r) / (s \times r)$

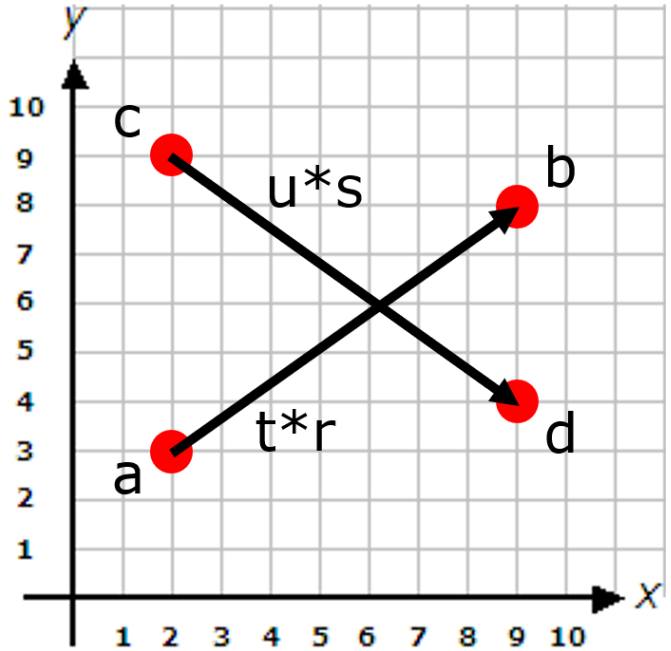
Intersection Solve

- t = ((c-a) × s) / (r × s)
- u = ((a-c) × r) / (s × r)
- Another property:
  - (r × s) = -(s × r)
- Rearrange above
  - u = ((a-c) × r) / (s × r)
  - u = ((c-a) × r) / (r × s)



Optimized Version

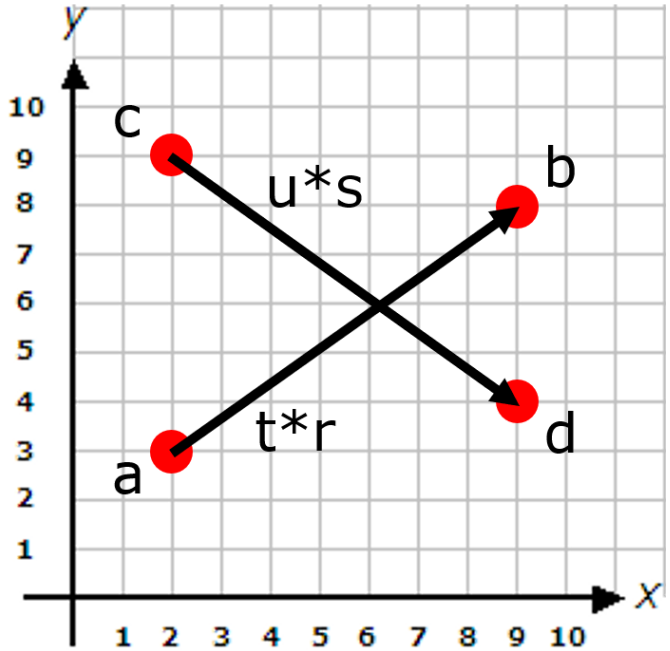
- t = ((c-a) × s) / (r × s)
- u = ((c-a) × r) / (r × s)
- (c-a) used twice
- (r × s) used twice



- Compute them only once!

Final Version

- t = ((c-a) × s) / (r × s)
- u = ((c-a) × r) / (r × s)
- Intersection if:
  - t ∈ [0, 1] and
  - u ∈ [0, 1]
- Intersection point:
  - p = a + t*r or
  - P = c + u*s

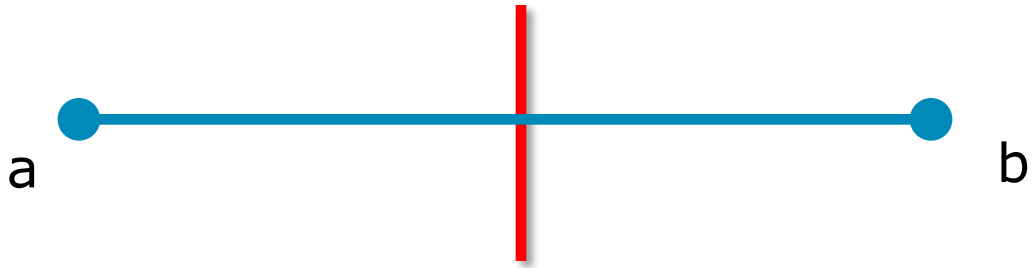


Line Segment Intersection

```
1.  struct Intersect { bool result; Vec2 pos; };
2.  Intersect LineIntersect(Vec2: a, b, c, d)
3.      Vec2 r = (b – a);
4.      Vec2 s = (d – c);
5.      float rxs = r×s;
6.      Vec2 cma = c – a;
7.      float t = (cma×s) / rxs;
8.      float u = (cma×r) / rxs;
9.      if (t >= 0 && t <= 1 && u >= 0 && u <= 1)
10.         return { true,  Vec2(a.x + t*r.x, a.y + t*r.y) };
11.     else
12.         return { false, Vec2(0,0) };
```

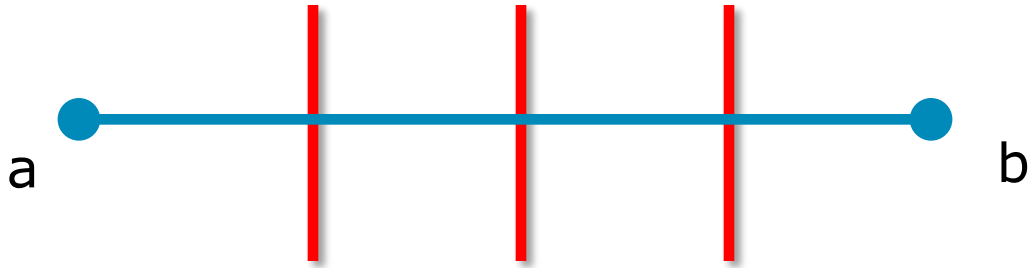
2D Game Visibility

- Now that we know how to do line segment intersection, how do we use it for visibility?
- If line segment ab does not collide with any vision-blocking entities, b is visible to a



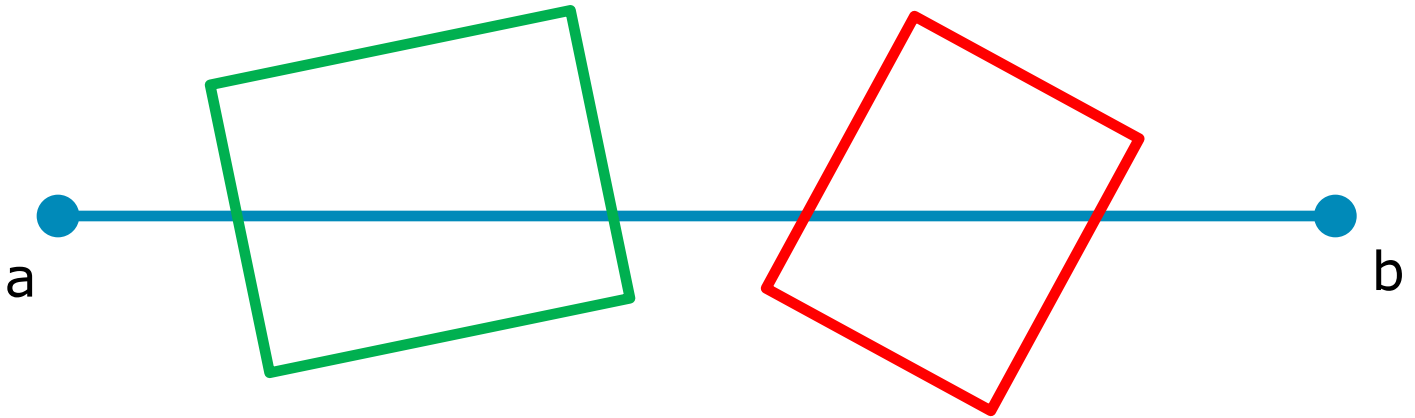
2D Game Visibility

- Now that we know how to do line segment intersection, how do we use it for visibility?
- If line segment ab does not collide with any vision-blocking entities, b is visible to a



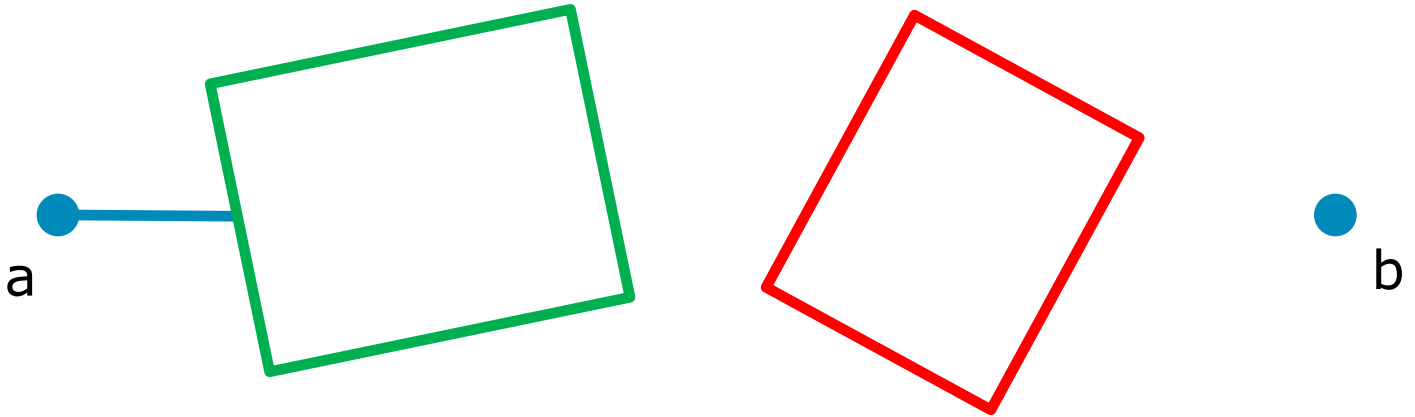
2D Game Visibility

- What about non-line game objects?
- Easy: they are made up of line segments



2D Game Lighting

- Entity can see up to the closest intersection
- Anything beyond that is outside vision



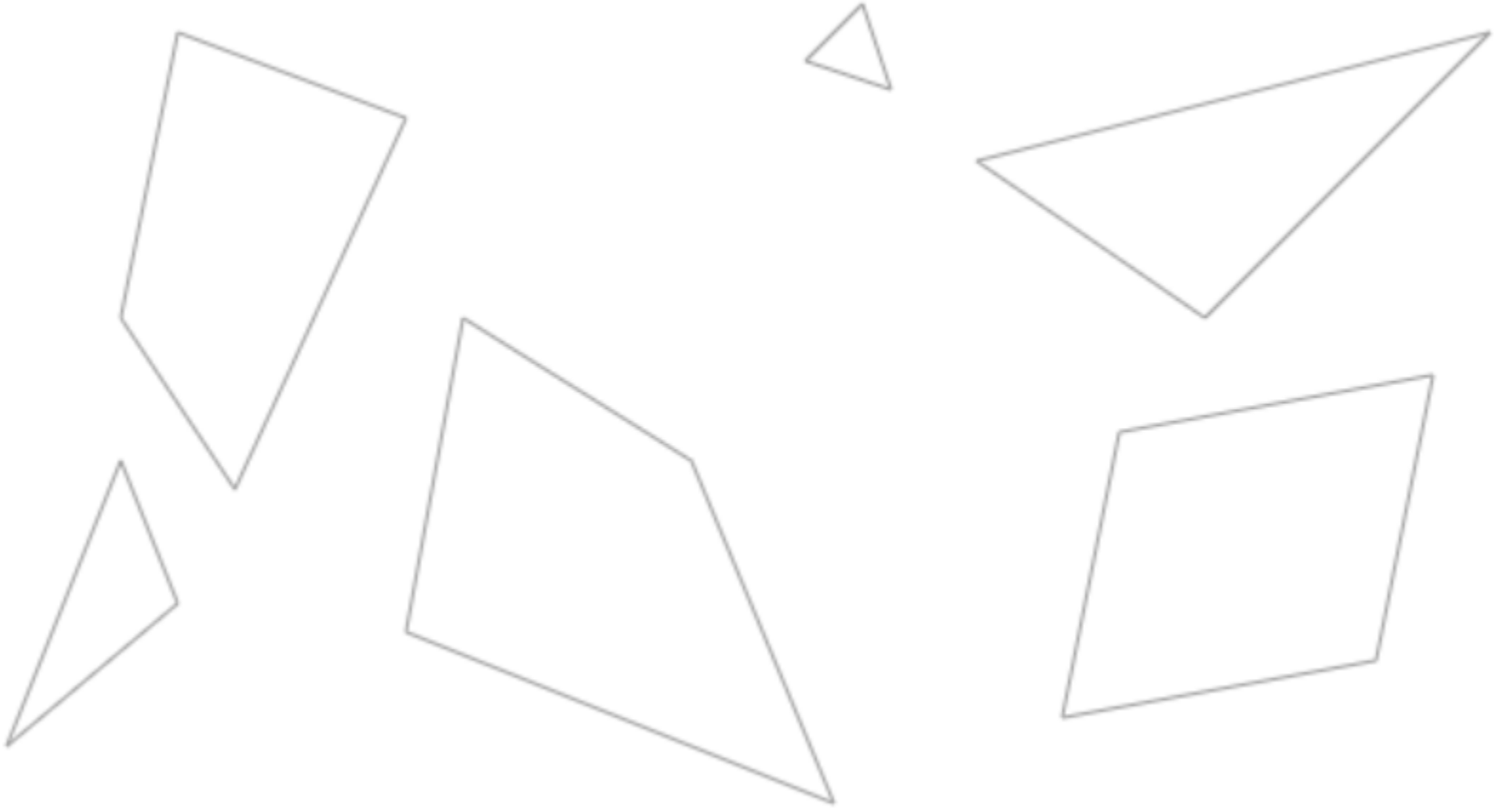
2D Game Lighting Effects

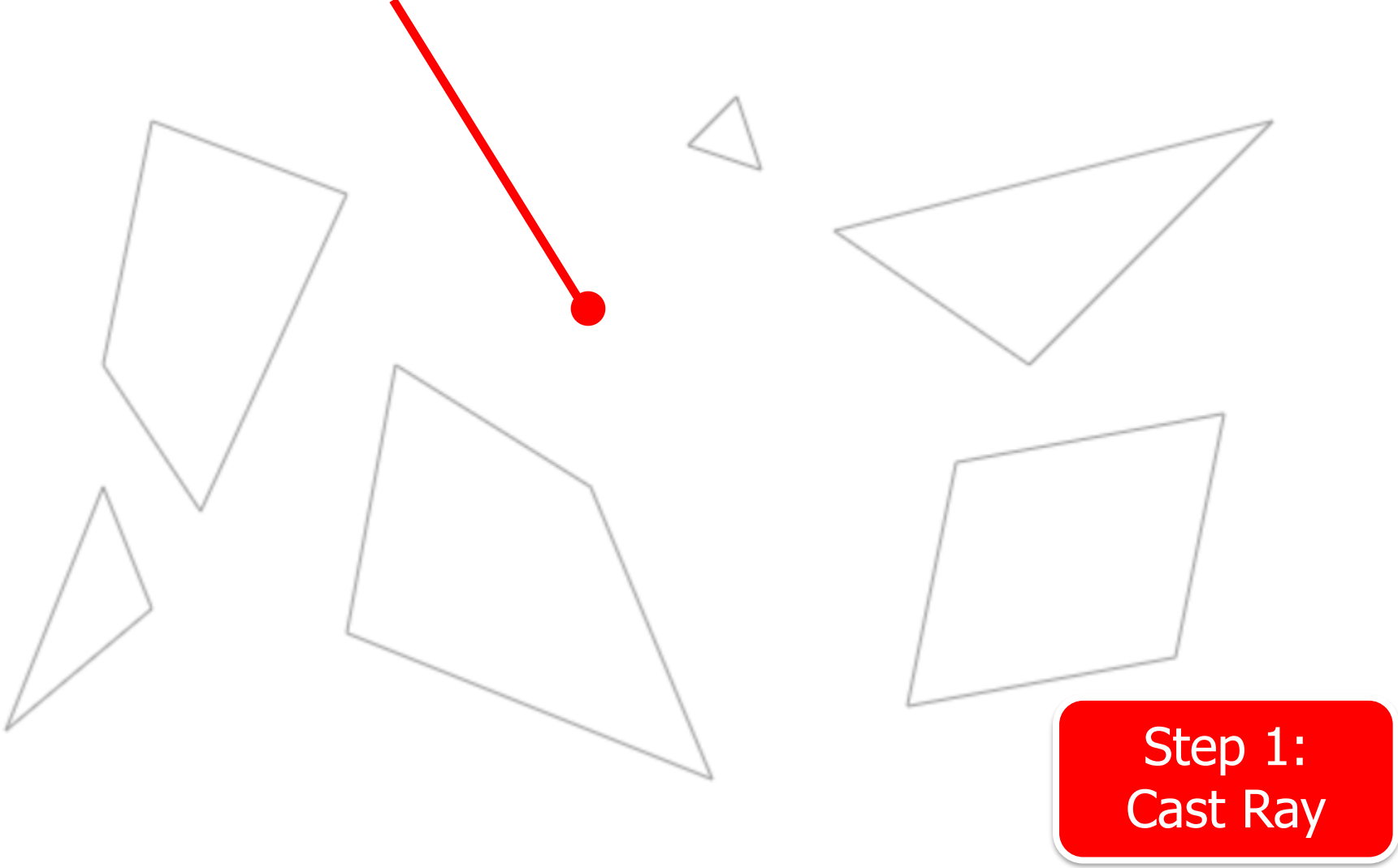
- Light travels outward in all directions
- How do we calculate everywhere it lands?

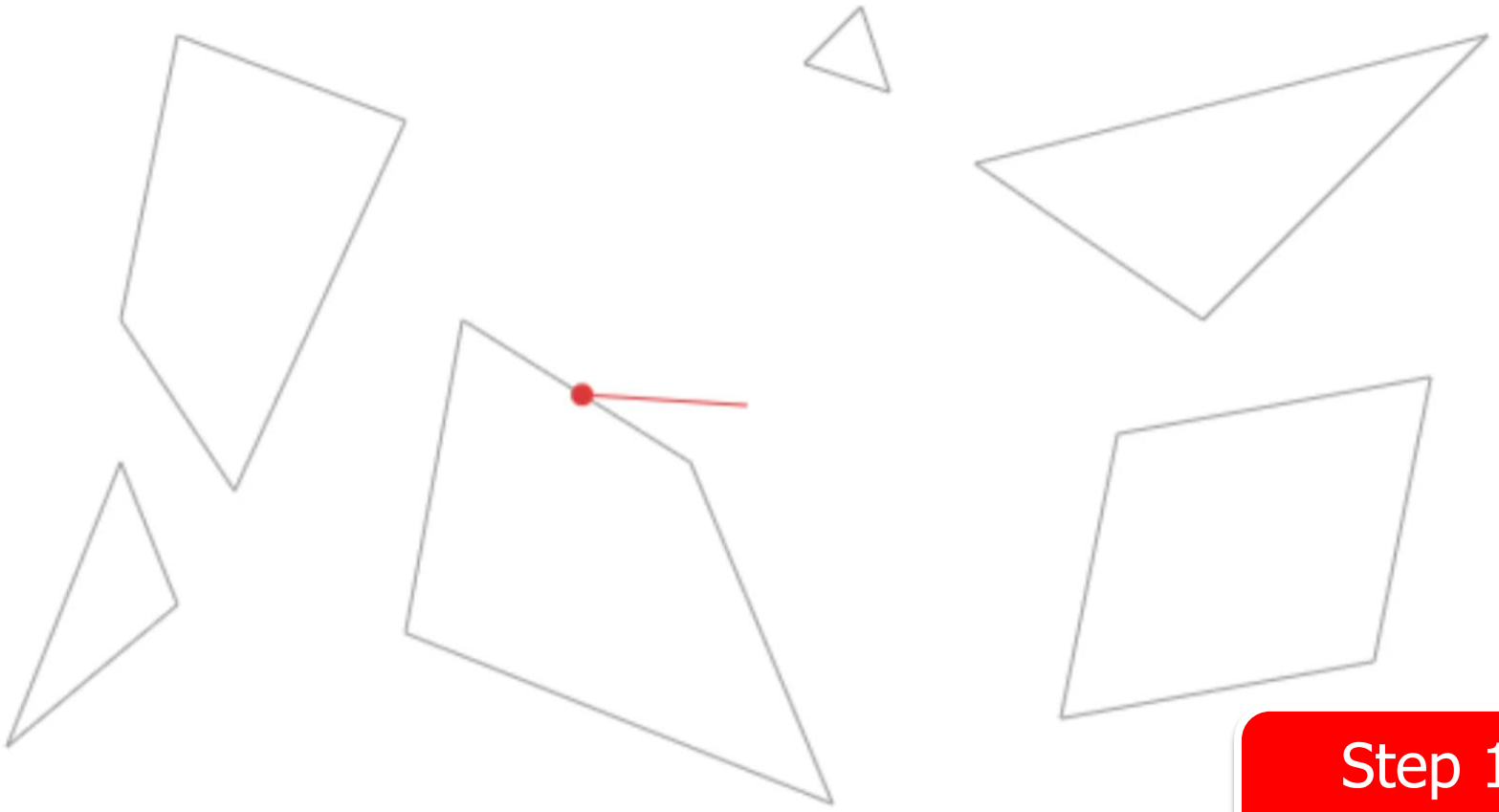
2D Game Lighting Effects

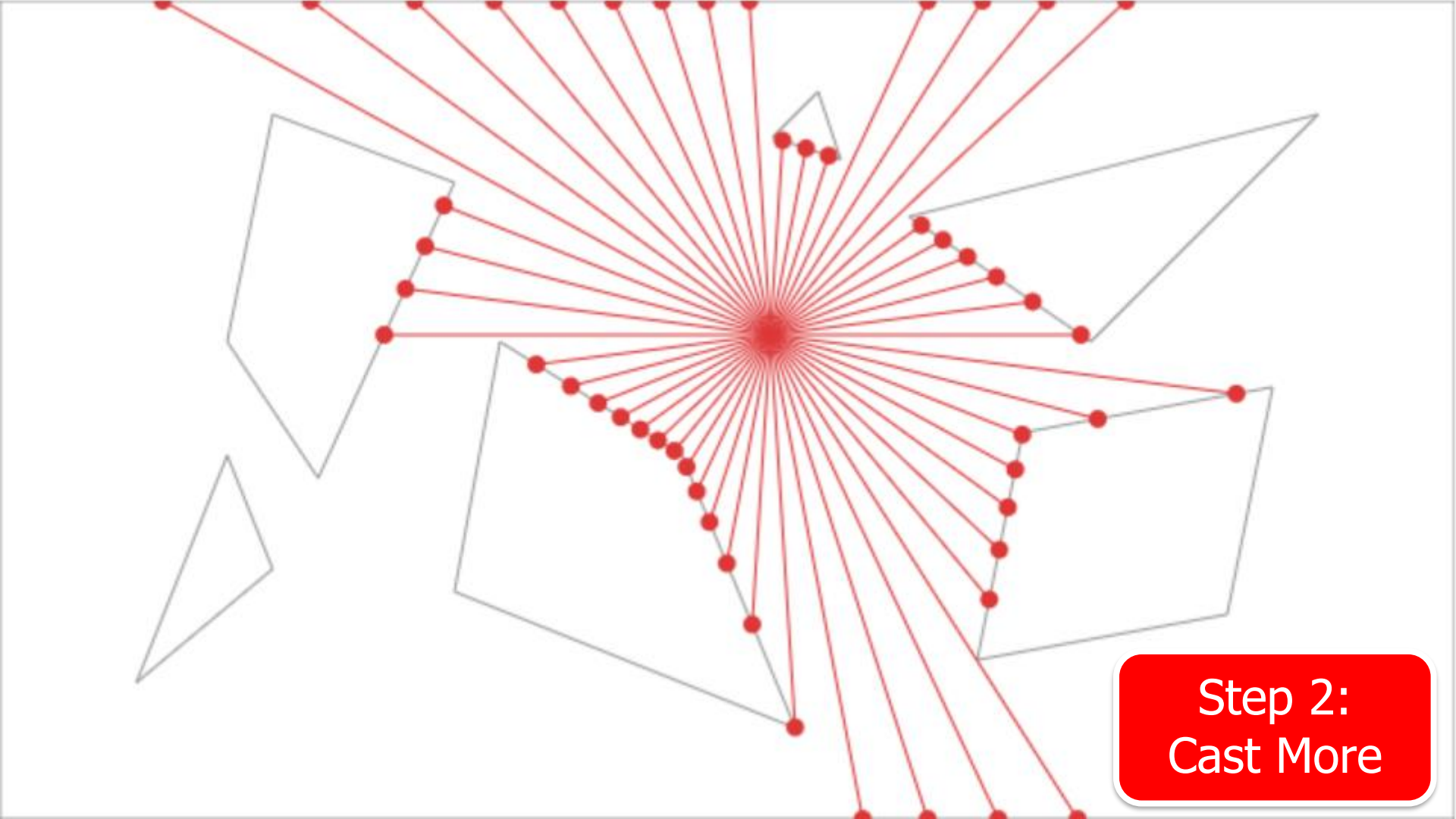
- Light travels outward in all directions
- How do we calculate everywhere it lands?
- There are several possible solutions

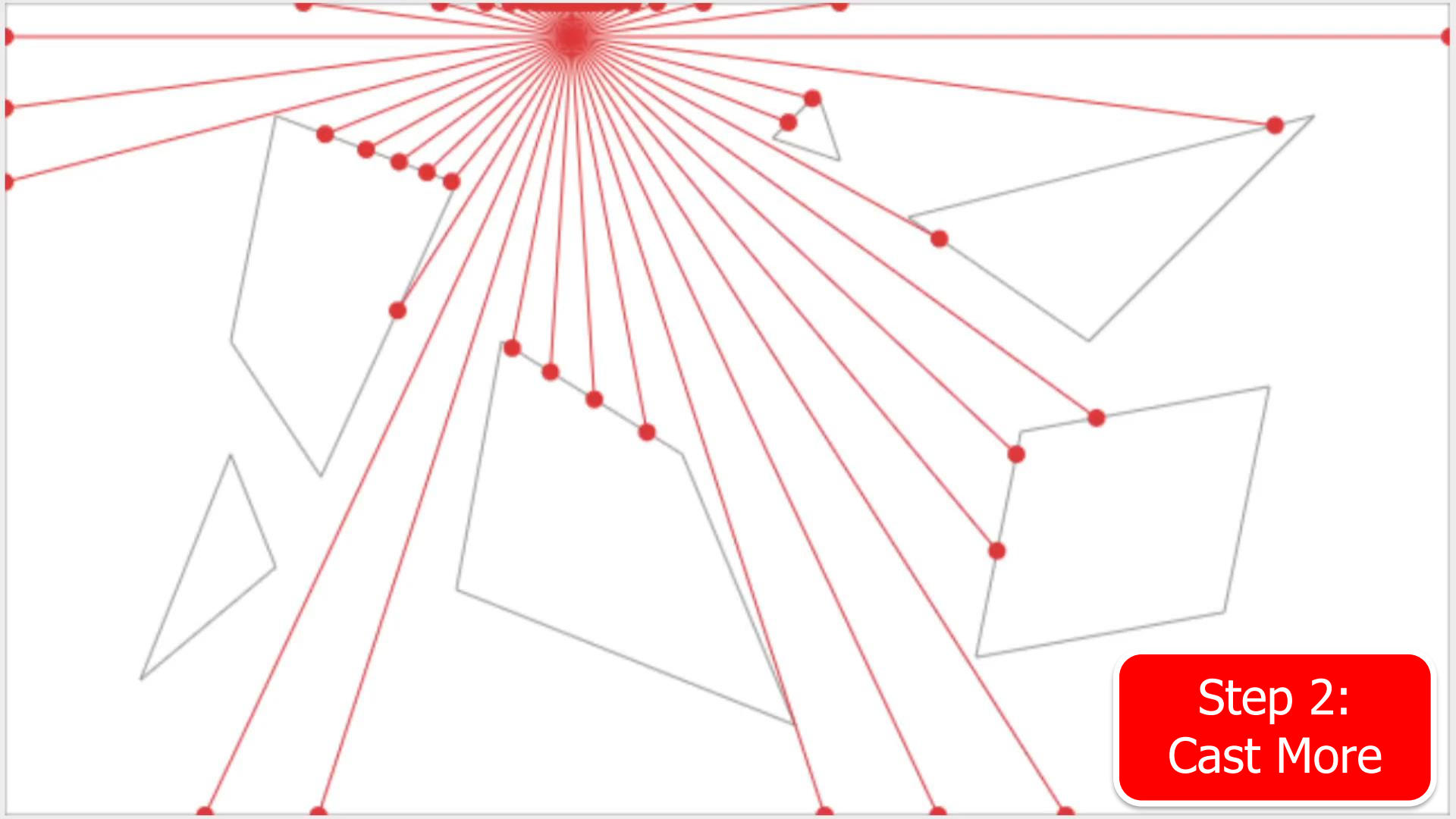
- Online interactive demos / tutorial
  - https://ncase.me/sight-and-light/
  - https://www.redblobgames.com/articles/visibility/

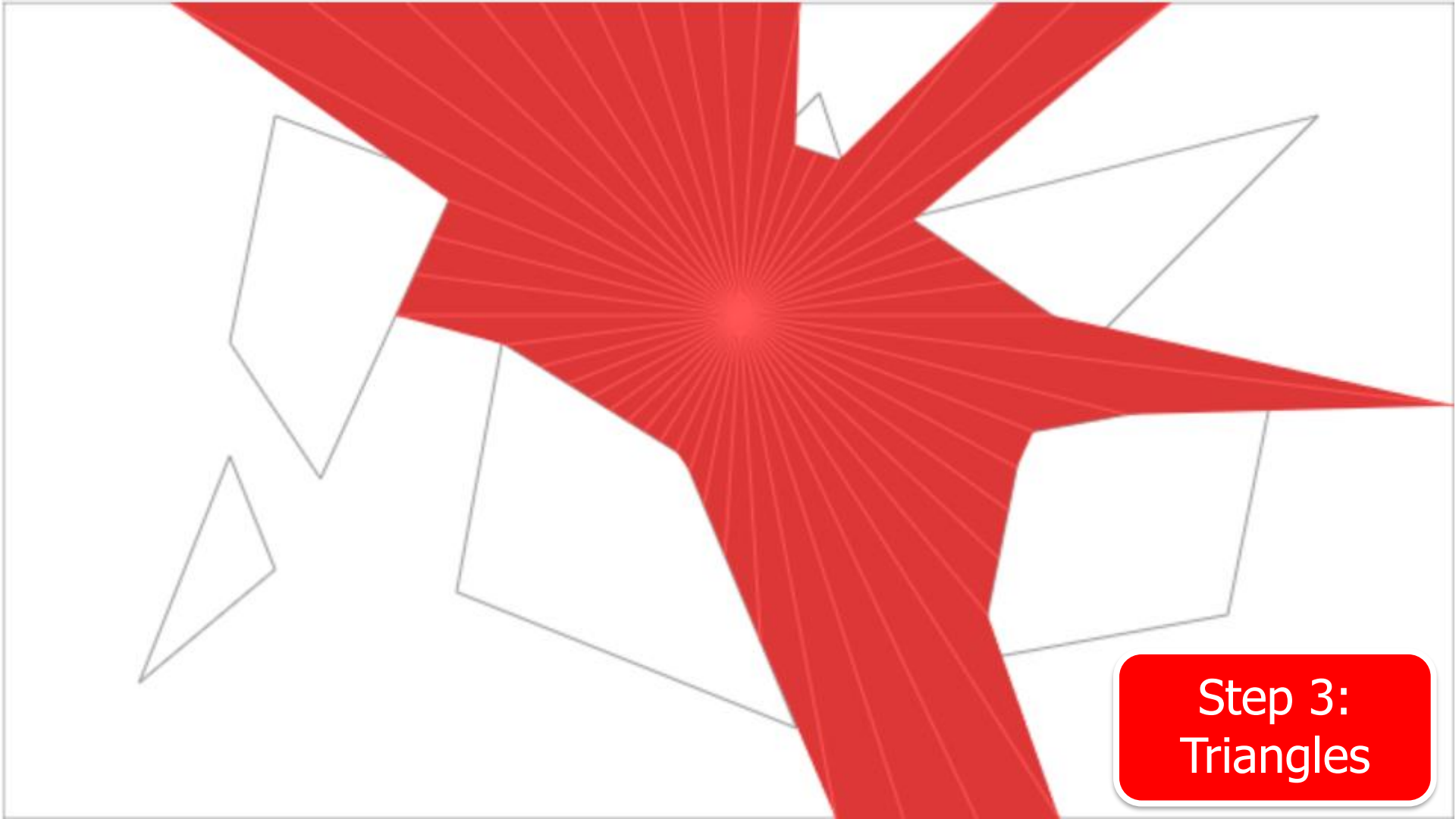


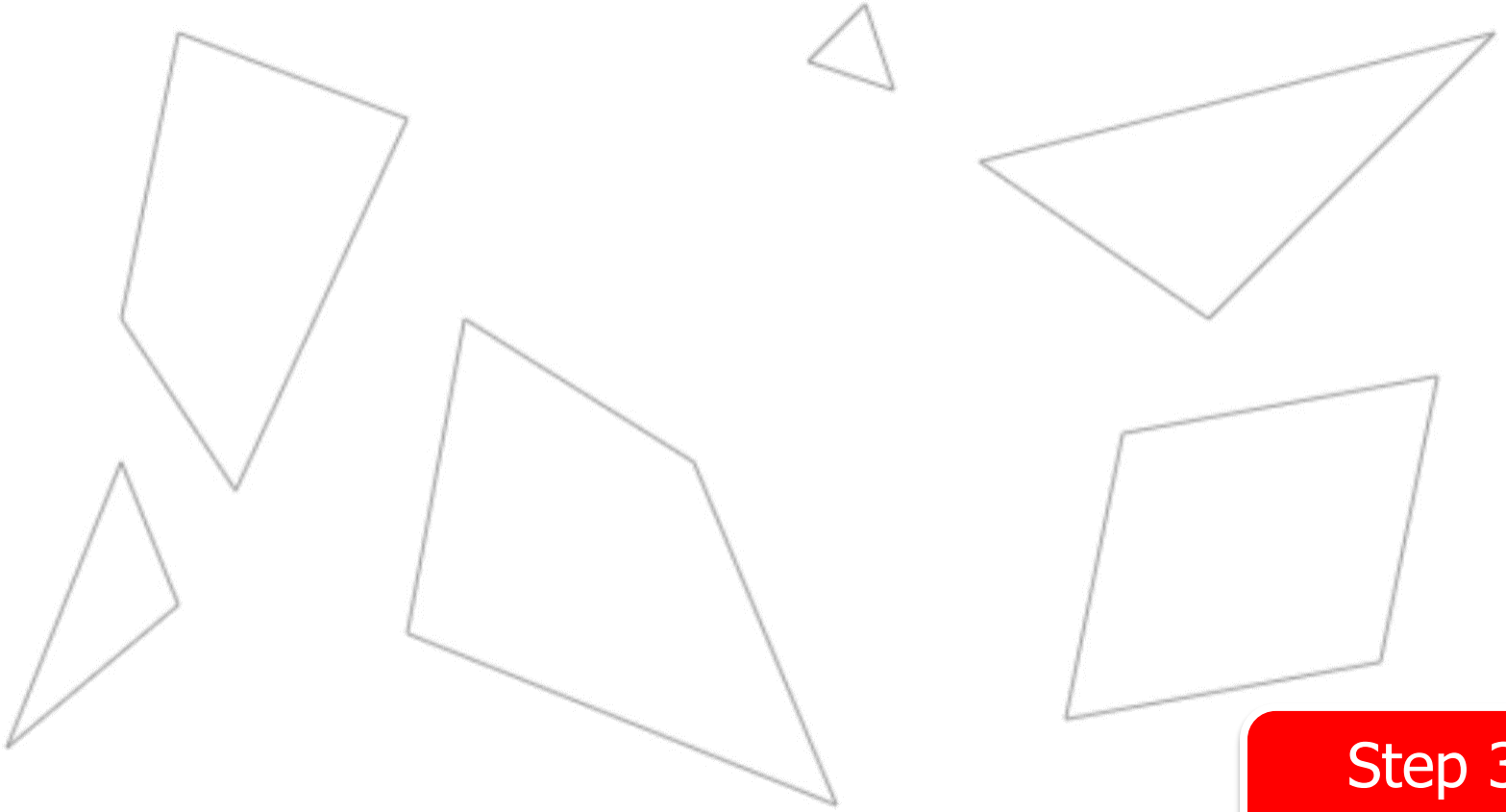

Step 1:
Cast Ray


Step 1:
Cast Ray

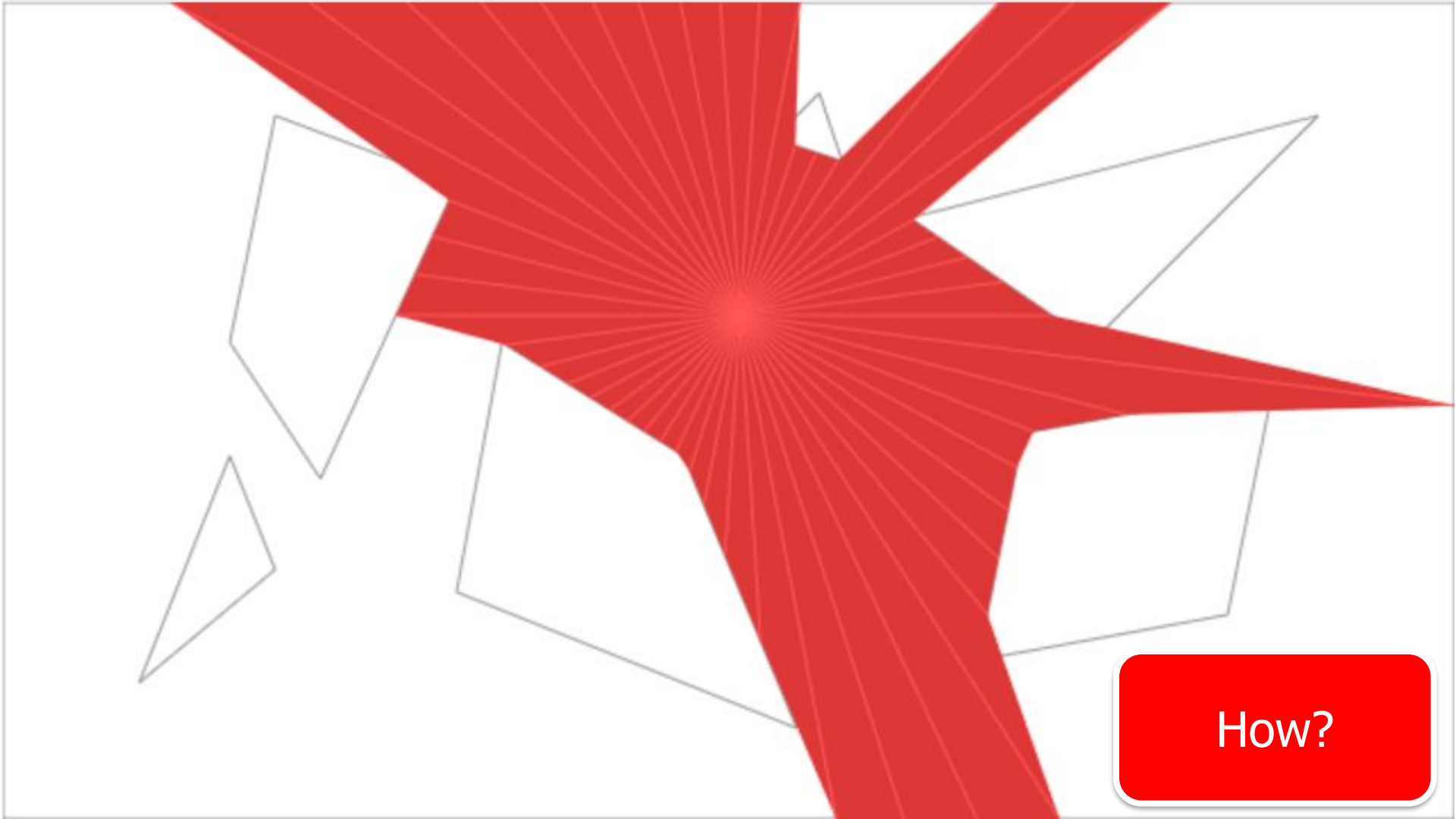

Step 2:
Cast More

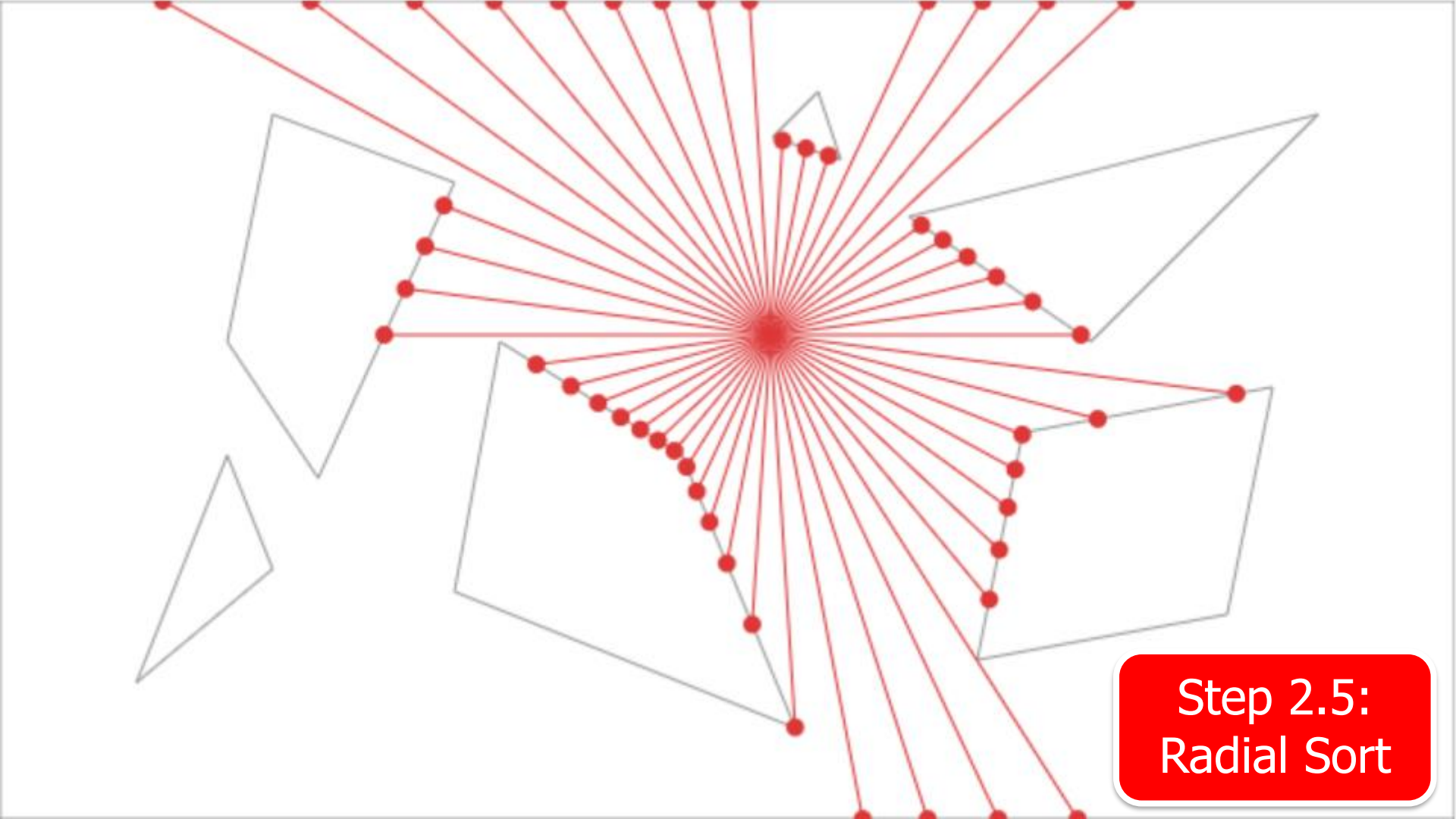

Step 2:
Cast More

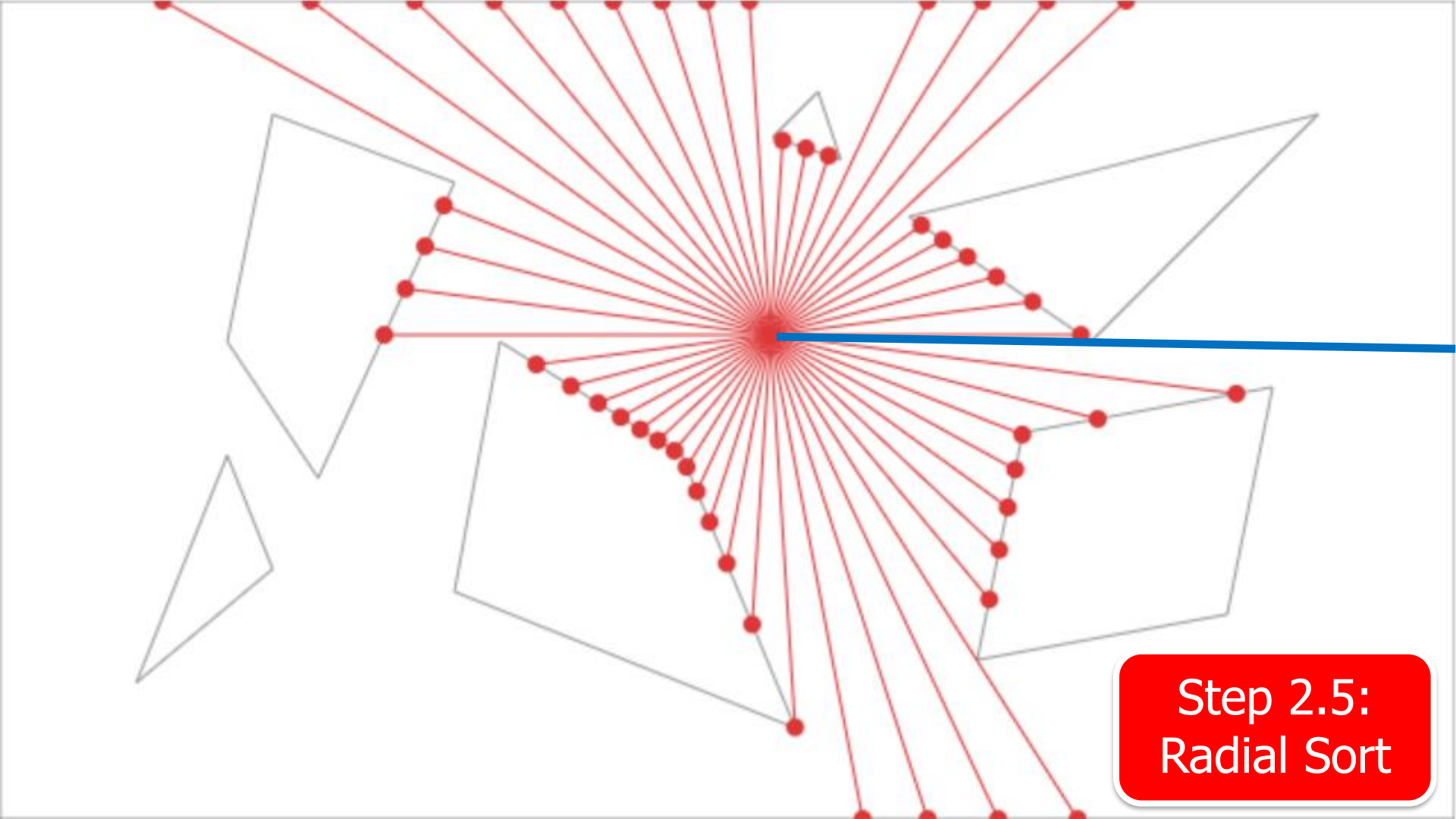

Step 3:
Triangles

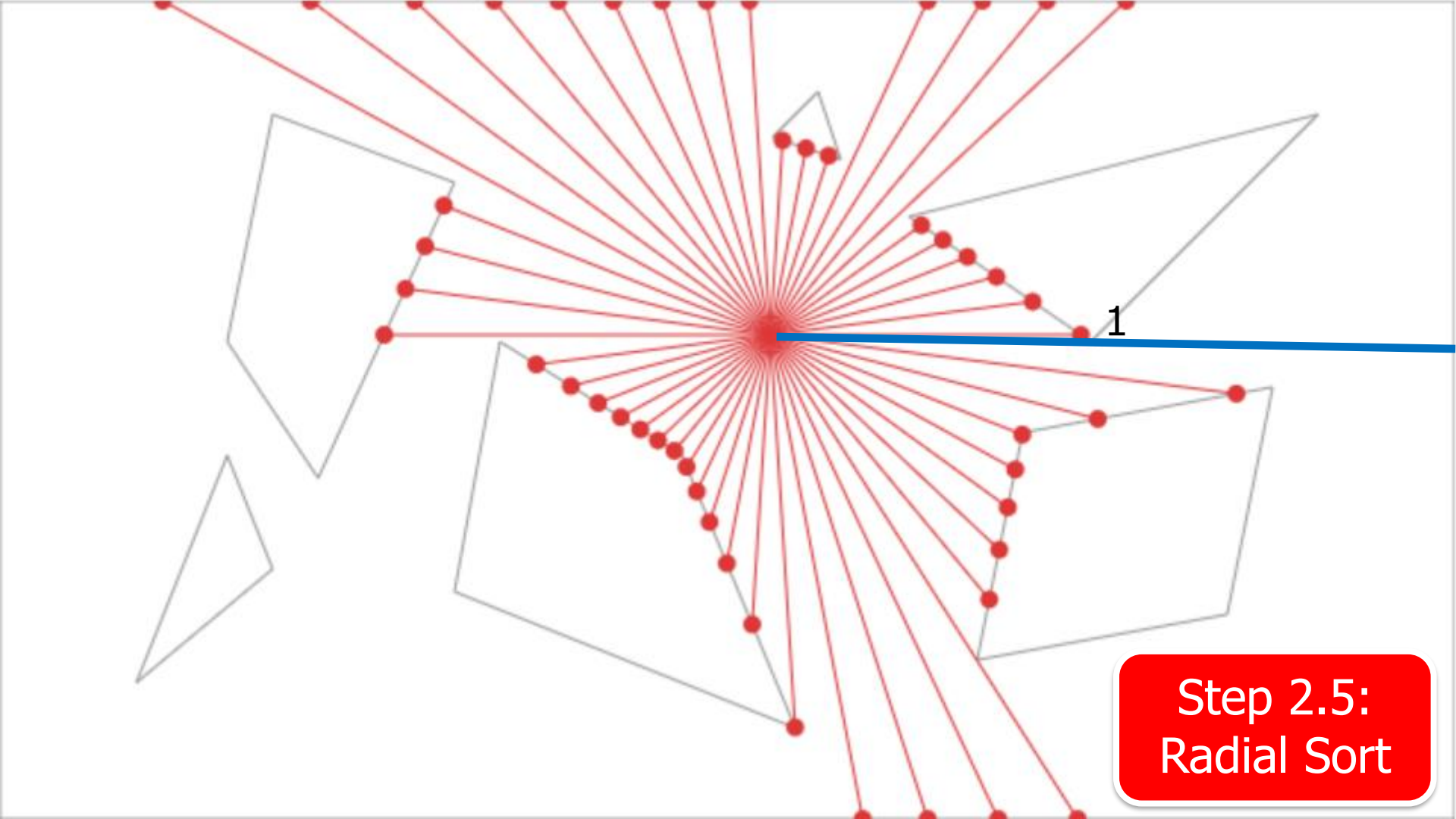


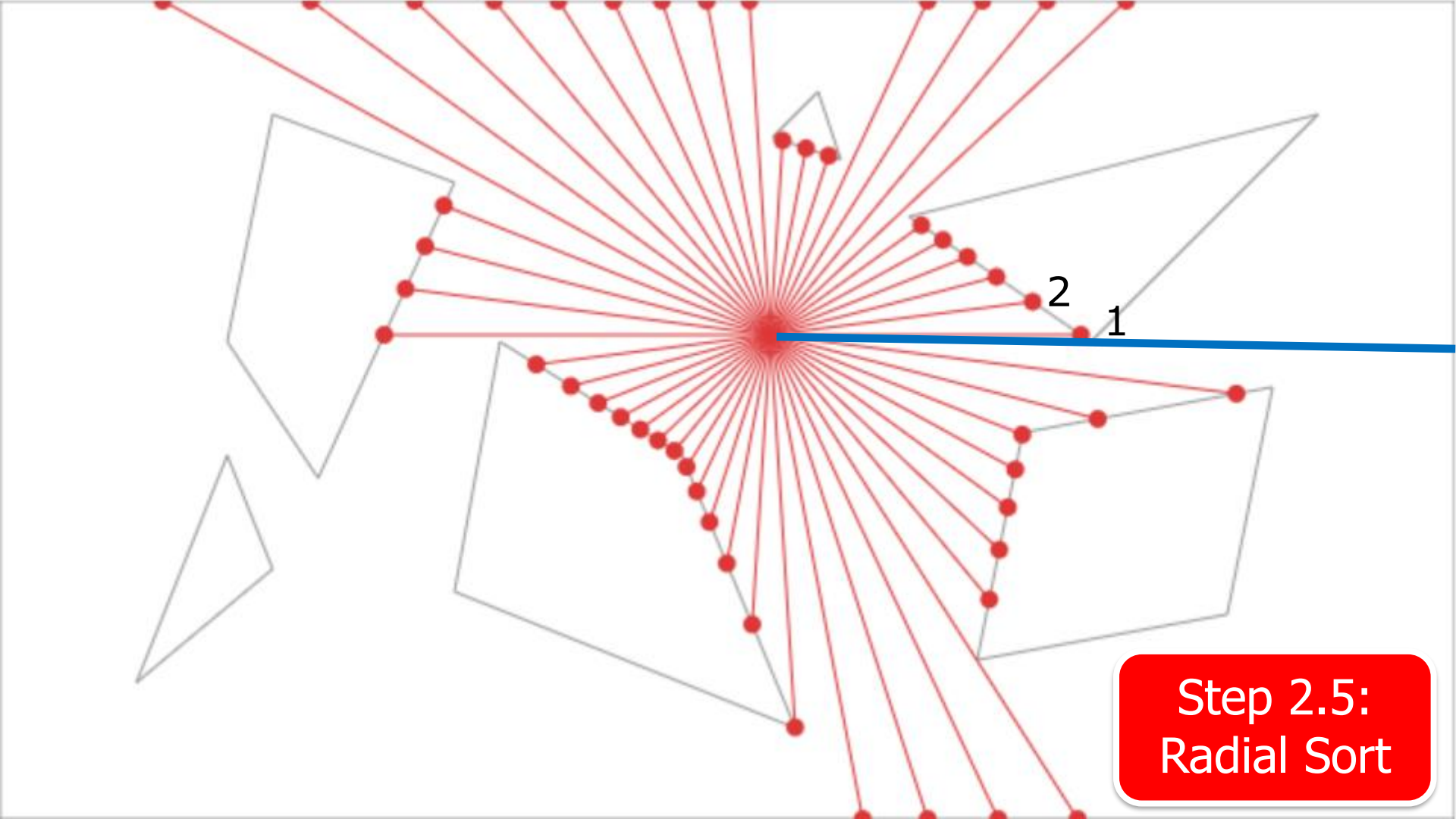
Step 3:
Triangles

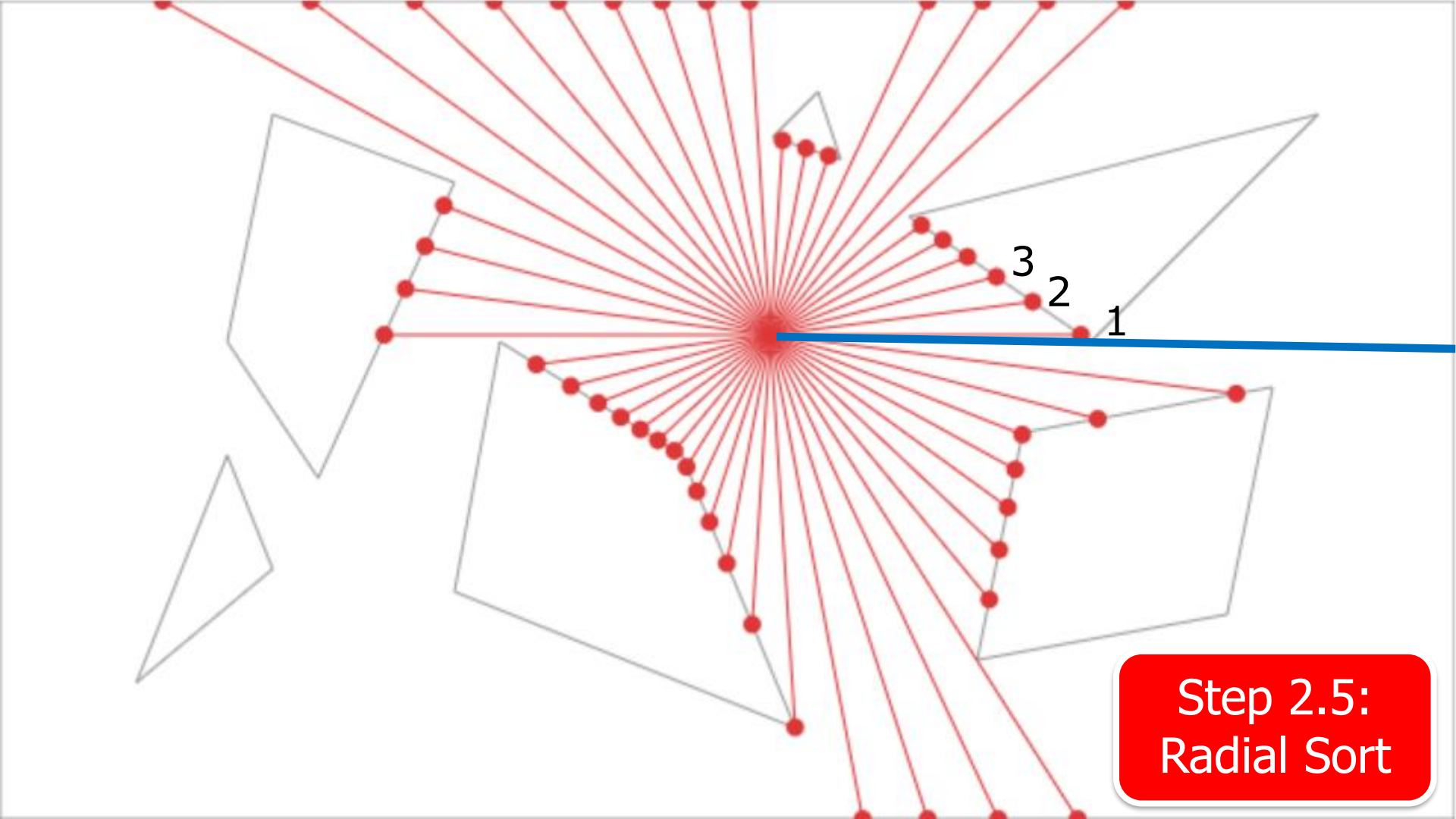

How?

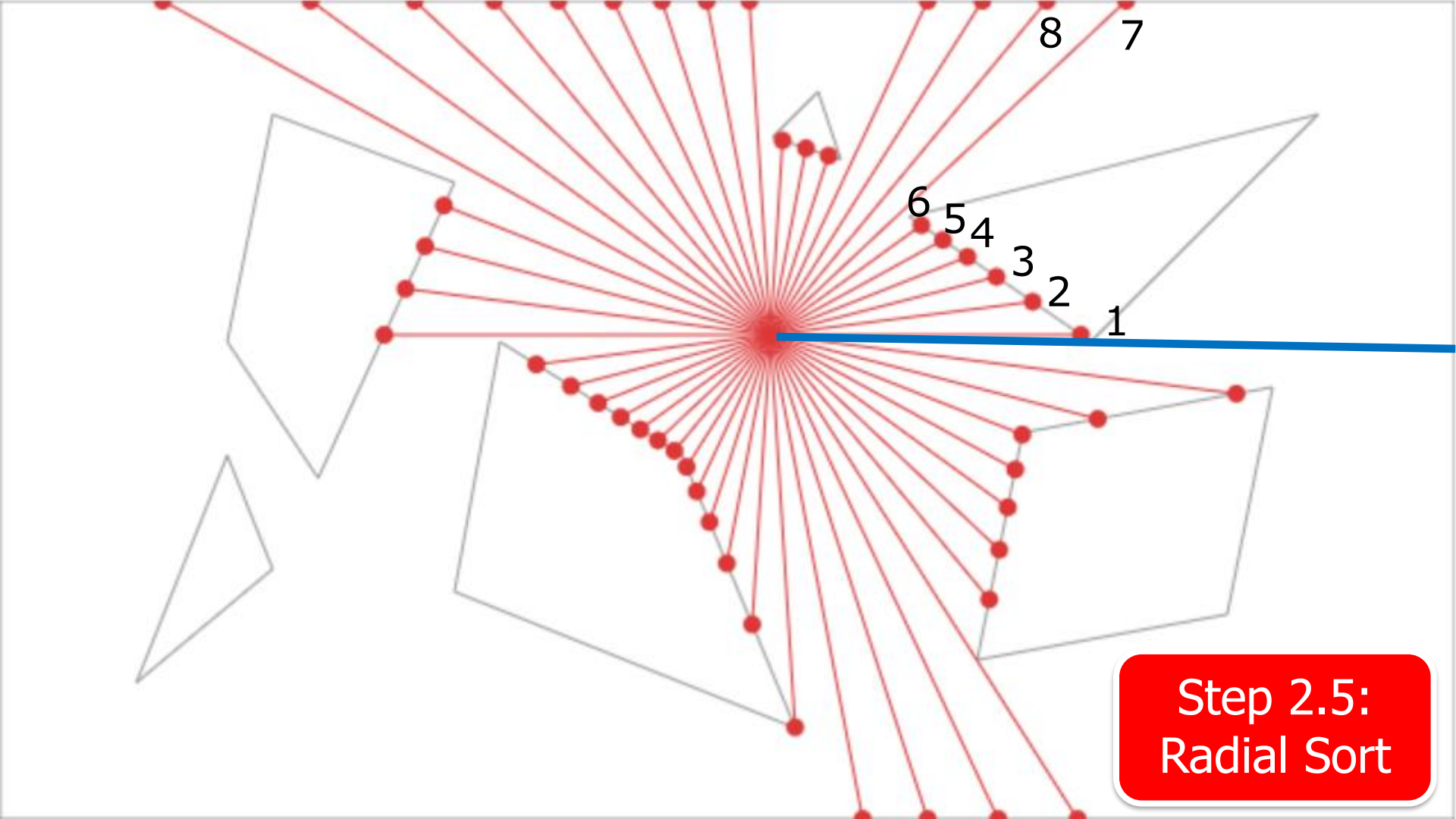

Step 2.5:
Radial Sort

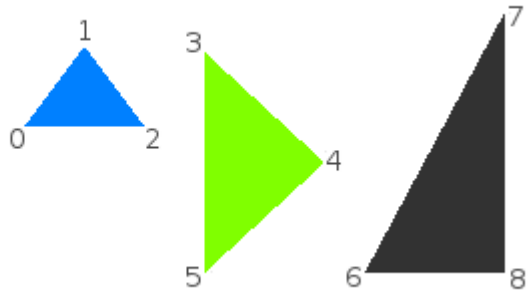

Step 2.5:
Radial Sort


1
Step 2.5:
Radial Sort


2
1
Step 2.5:
Radial Sort

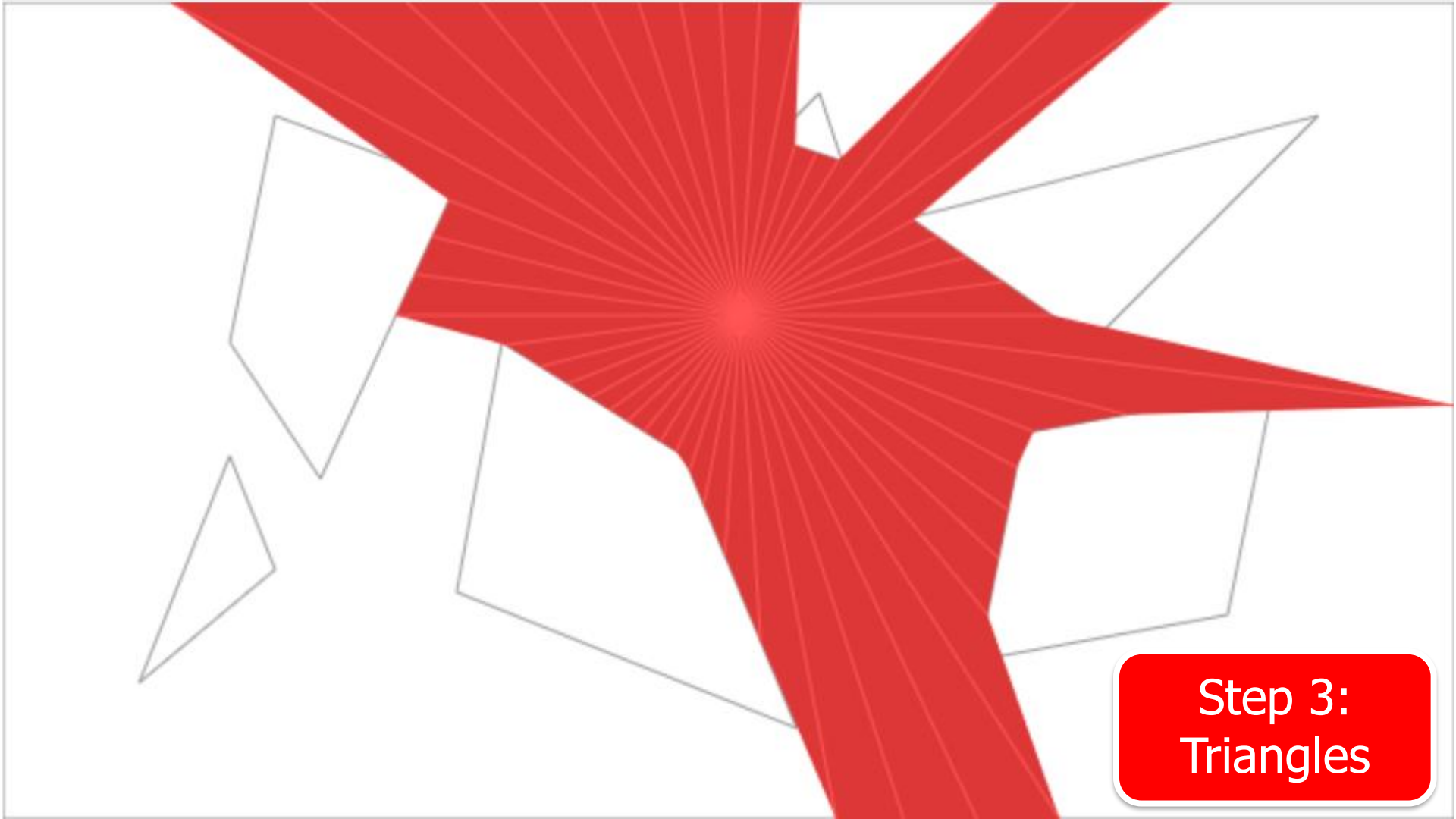

3
2
1
Step 2.5:
Radial Sort

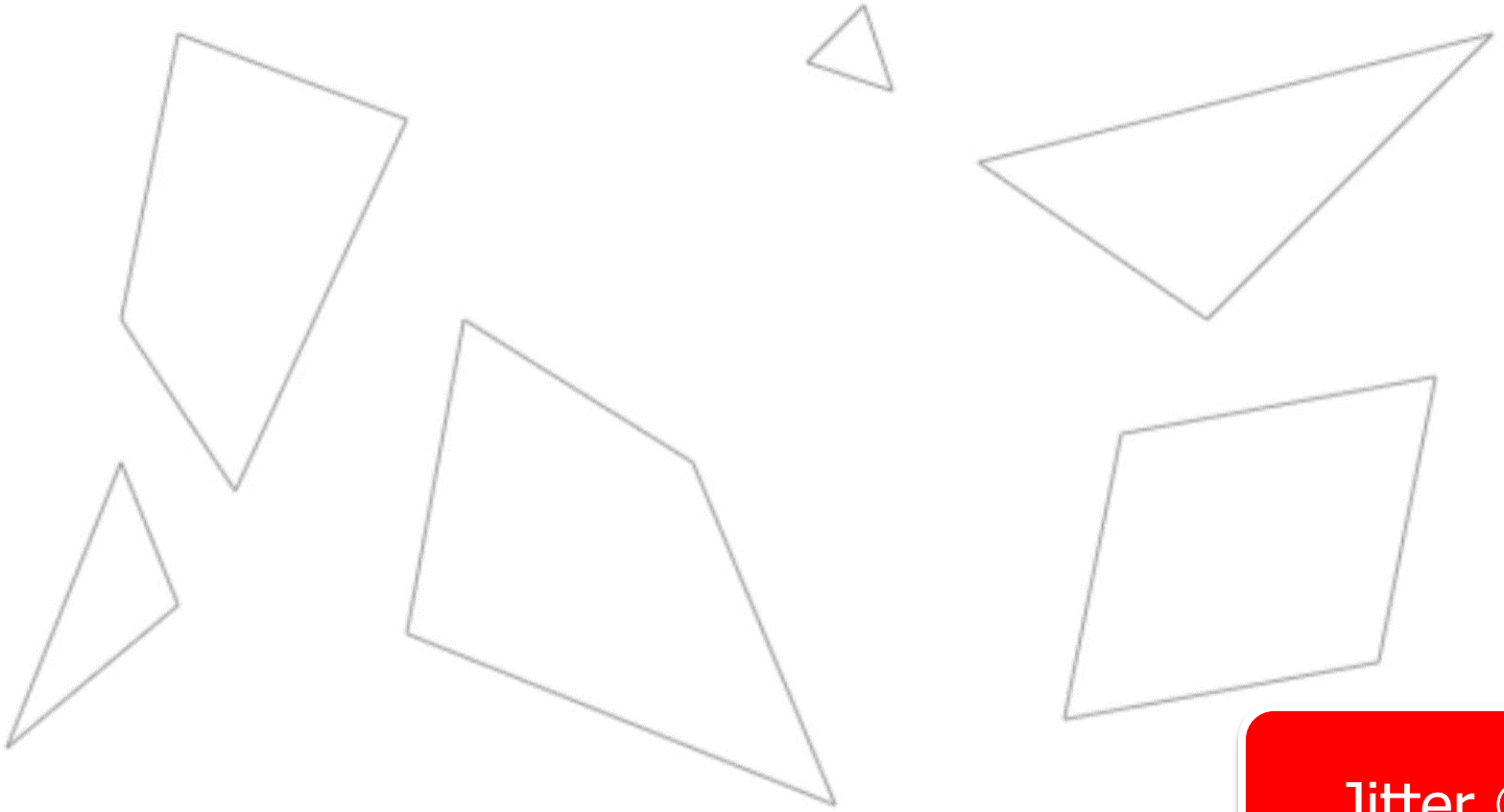

8
7
6
5
4
3
2
1
Step 2.5:
Radial Sort

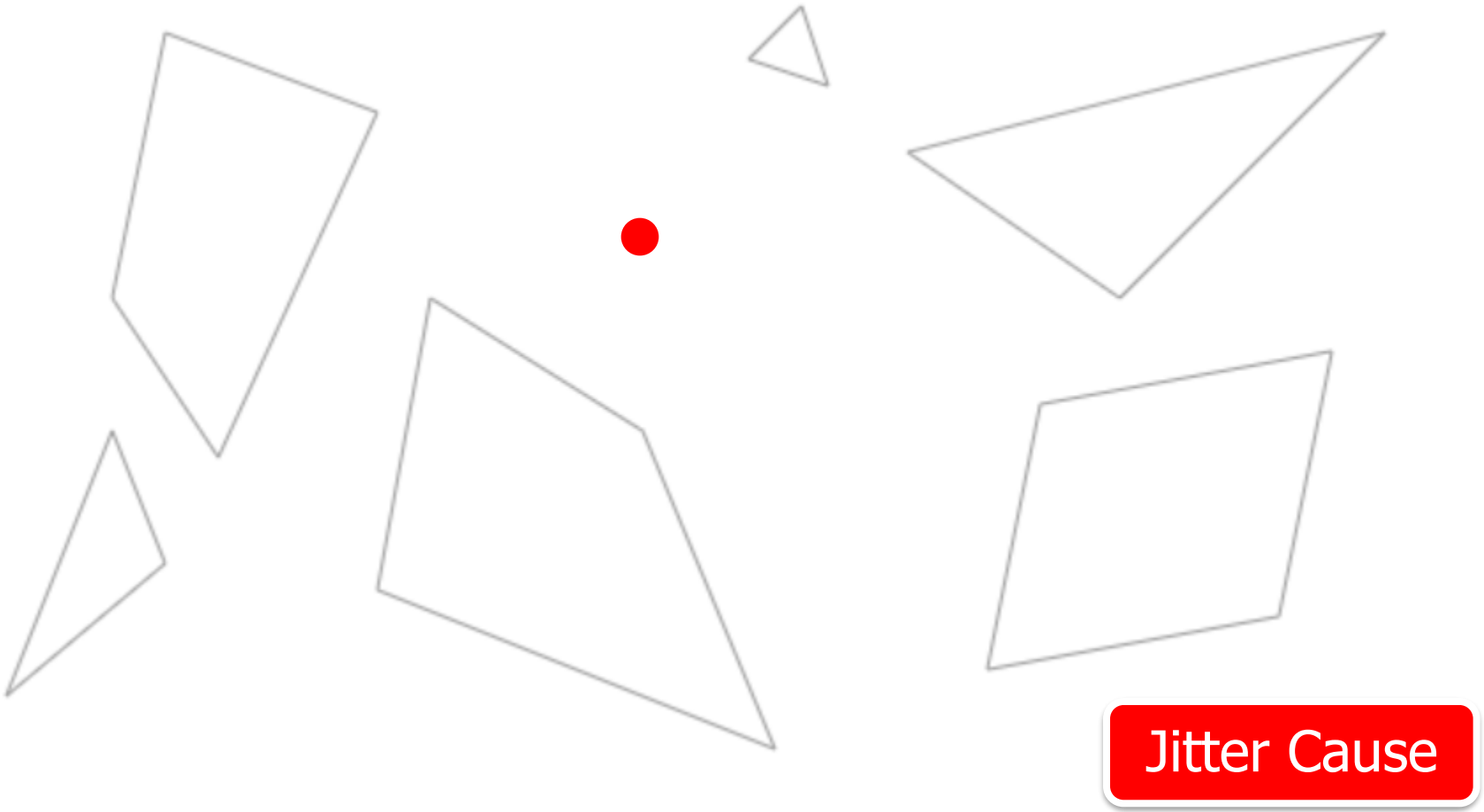
`sf::Triangles`	A set of unconnected triangles.	
`sf::TriangleFan`	A set of triangles connected to a central point. The first vertex is the center, then each new vertex defines a new triangle, using the center and the previous vertex.	

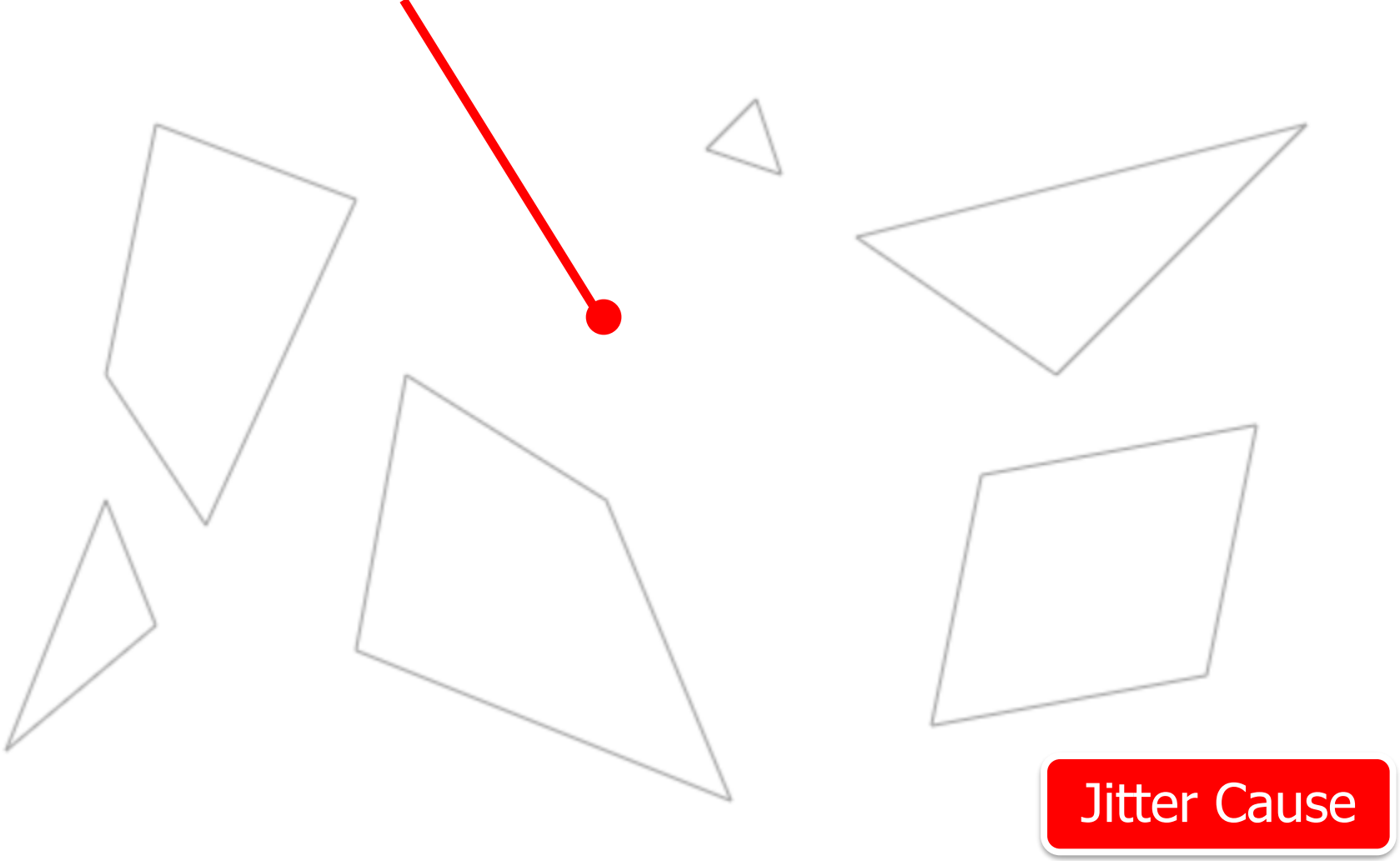
https://www.sfml-dev.org/tutorials/2.6/graphics-vertex-array.php

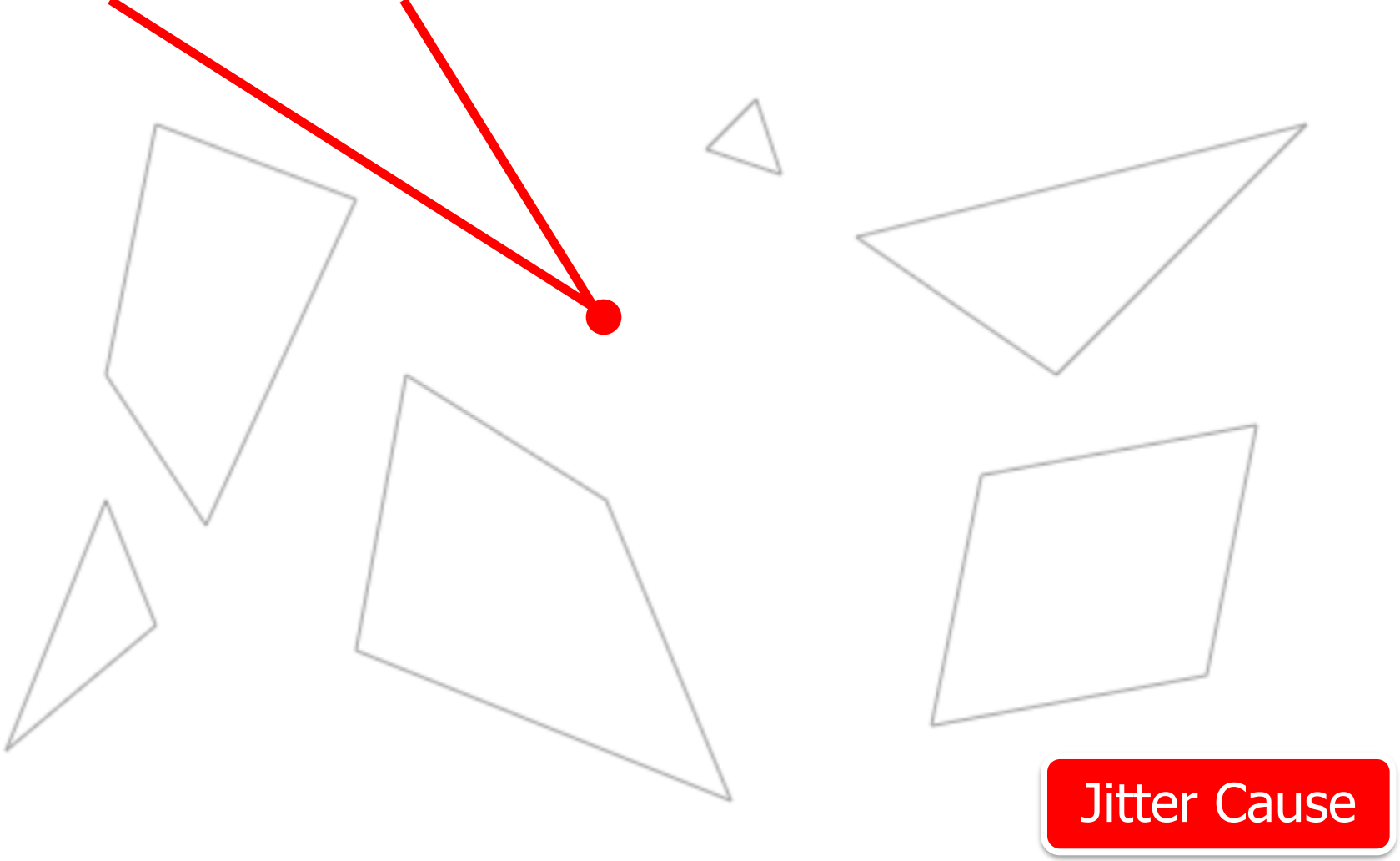
Draw Polys
Triangle Fan

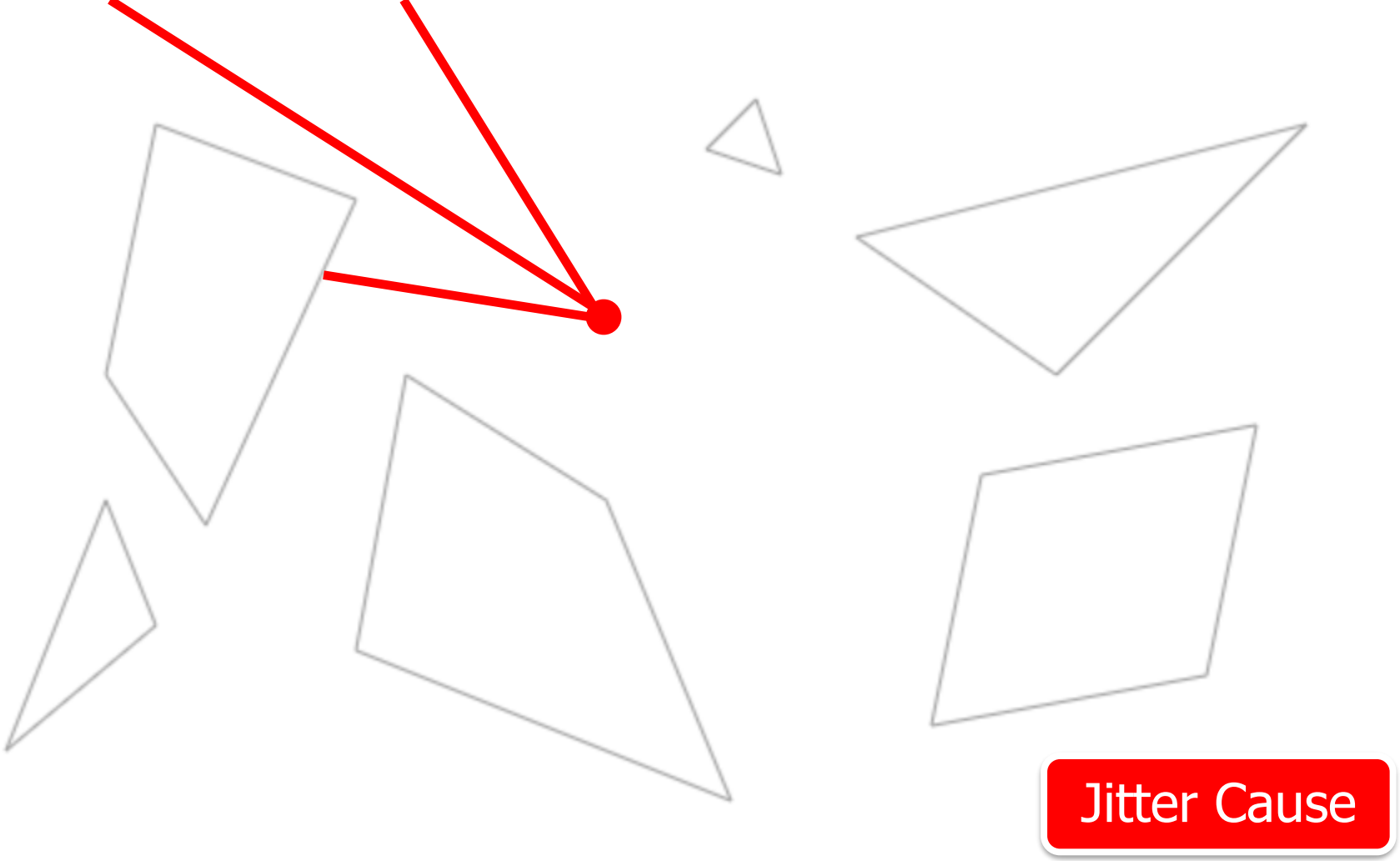

Step 3:
Triangles

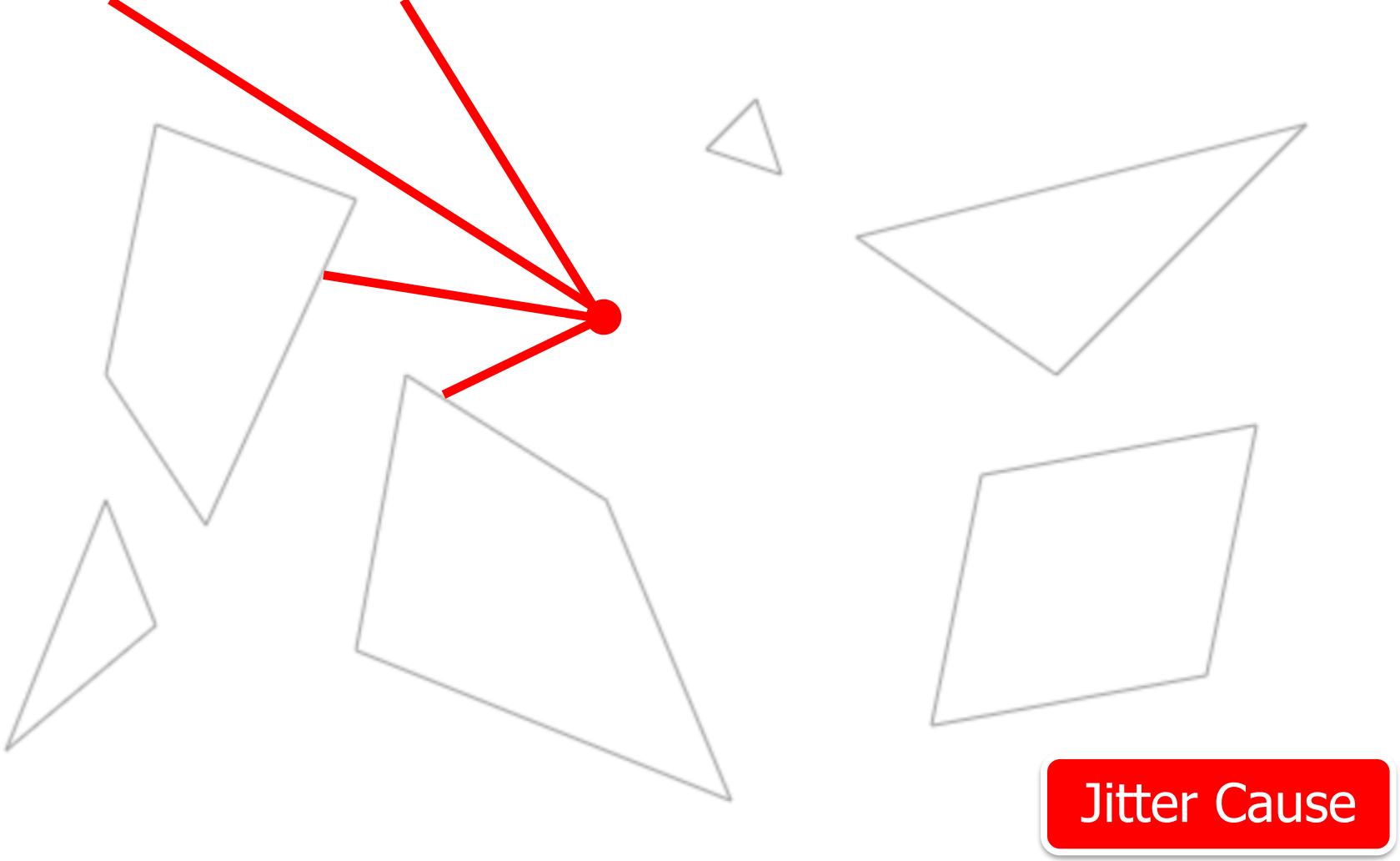

Jitter ☹

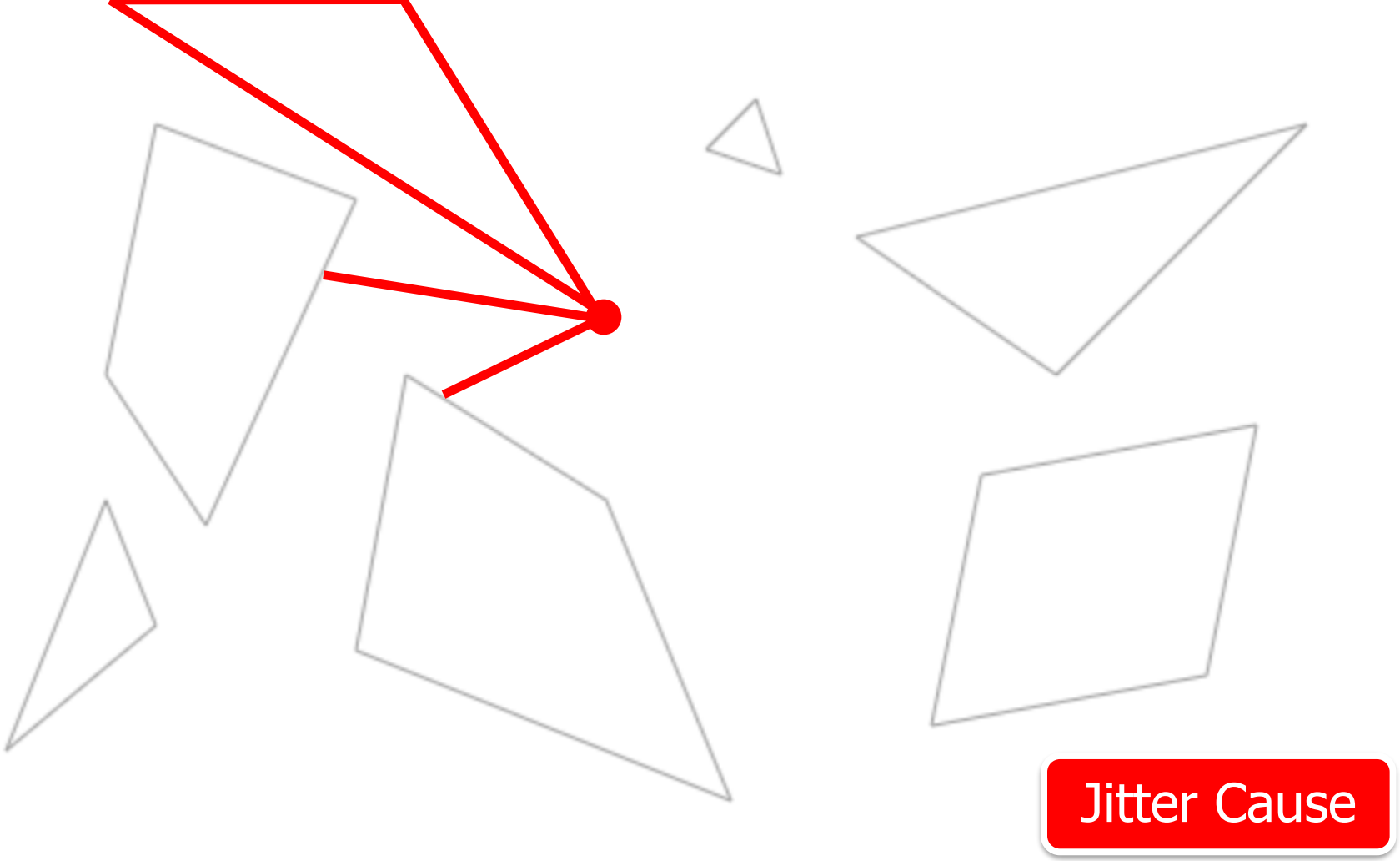

Jitter Cause

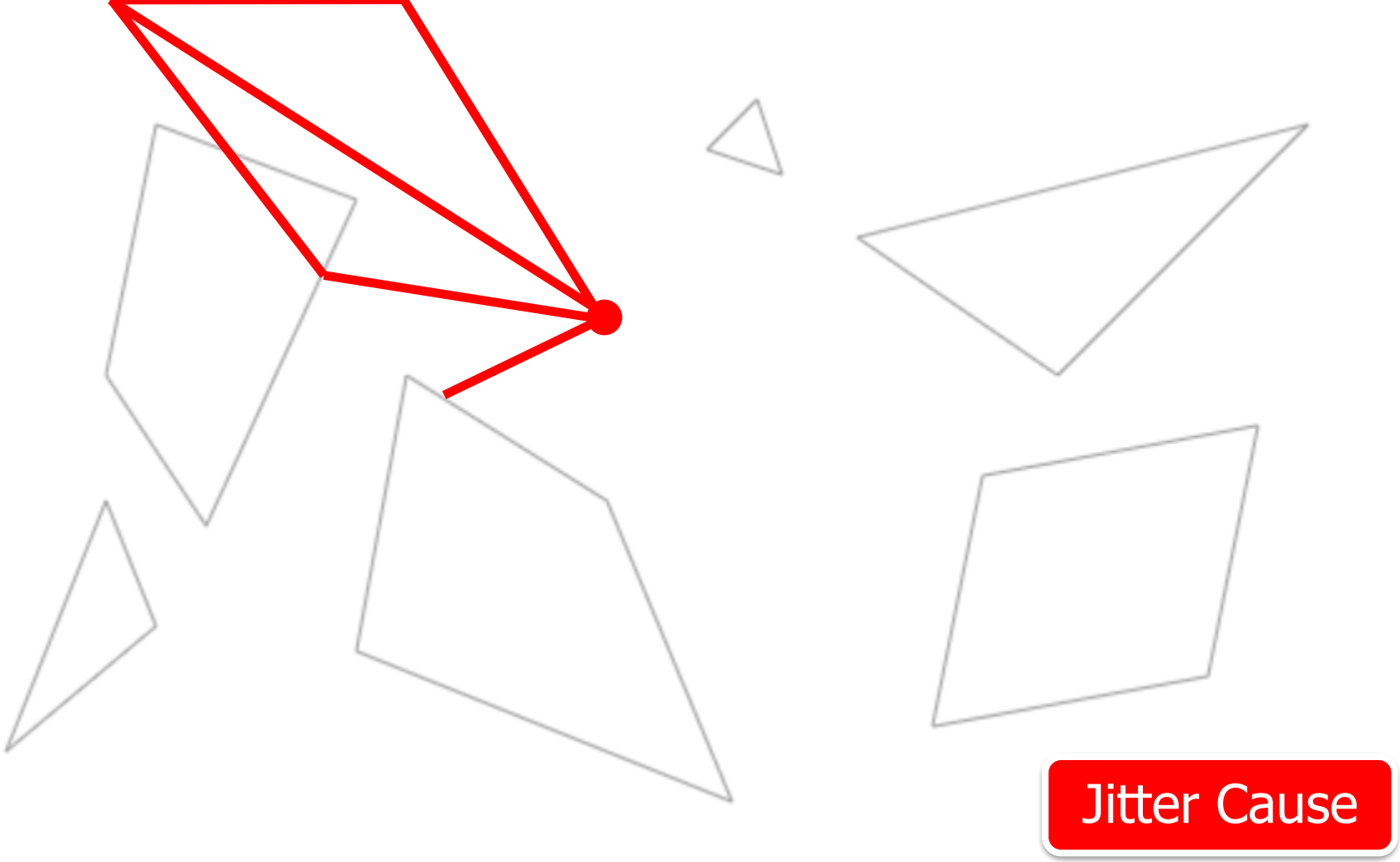

Jitter Cause

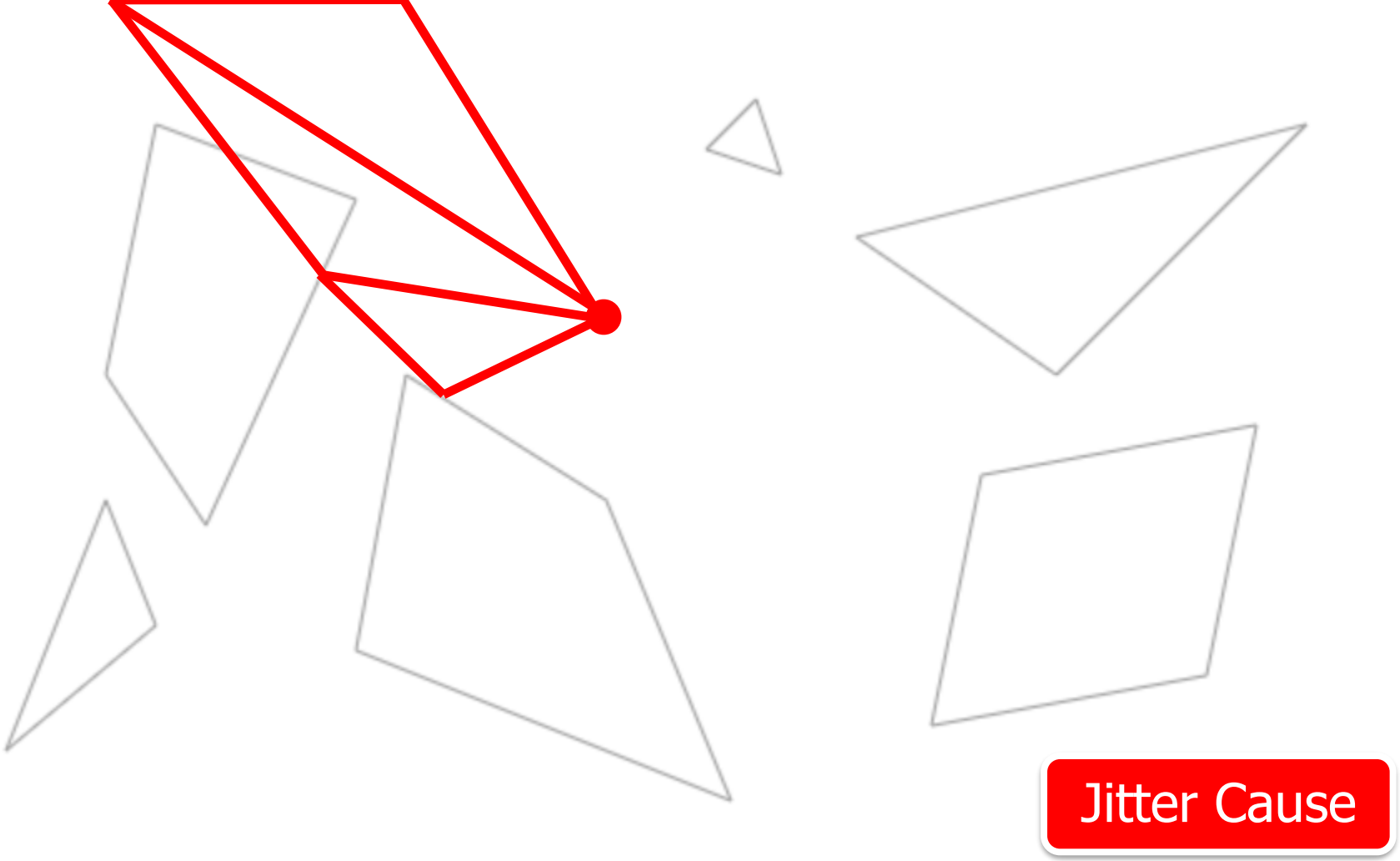

Jitter Cause

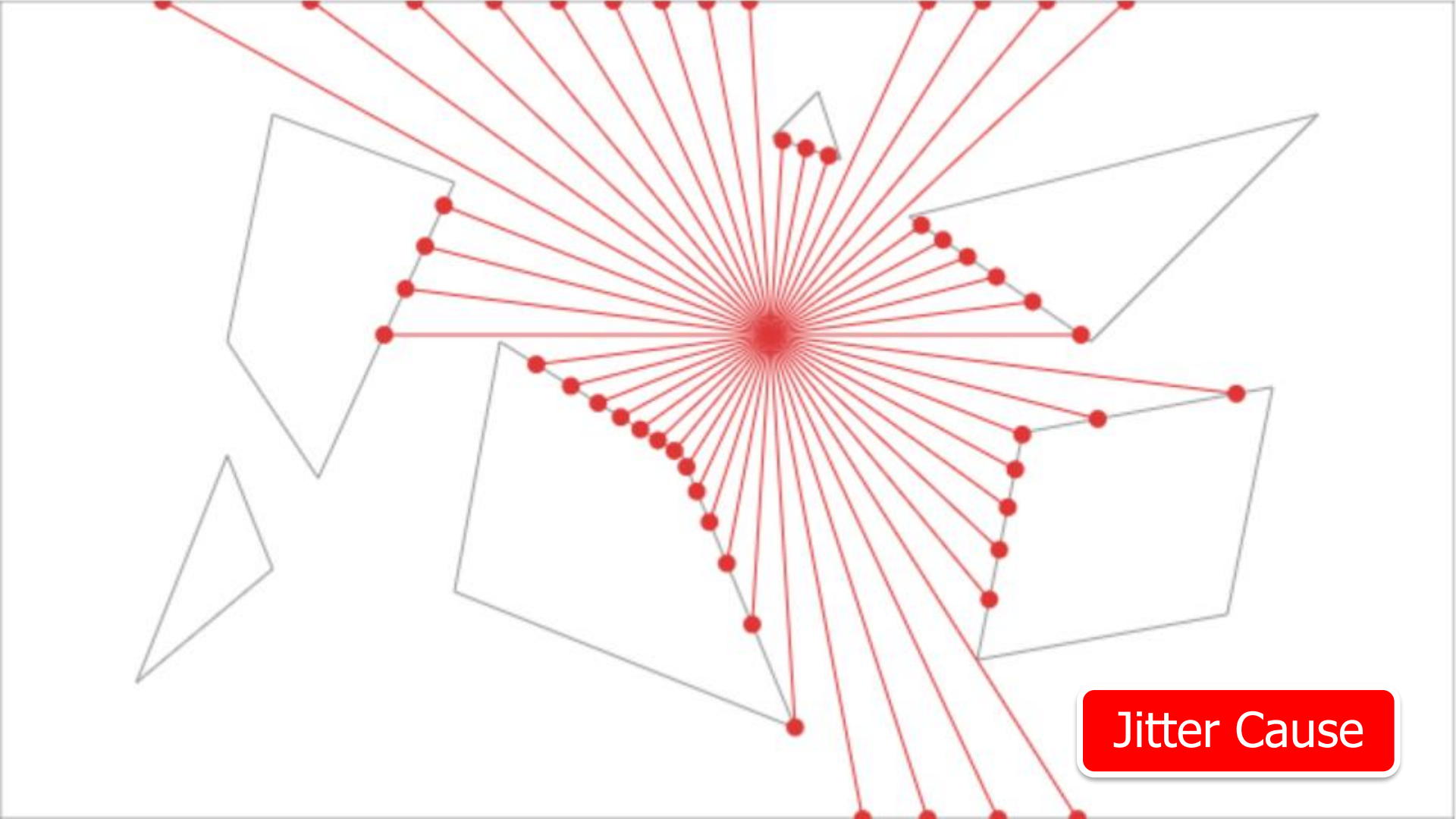

Jitter Cause

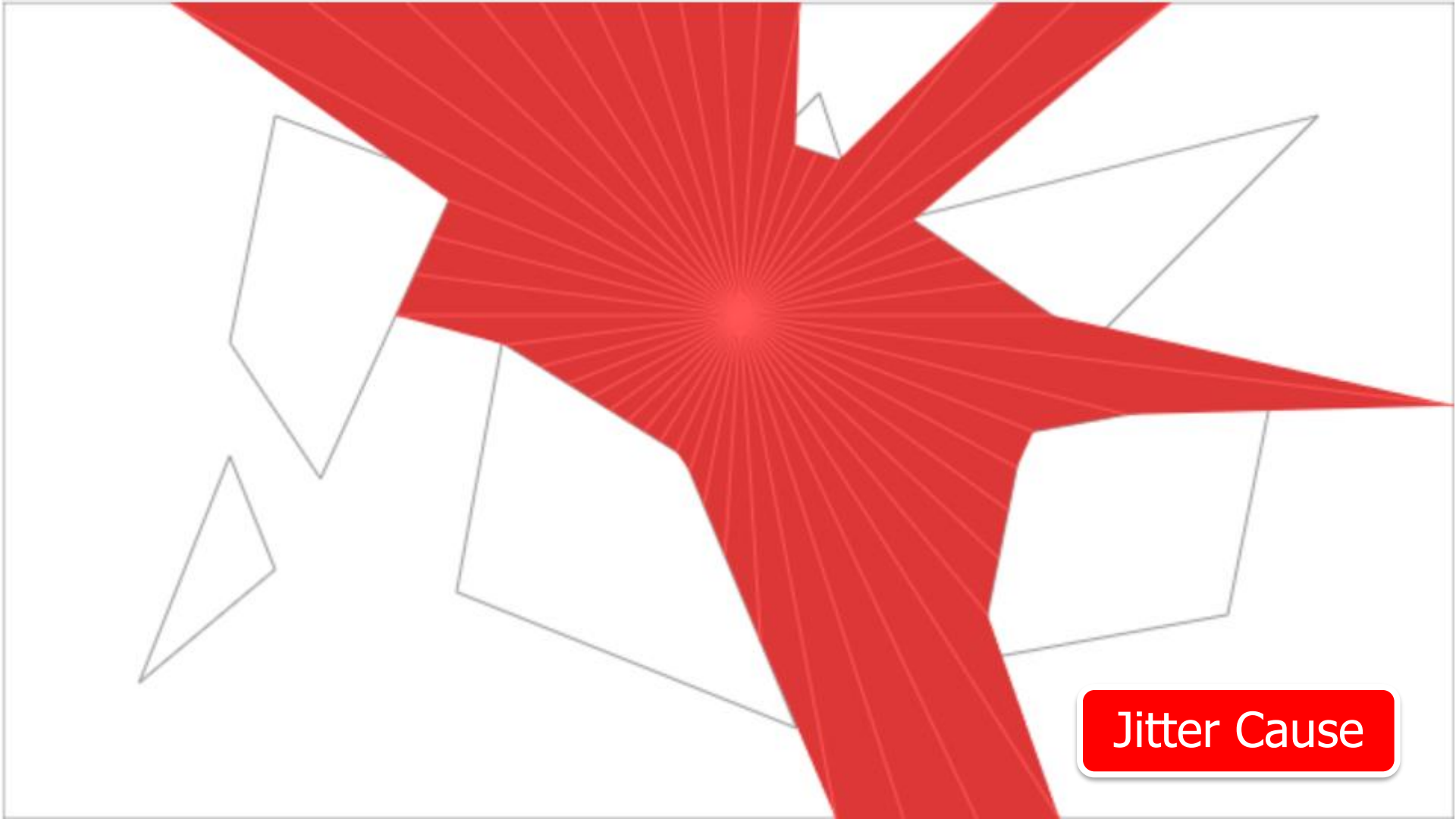

Jitter Cause

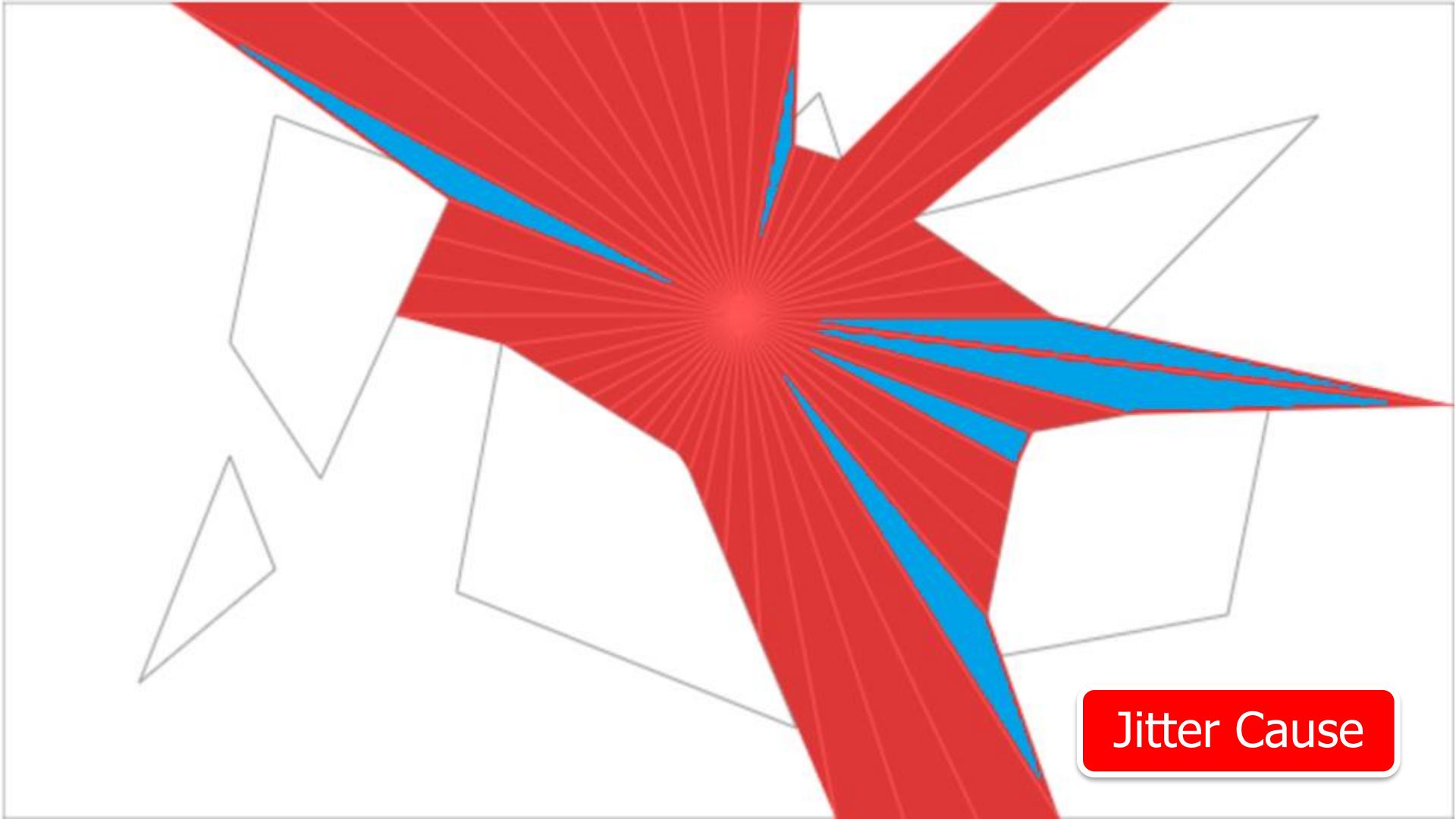

Jitter Cause

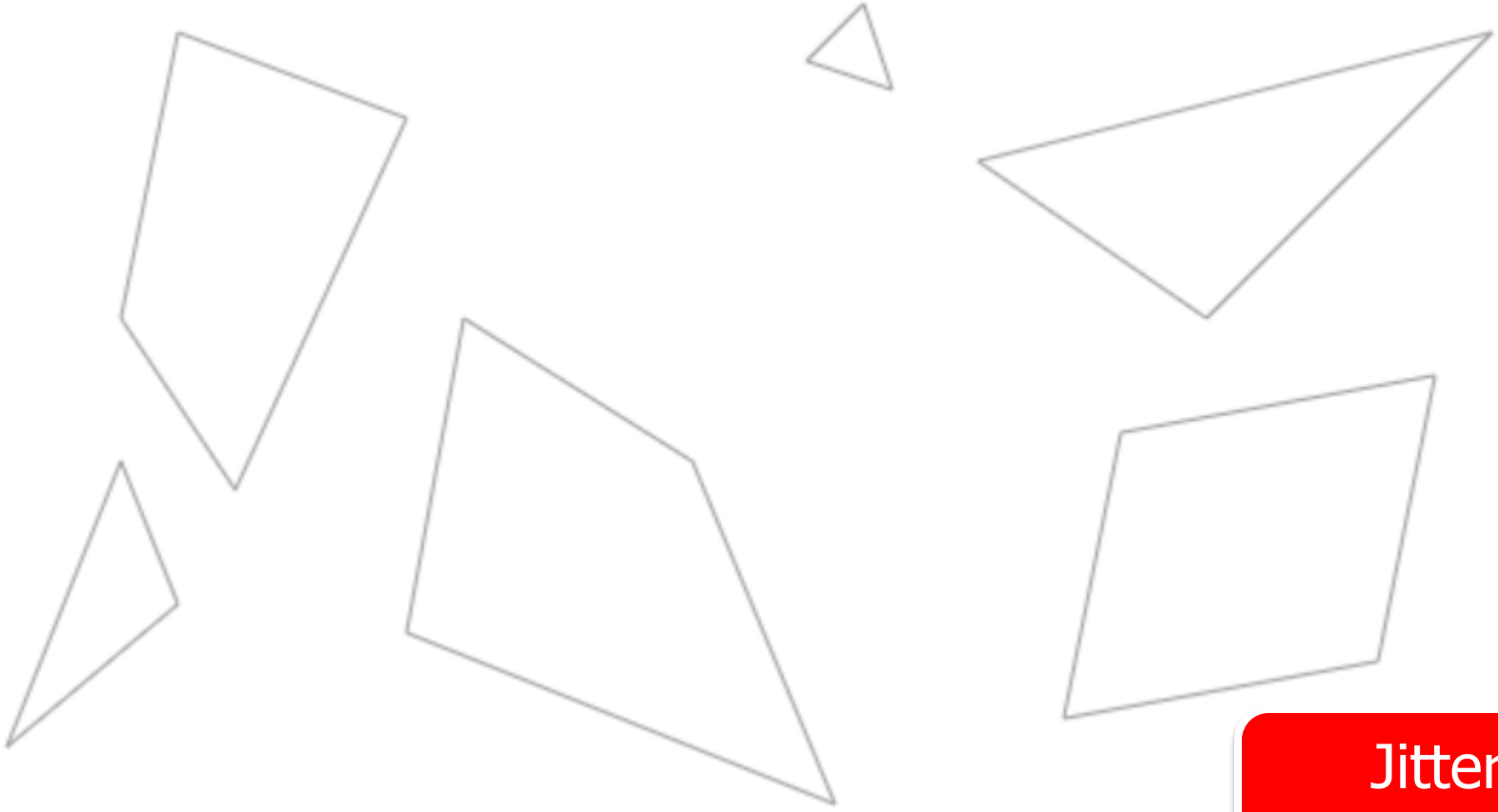

Jitter Cause


Jitter Cause

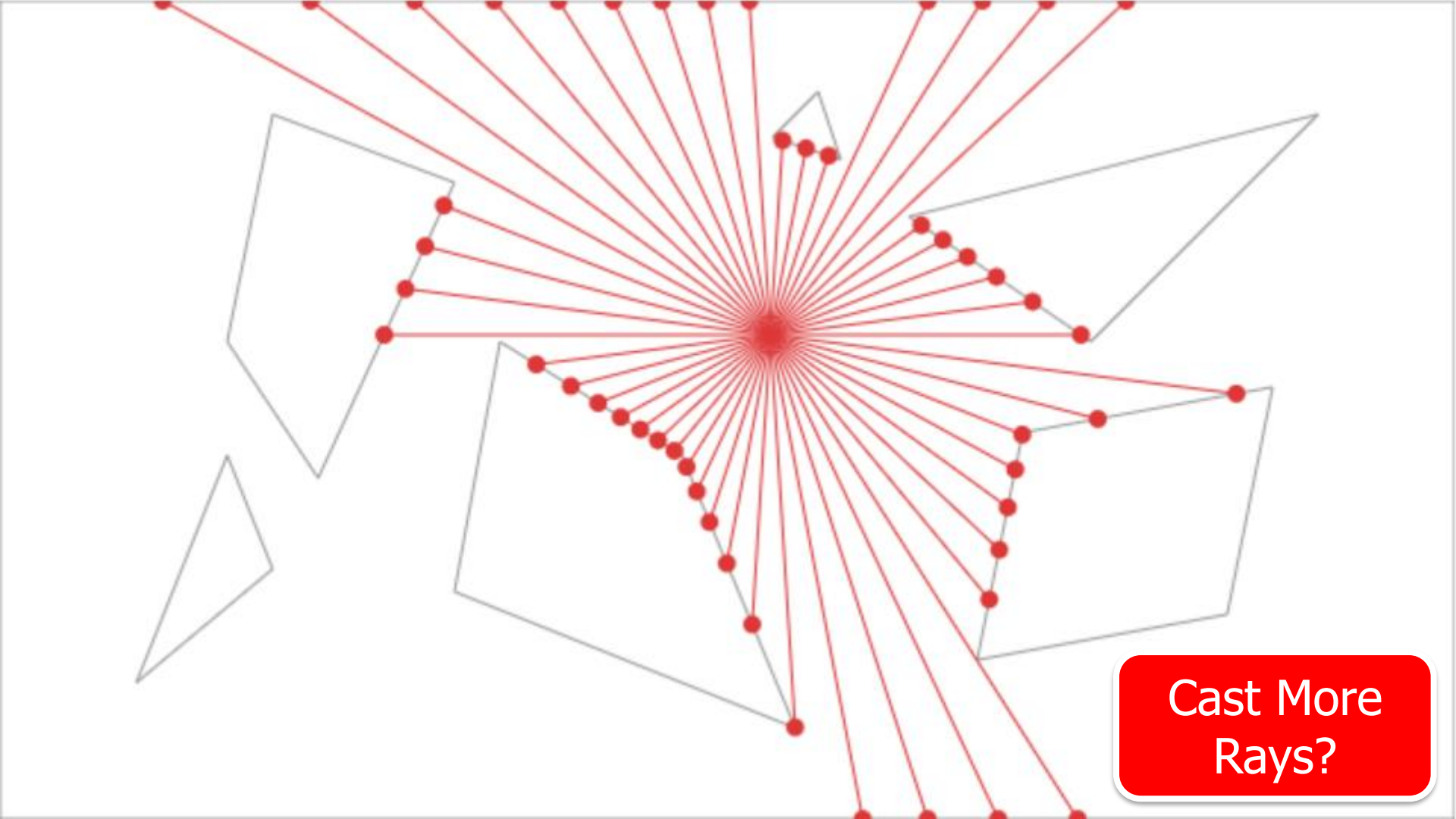

Jitter Cause

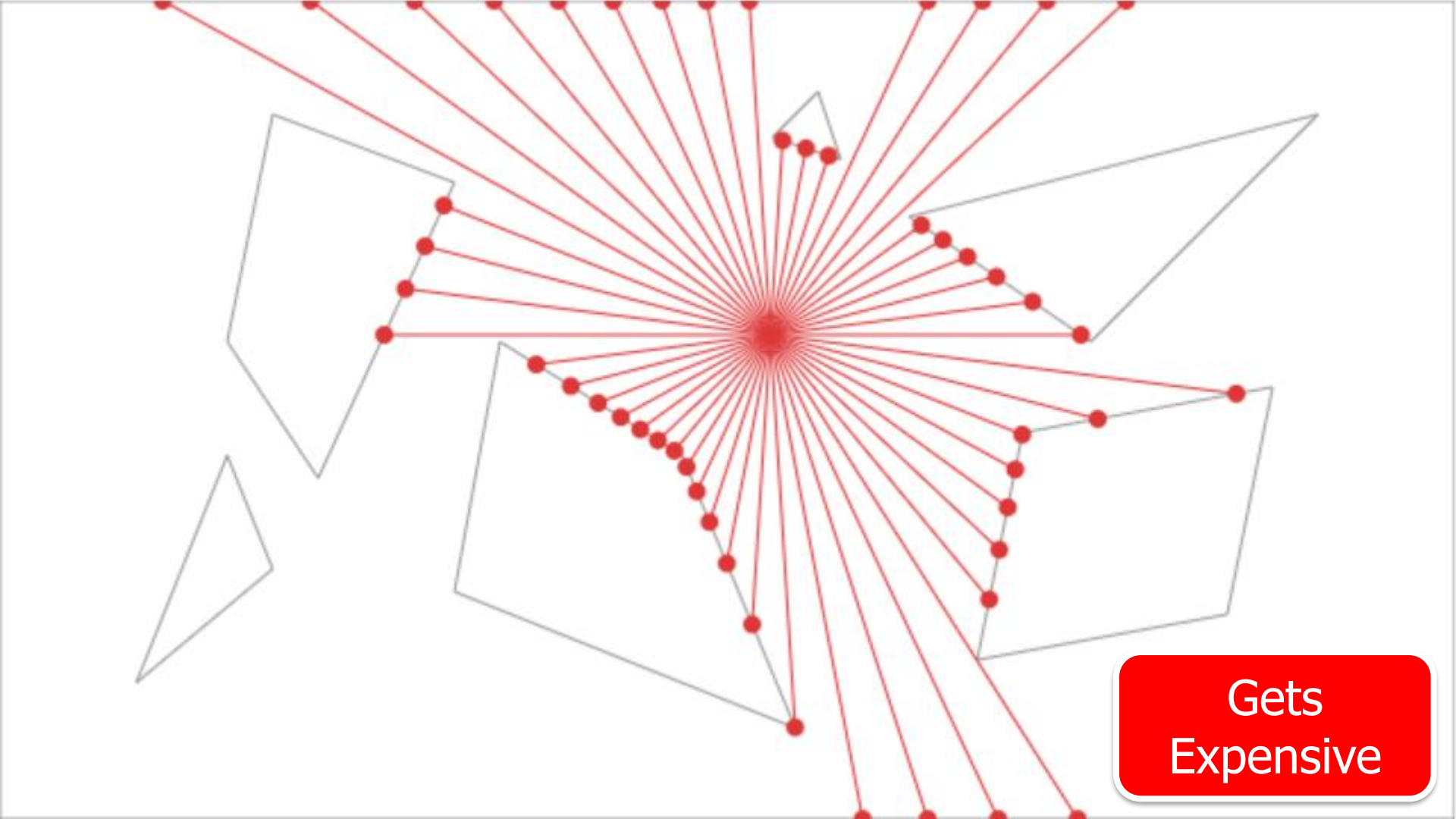

Jitter Cause

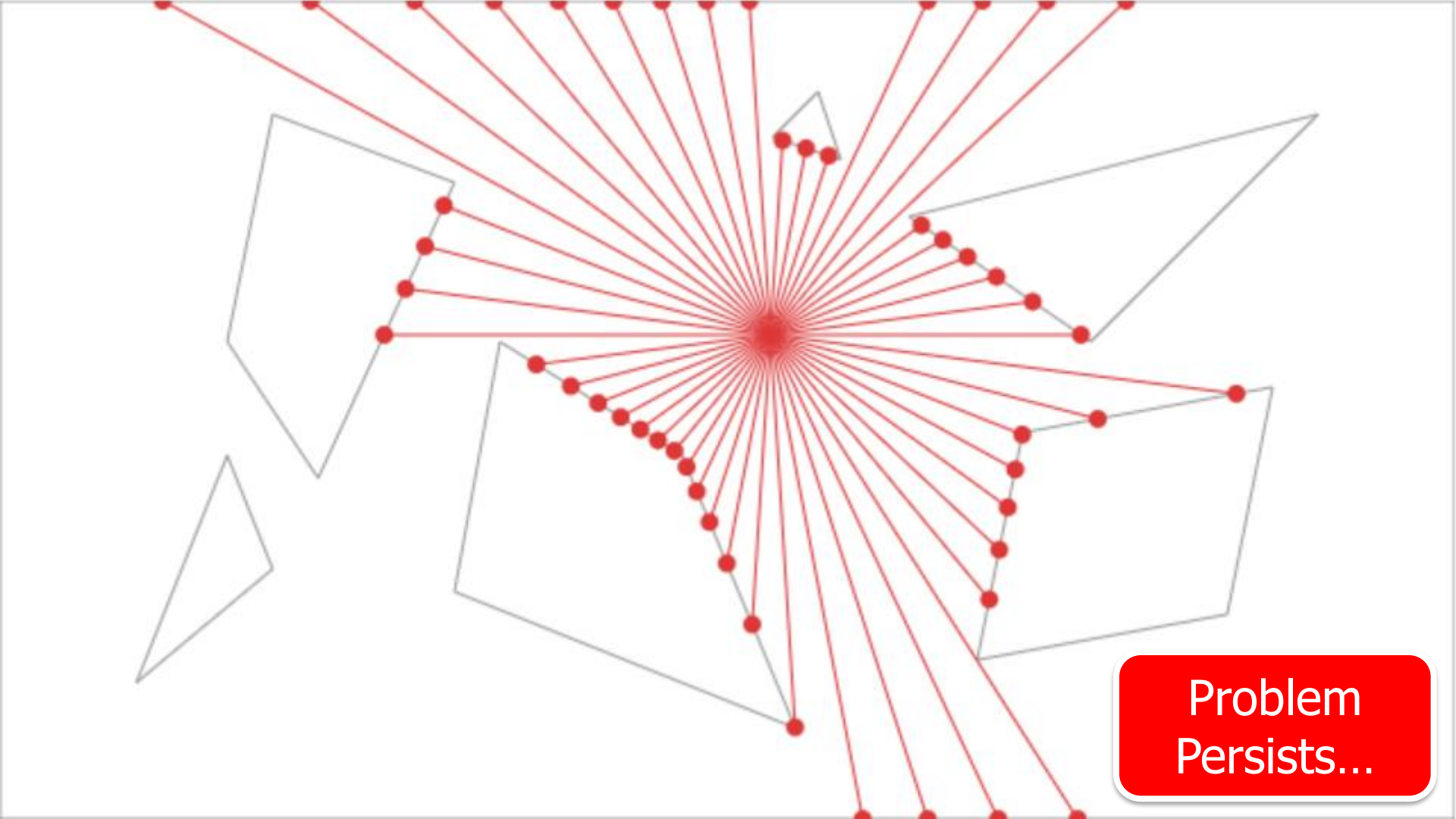

Jitter Cause

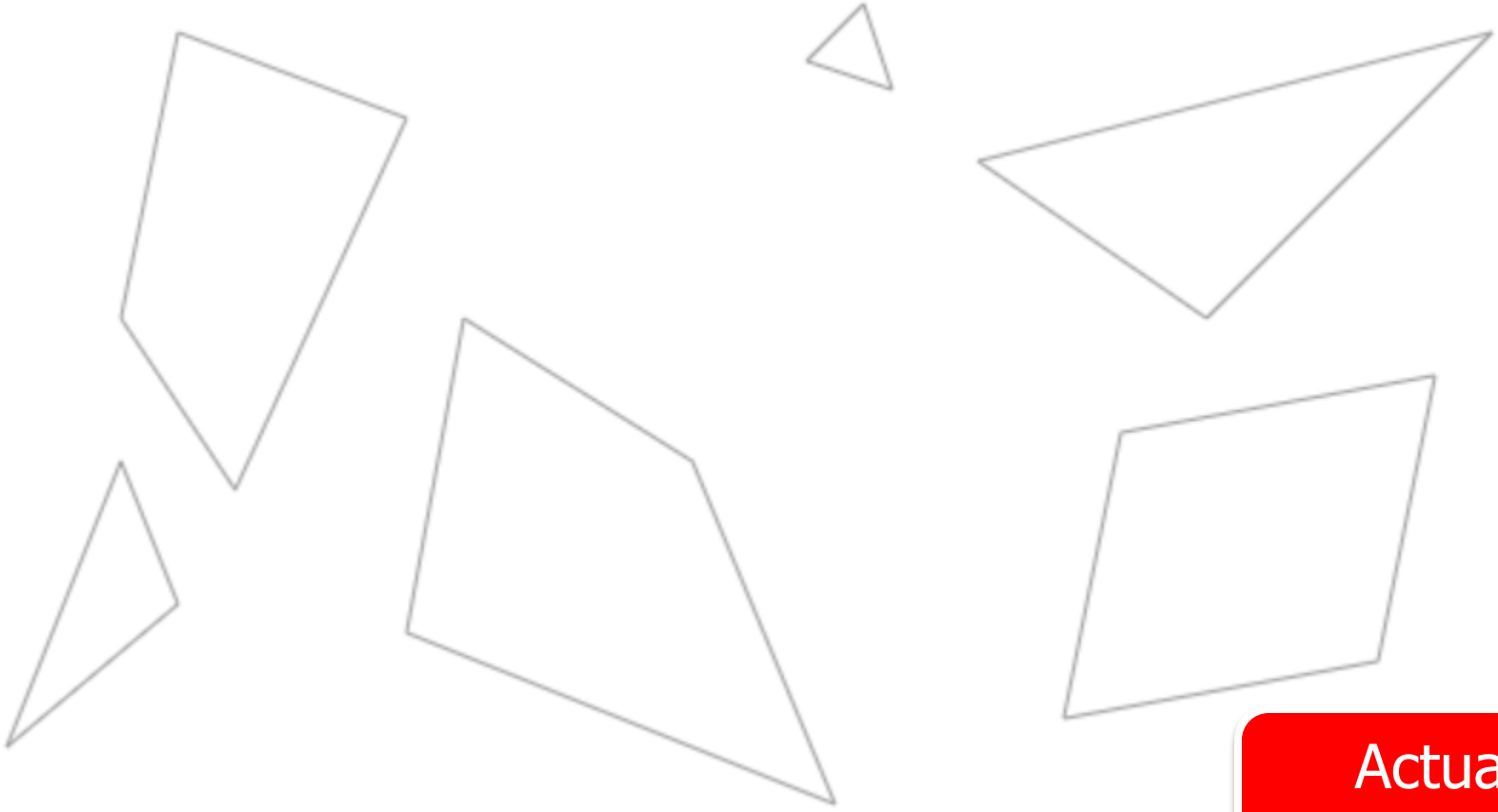


Jitter
Solution

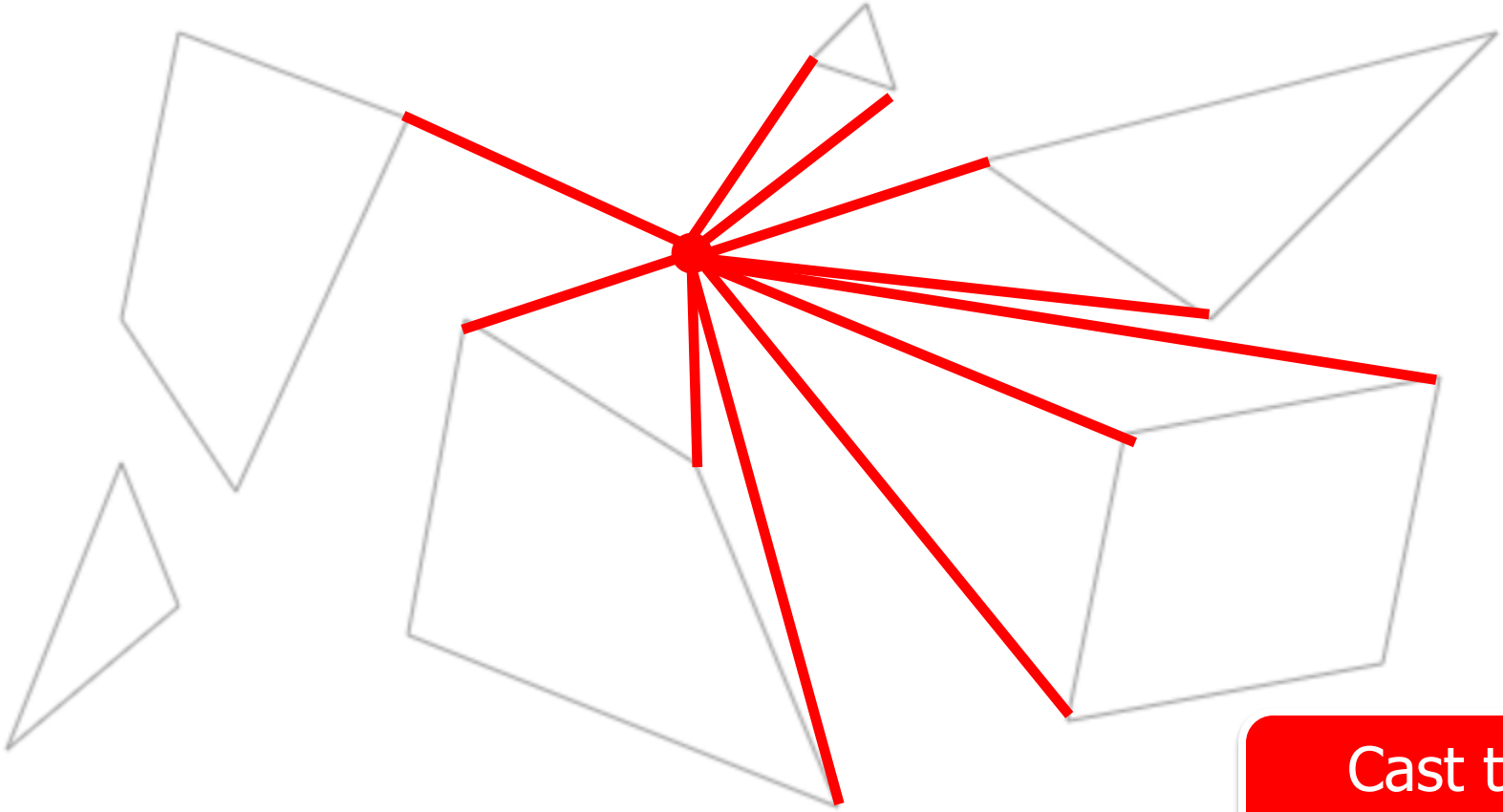

Cast More
Rays?

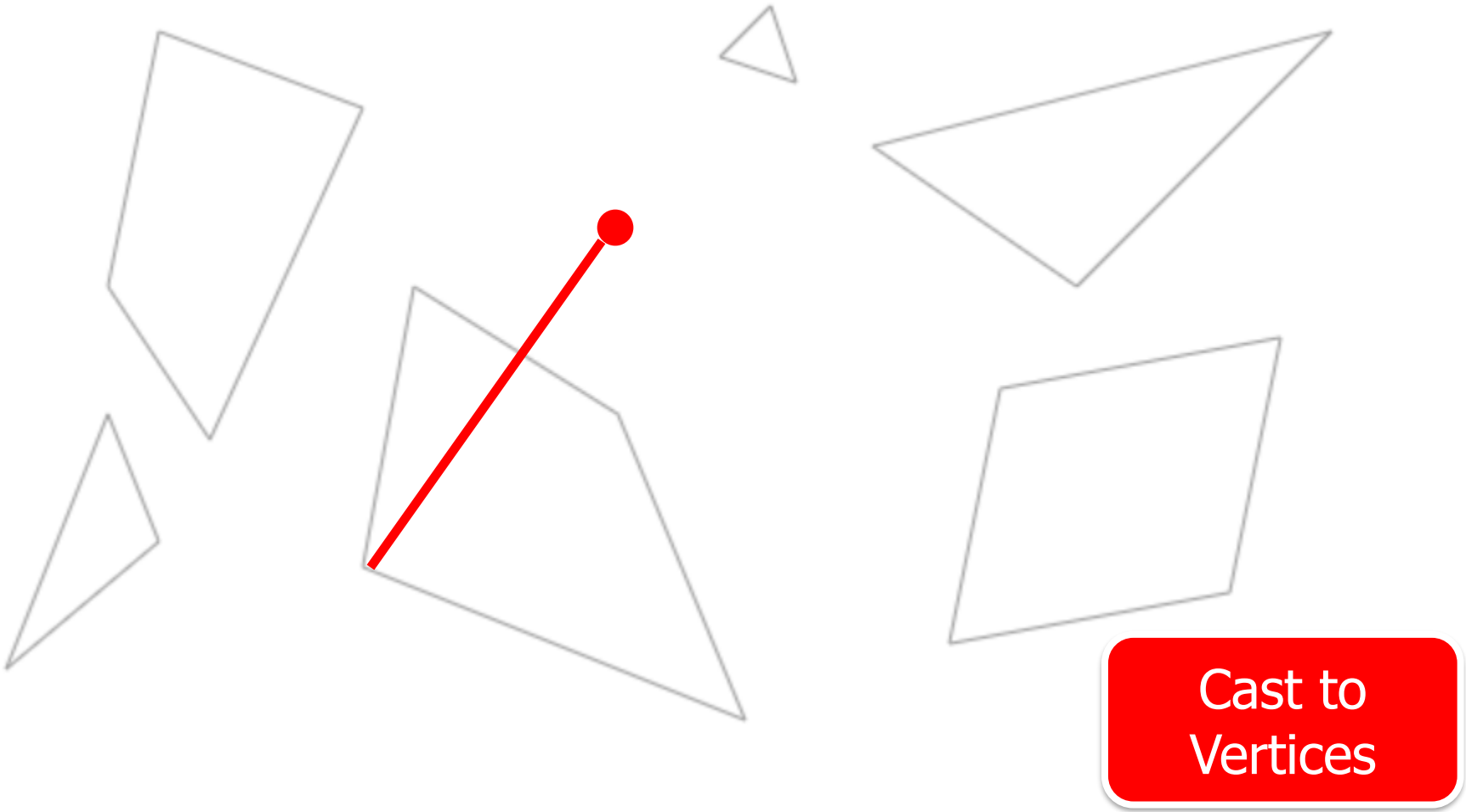

Gets
Expensive

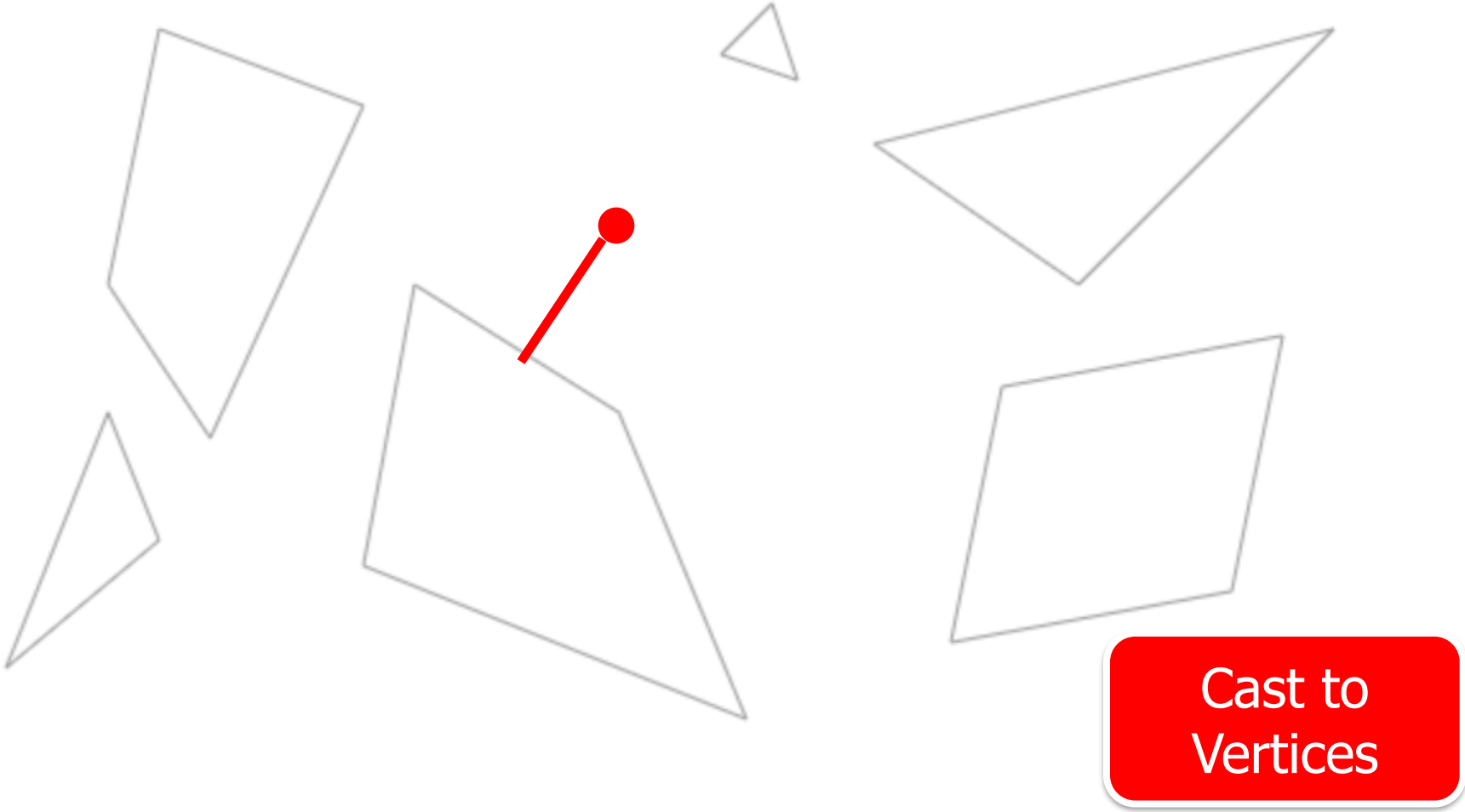

Problem
Persists…

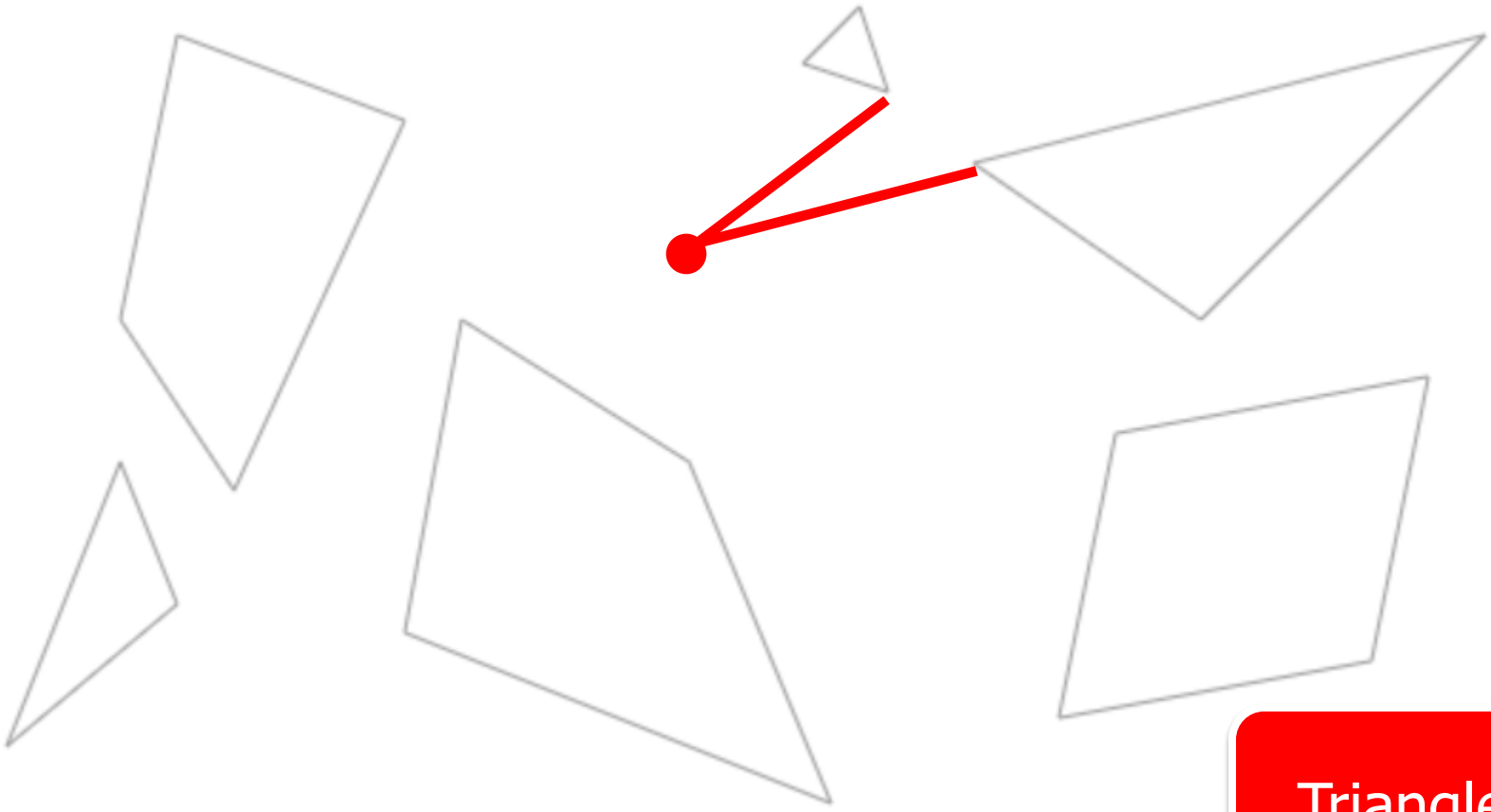


Actual Solution

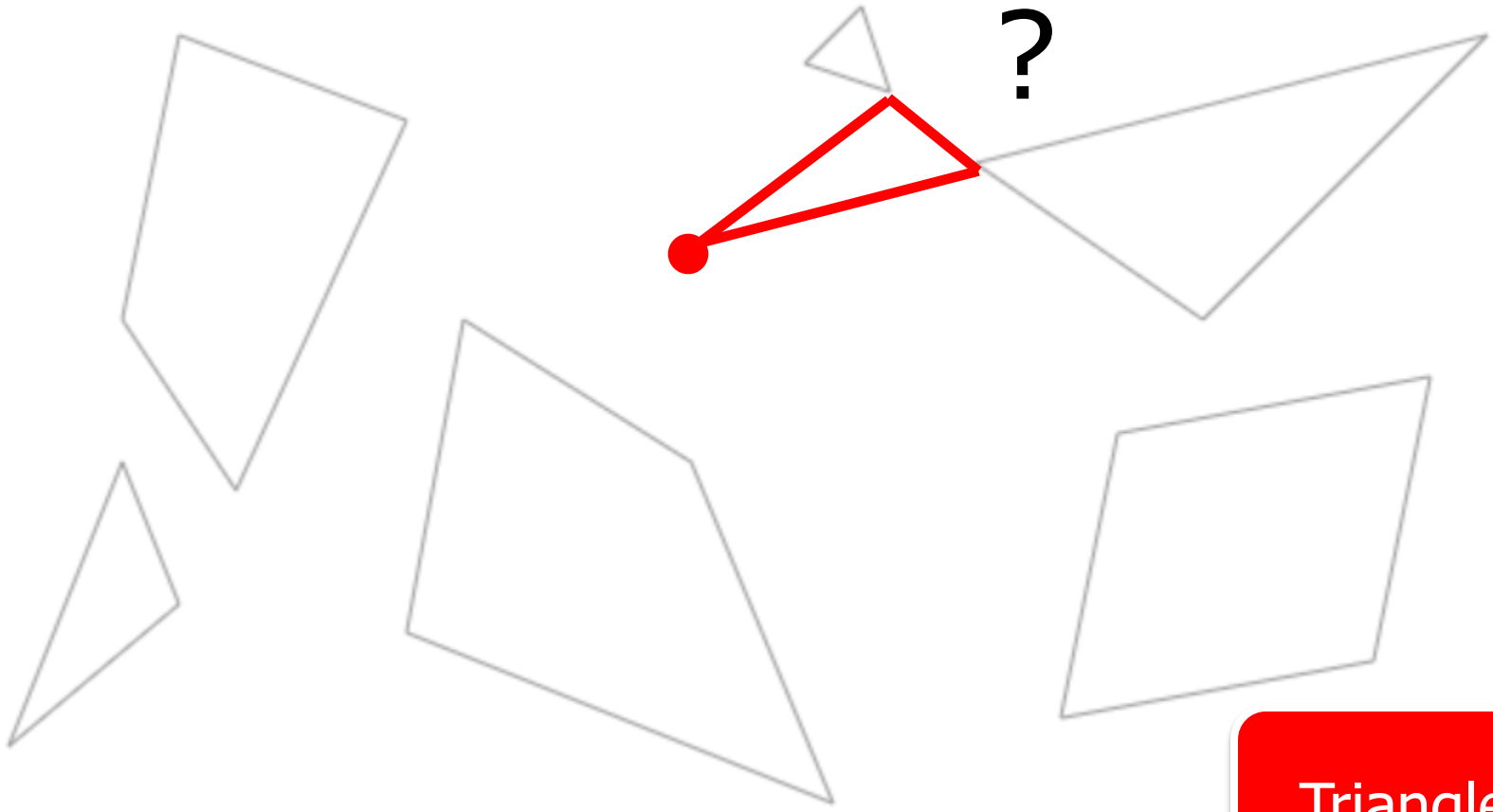

Cast to
Vertices


Cast to
Vertices

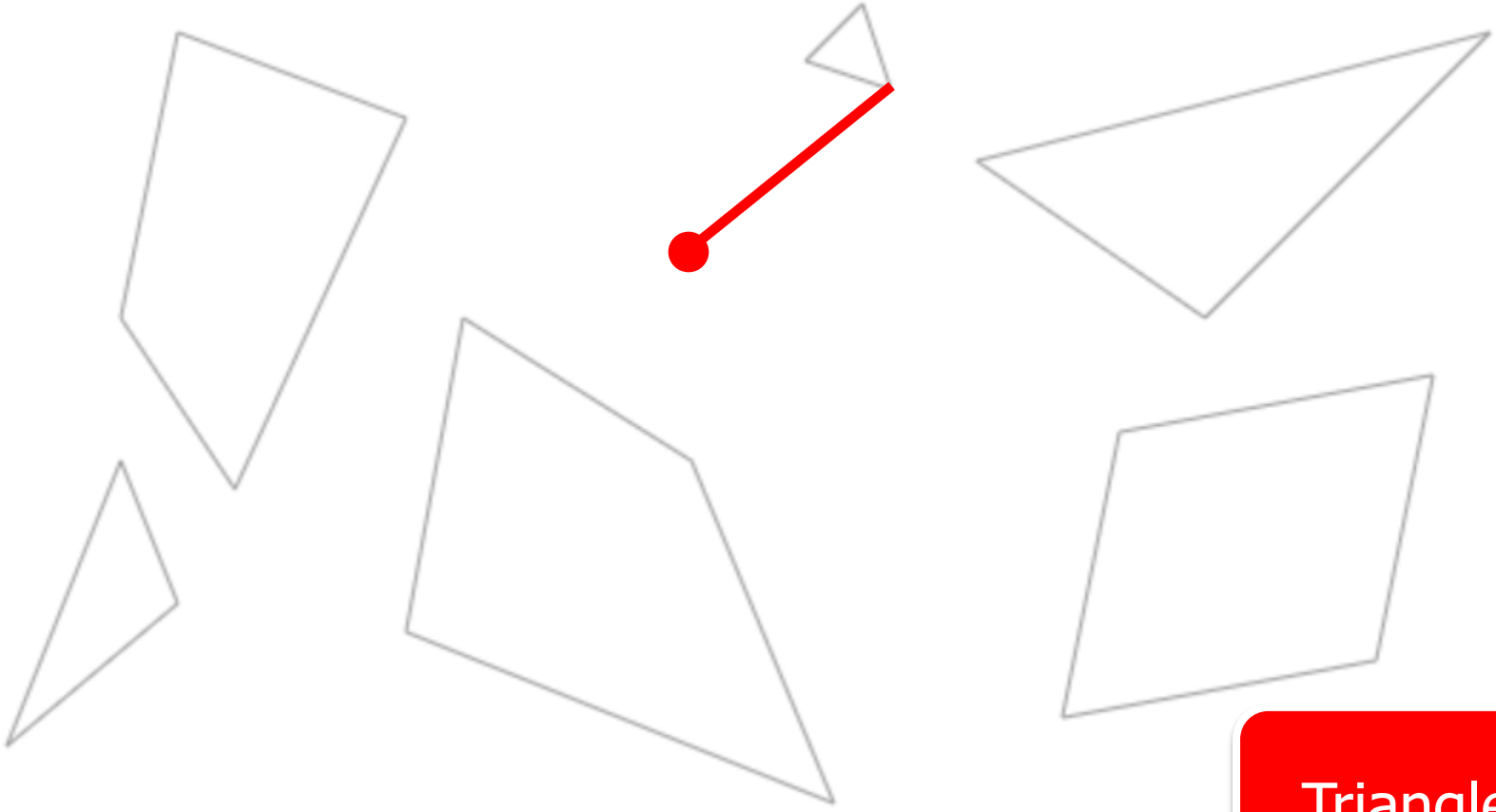

Cast to
Vertices



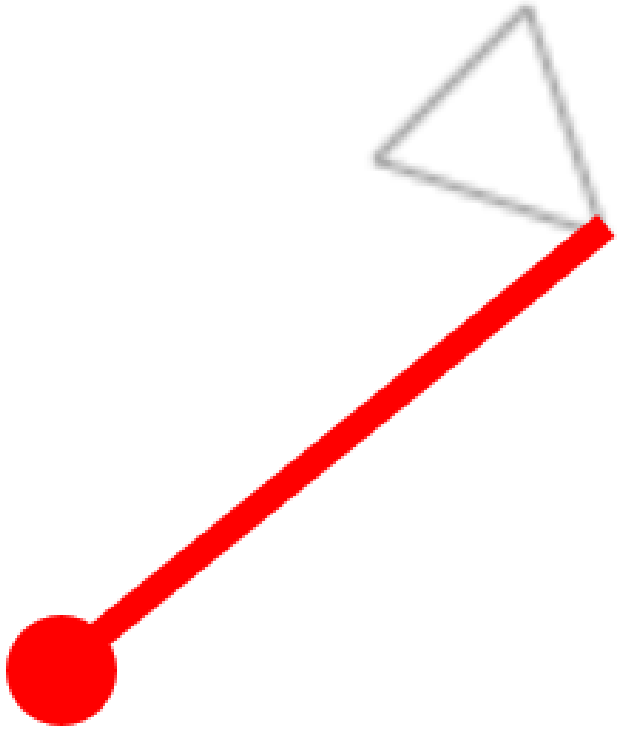
Triangles?



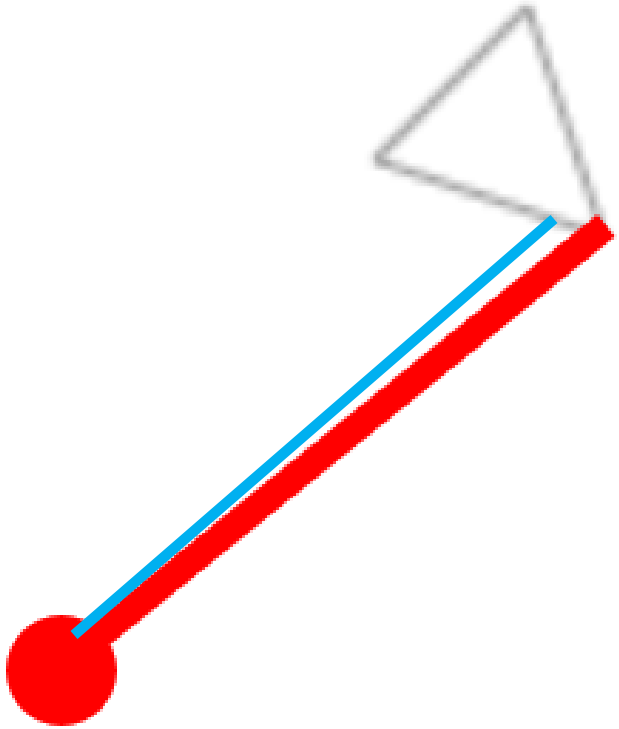
Triangles?



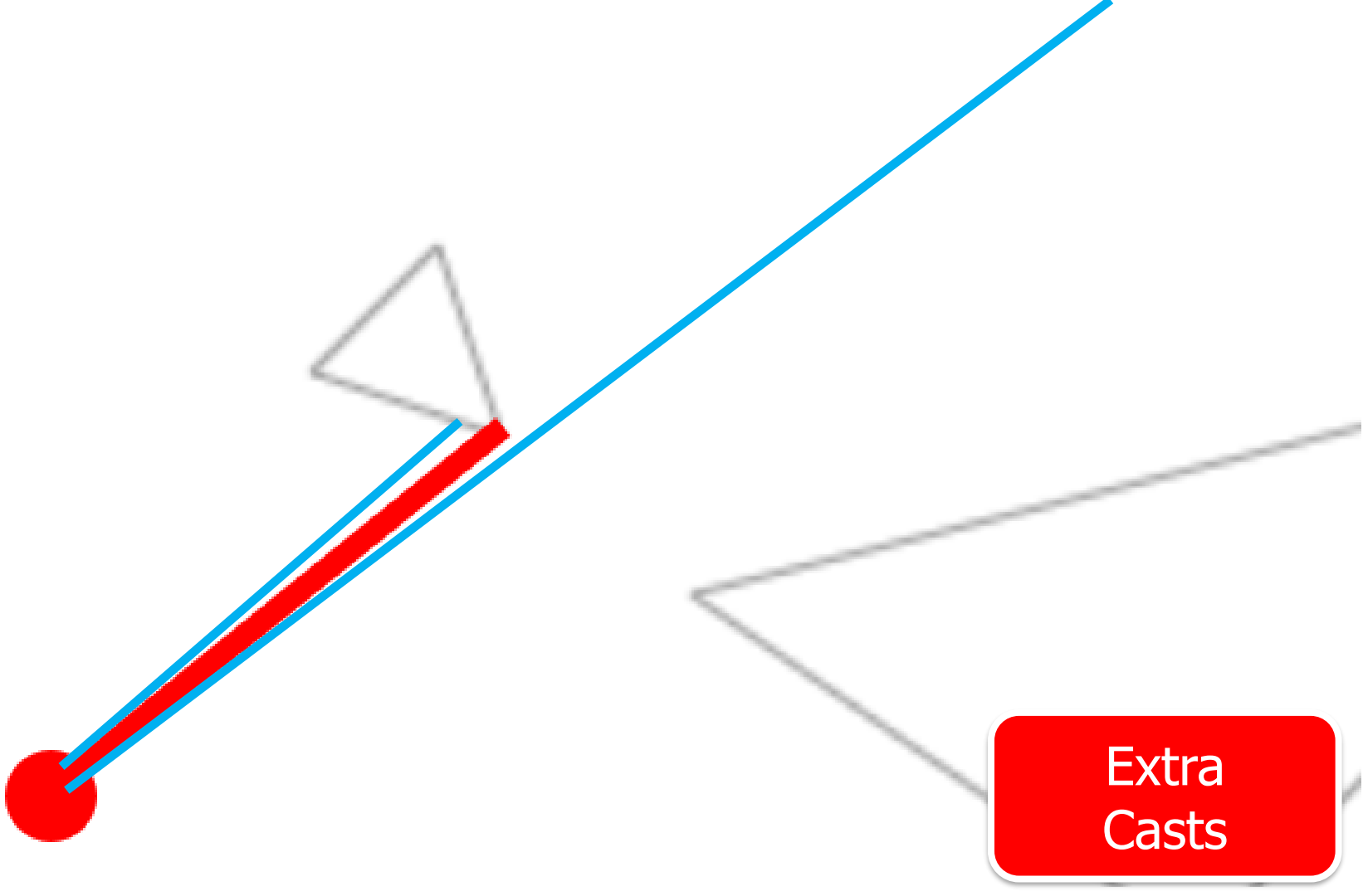
Triangles?

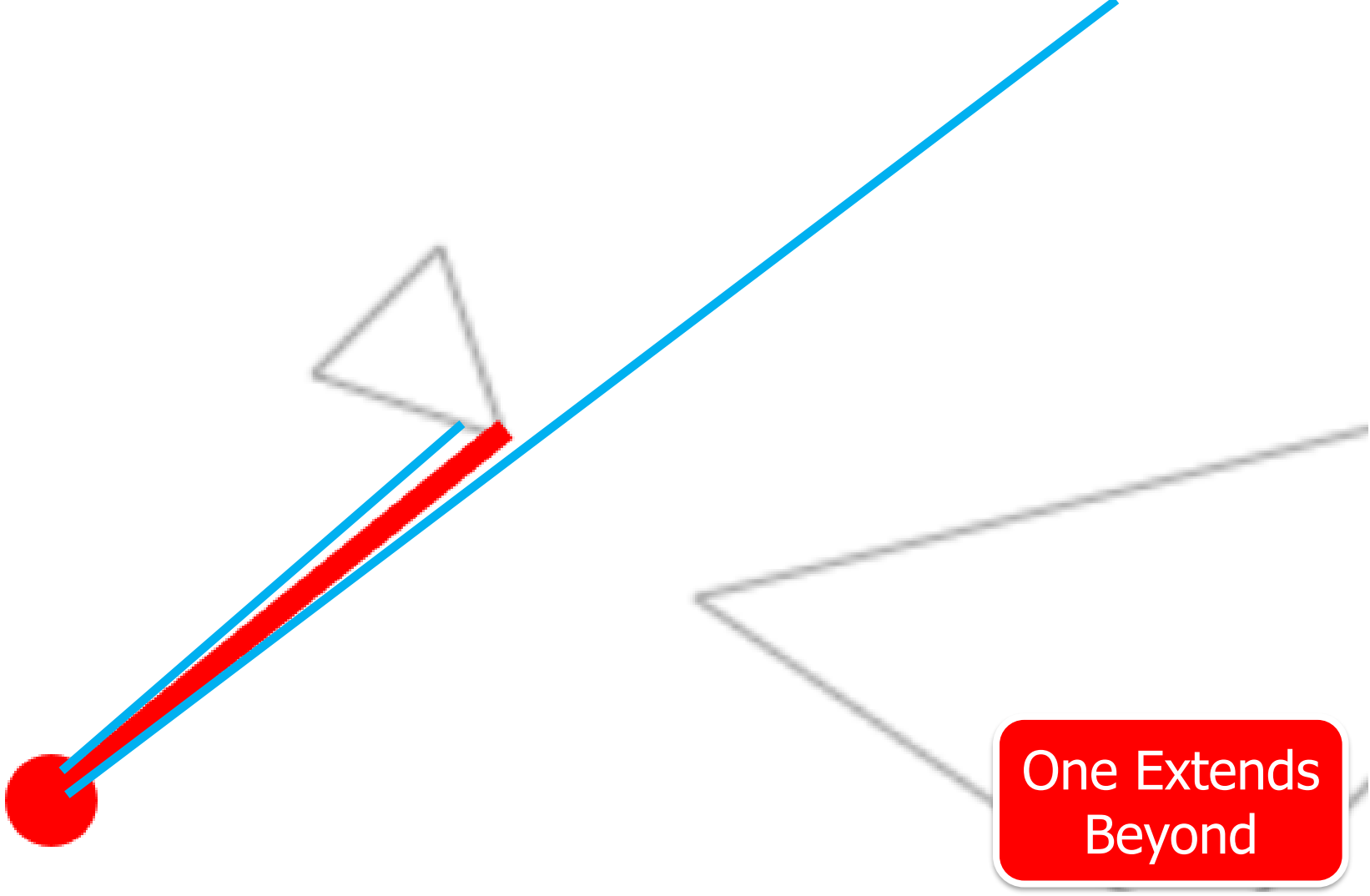


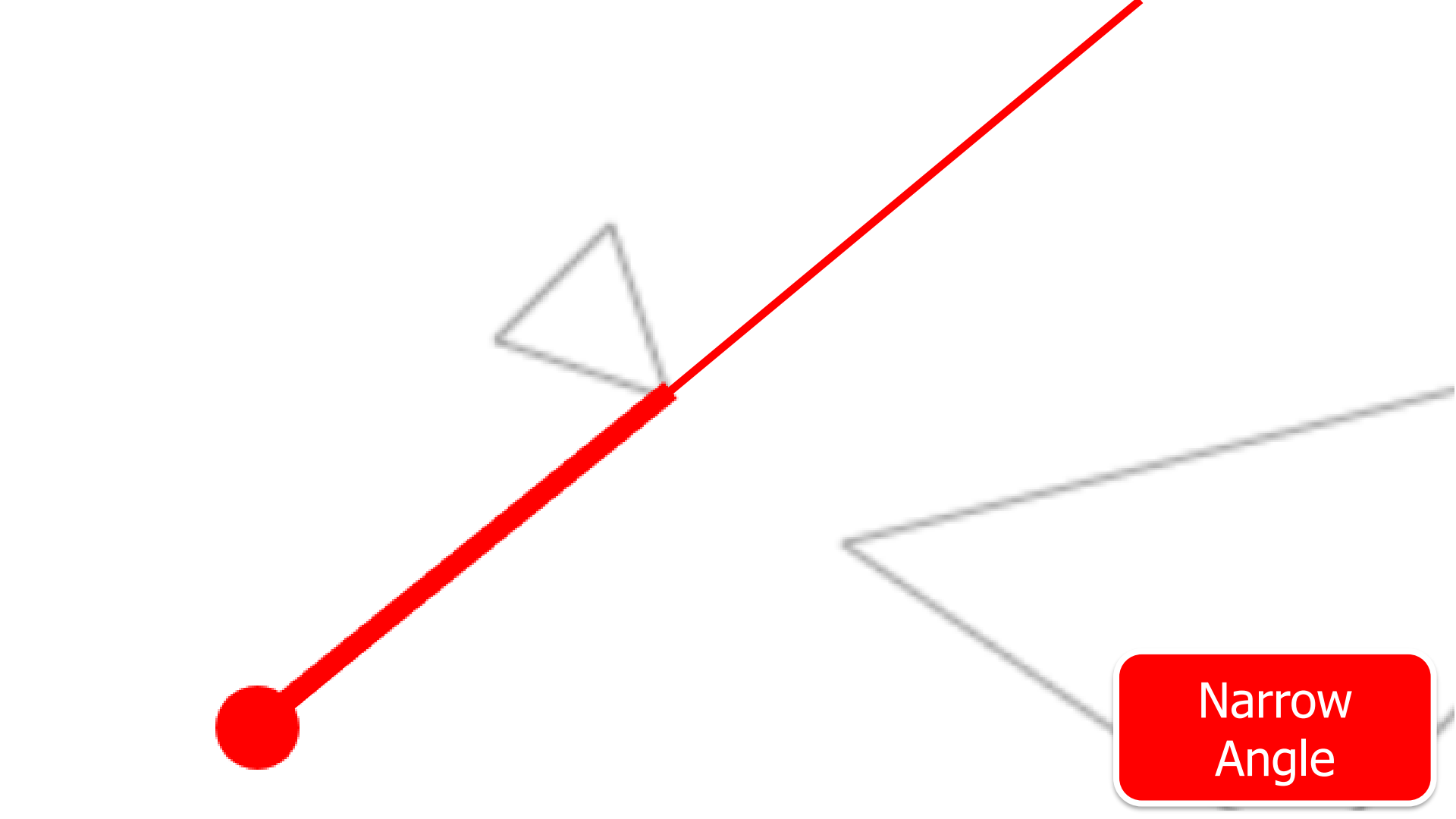
Extra
Casts

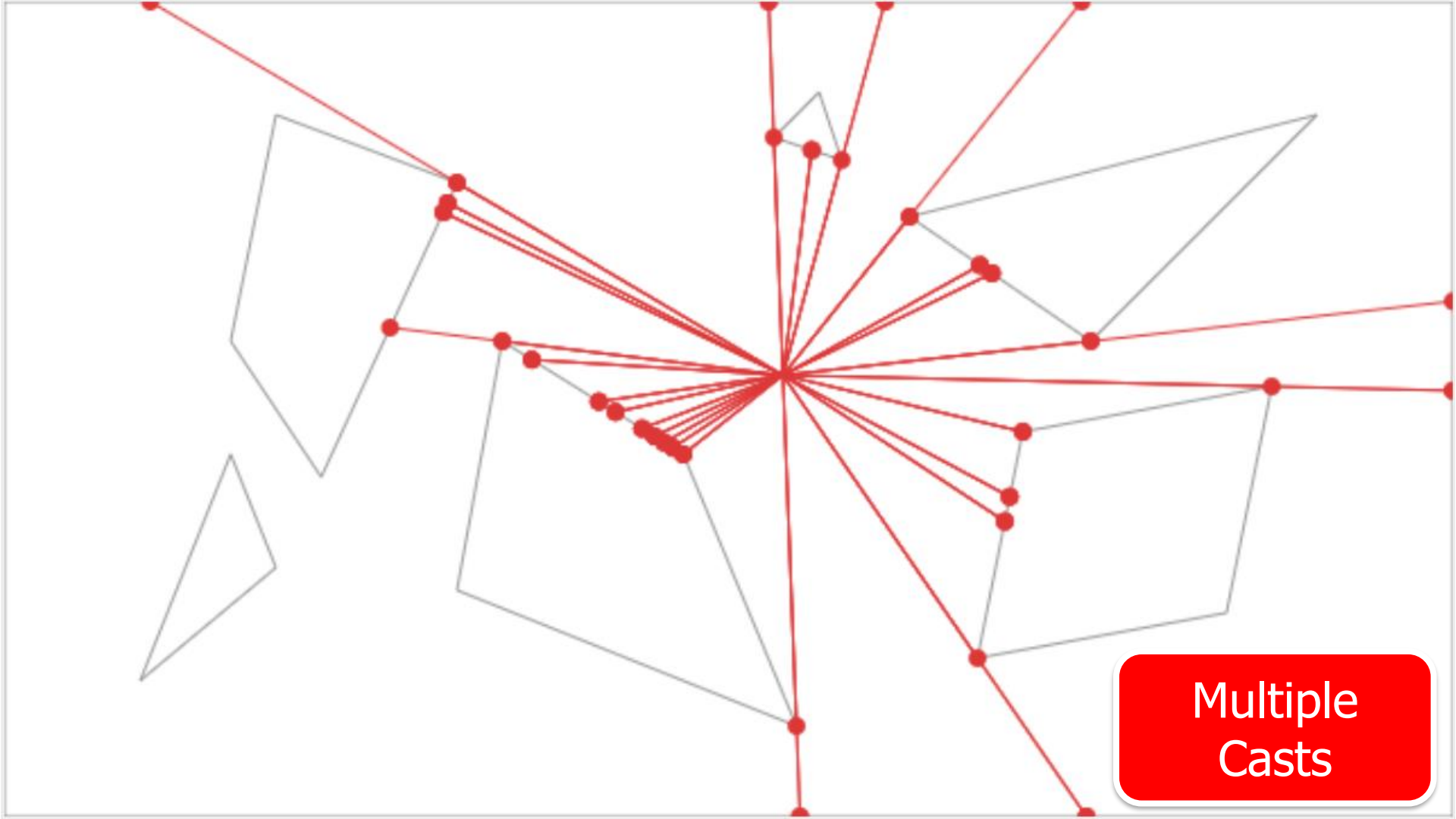


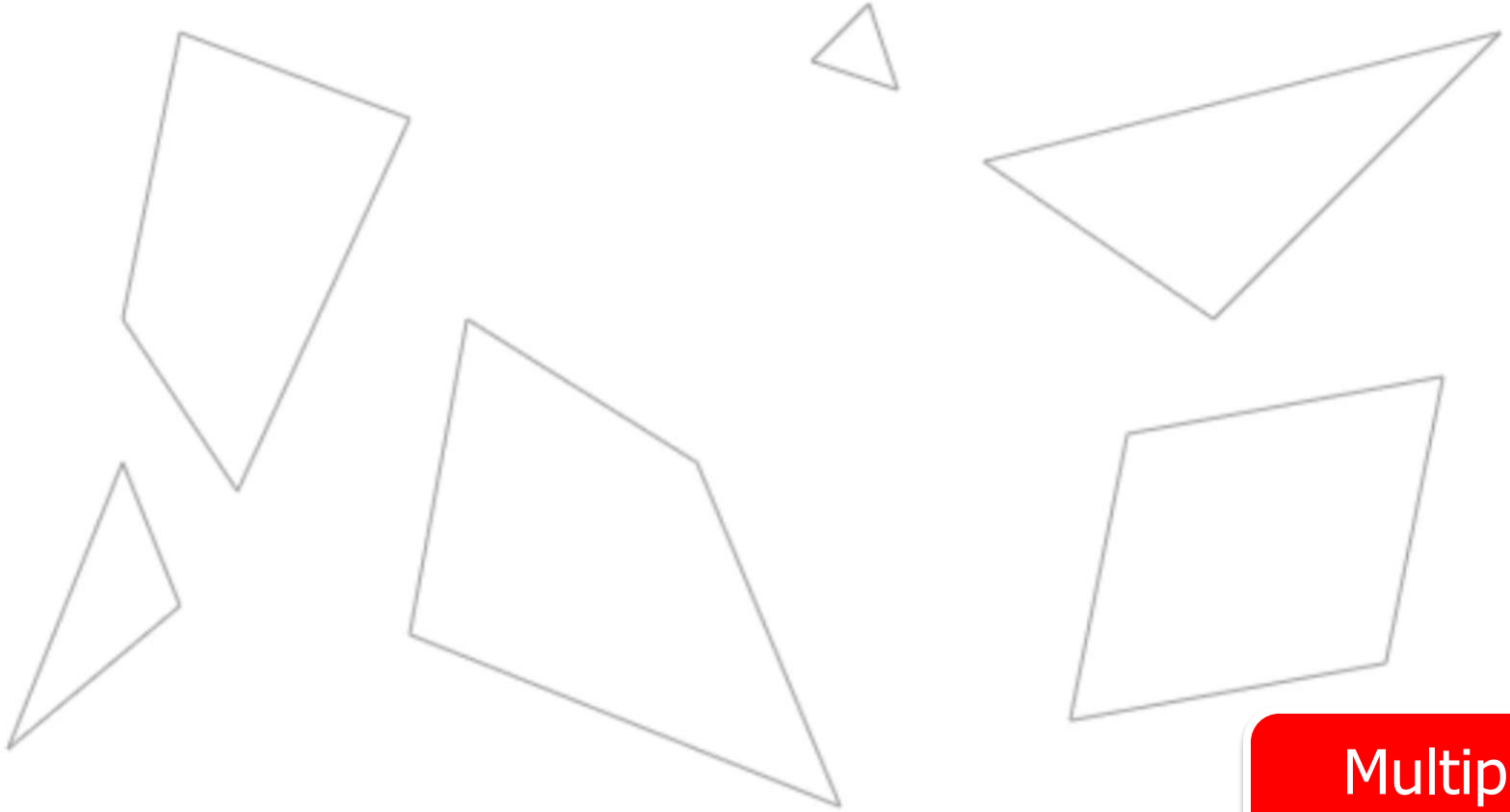
Extra
Casts


Extra
Casts


One Extends
Beyond


Narrow
Angle

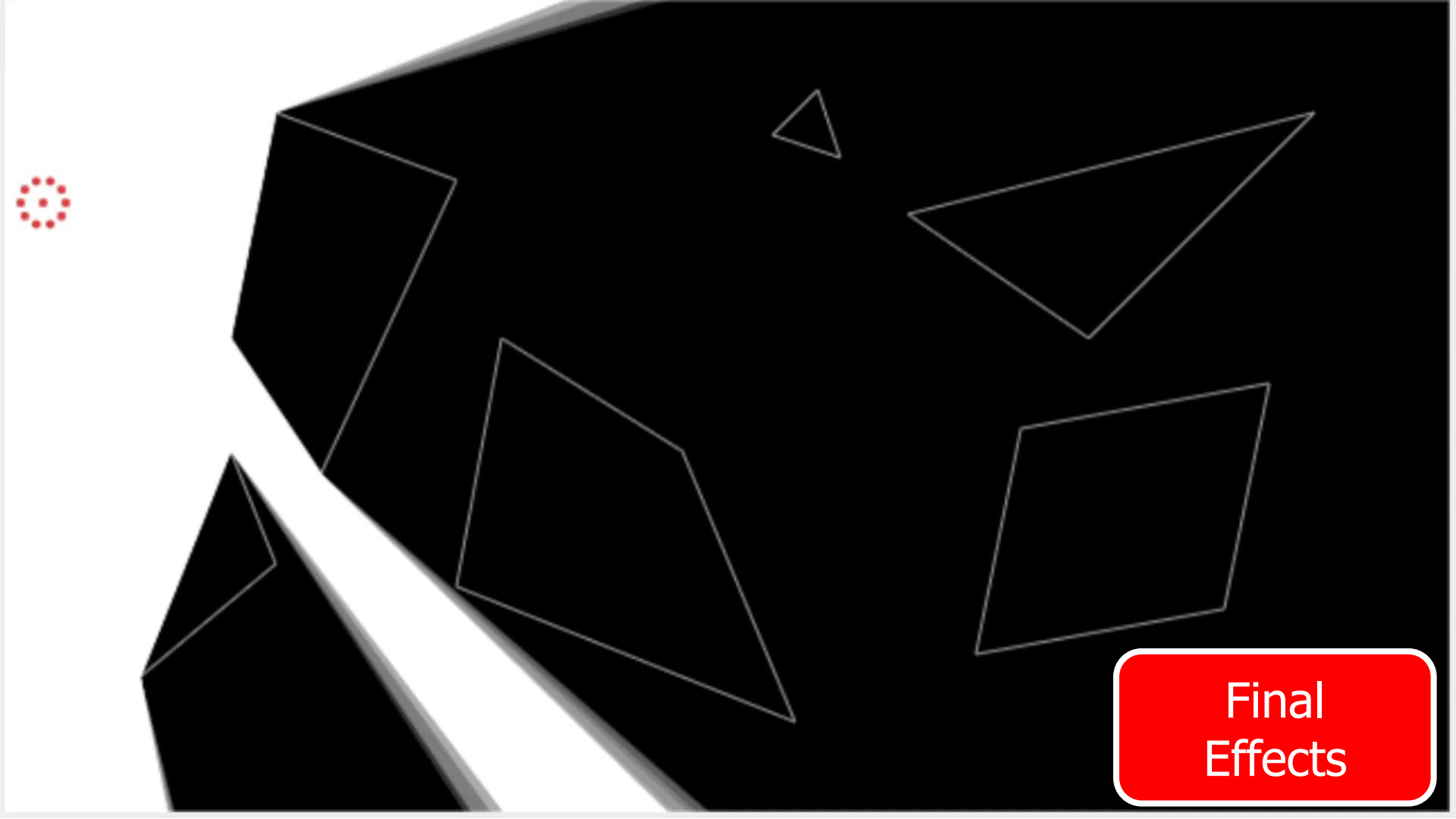

Multiple
Casts



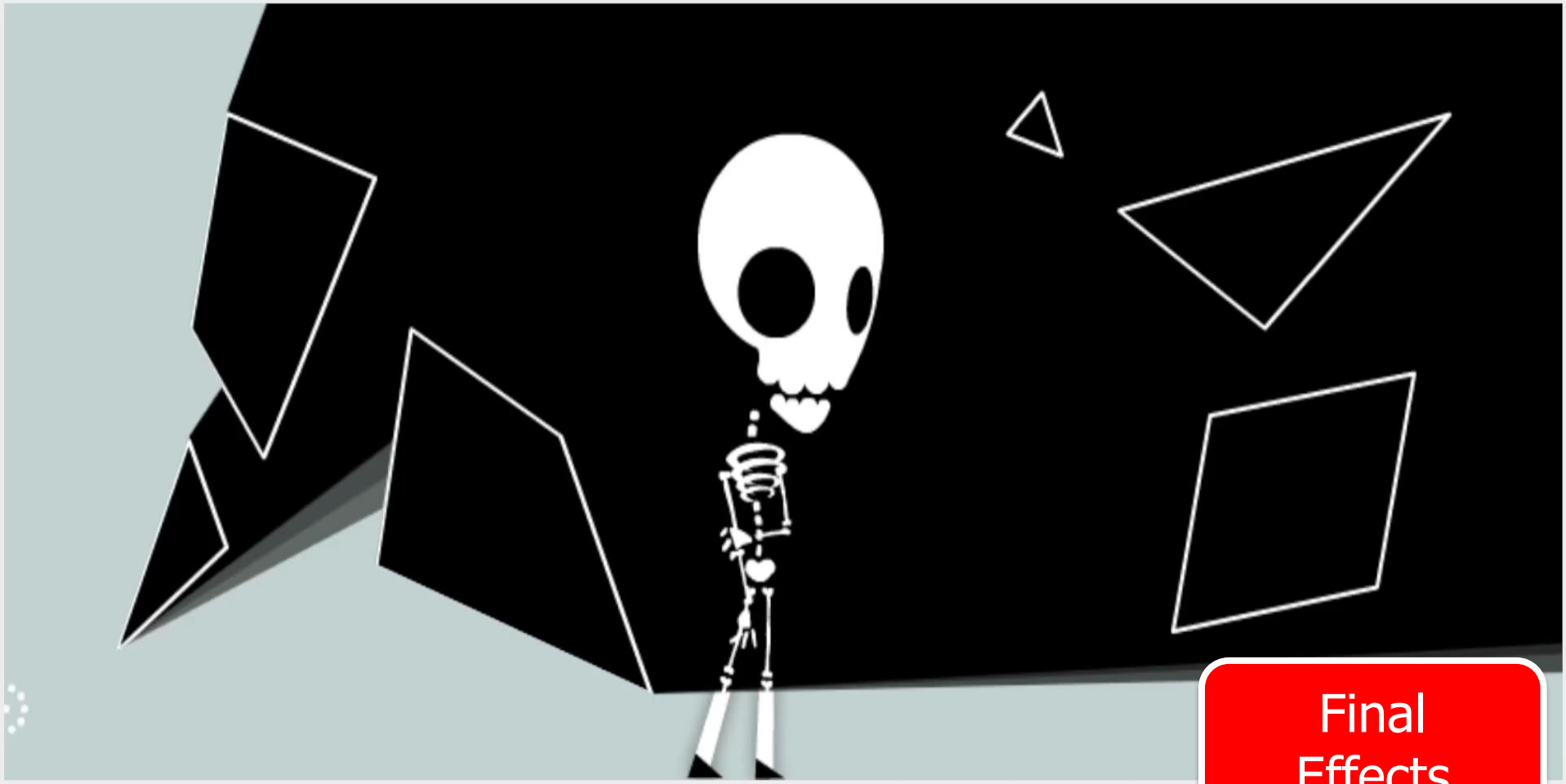
Multiple Casts

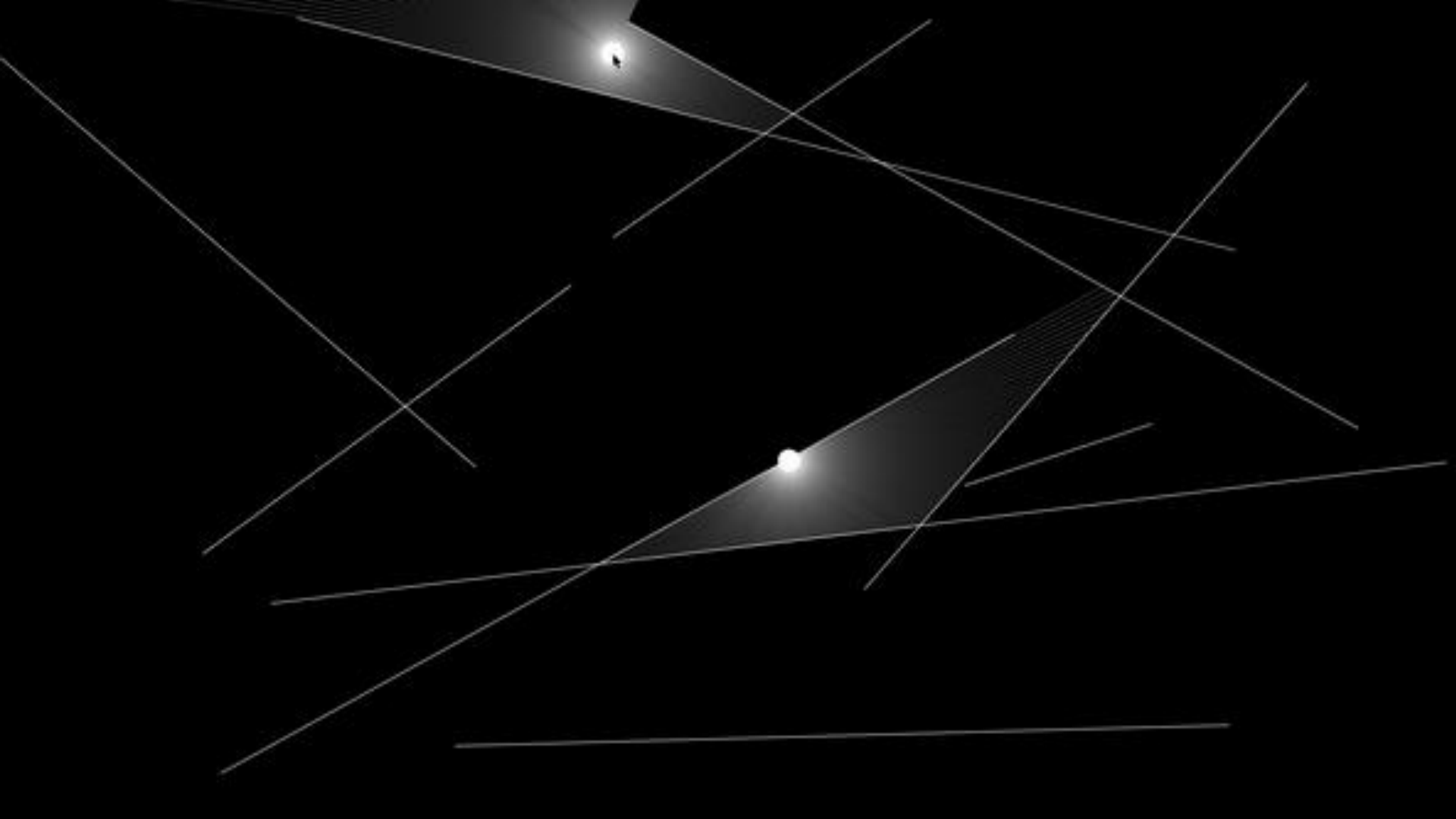

Final
Triangles

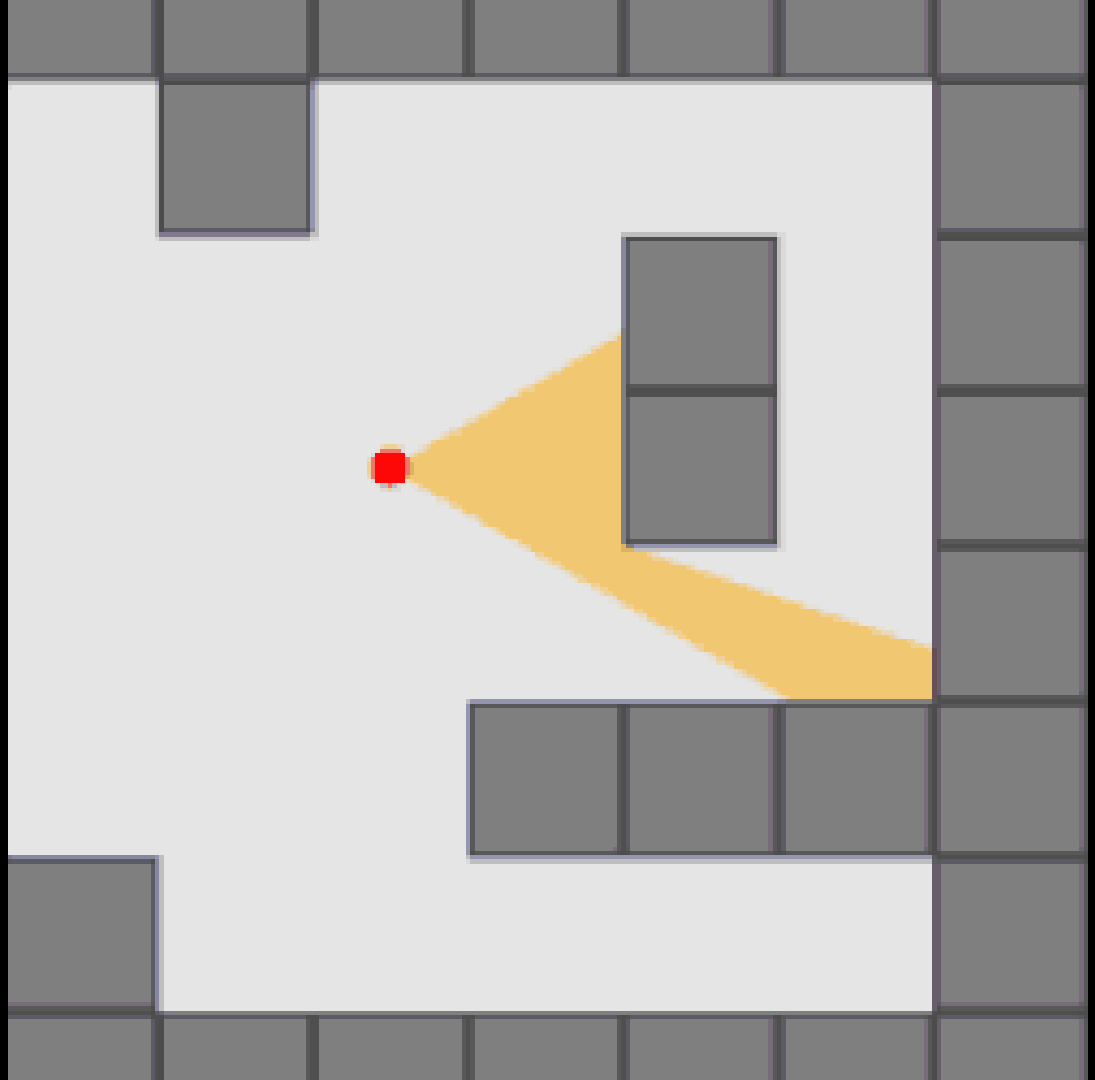

Final
Triangles


Final
Effects

Final
Effects


Final
Effects






Third Eye Crime – Moonshot Games